INDIRA GANDHI NATIONAL OPEN UNIVERSITY

BACHELOR OF COMPUTER APPLICATIONS

Synopsis

FARMERS DIRECT SELLING PORTAL

by

Name: ABHINANDHANA M

Enrollment No: 198292000

Under the guidance of: KEERTHANA BALAKRISHNAN

Submitted to the School of Computer and Information Sciences,

IGNOU

in partial fulfillment of the requirements for the award of the degree

Bachelor of Computer Applications (BCA)

Indira Gandhi National Open University

Maidan Garhi

New Delhi-110068

2021

TABLE OF CONTENTS

S.NO	CONTENTS
1	PROFORMA
2	GUIDE'S BIODATA
3	CERTIFICATE OF ORIGINALITY
4	CERTIFICATE OF COMPLETION
5	ACKNOWLEDGEMENT
6	CERTIFICATE
7	ORGANISATION PROFILE
8	ABSTRACT
9	PROJECT REPORT

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled **FARMERS DIRECT SELLING PORTAL** Submitted to Indira Gandhi Open National University in partial fulfillment of the requirement for the award of the degree of BACHELOR OF COMPUTER APPLICATIONS (BCA), is an original work carried out by Mr./Mrs. **ABHIANDHANA M** Enrolment No. **198292000** under the guidance of Mr./Mrs**.** **KEERTHANA BALAKRISHNAN .**

The matter embodied in this project is a genuine work done by the student and has not been submitted whether to this University or to any other University / Institute for the fulfillment of the requirement of any course of study.

Signature of the Student

Signature of the Guide

ABHINANDHANA.M

PADANNAYIL HOUSE

KUTTIKKAKA

P.O.EDAKKAD,KANNUR

670663,KERALA,INDIA

CERTIFICATE OF COMPLETION

This is to certify that Ms. ABHINANDHANA M, Enrolment No**:** 198292000 has successfully completed Project in BCSP-064 titled "**FARMERS DIRECT SELLING PORTAL**" under the guidance of Mrs. KEERTHANA BALAKRISHNAN

ABHINANDHANA M

Enrolment Number:198292000

Study Centre: SREE NARAYANA COLLEGE (1405)

Regional Centre: VATAKARA (83)

Place:KUTTIKKAKAM

Date:

KEERTHANABALAKRISHNAN

Lecturer in Computer Science

VINODALAYAM,

P.O. KUTHUPARAMBA,KANNUR

Place: THOKKILANGAD

Date:

ACKNOWLEDGEMENT

With proud and pleasure, I submit this project entitled "**FARMERS DIRECT SELLING PORTAL**". I am very much grateful to the almighty for giving me strength and ability to accomplish this task. I convey my hearty thanks to all the person who helped me in this attempt. I wish to express my heartfelt thanks to my project guide Mrs. **KEERTHANA BALAKRISHNAN** and **ALG INTERNATIONAL INSTITUTE OF TECHNOLOGY** for give me the chance to make this memorable project.

Sincerely

ABHINANDHANA M

ABSTRACT

Around many years, farmers in the India have had a little freedom in choosing markets and buyers for their product. Middleman is the one who handles the marketing. Presence of Too Many Intermediates/Middlemen results in the exploitation of both farmers and consumers with the middlemen offering lower prices to farmers and charging higher prices from the consumers.Many changes happened in the field of agriculture marketing of the country which are growing opportunities for the farmers to get benefit .

The application software is titled “farmers direct selling portal”. offers a wide range of operation such as online sales of fruits and vegetables. The fruits and vegetable images and details can directly uploads to application by registering and consumers can view and buy according to their wish. This leads farmers to get more profit. We can easily search and order product. It required less time for registration for farmers and searching of products. The system is user friendly and anyone having computer knowledge can handle it easily. Easy to use and less hardware requirements The main importance of this Project is to give the better rate to the farmer from the whole seller or from any user. Also the one most important thing in this project is that the farmer can sell the product by himself no need of the agent is required to sell the product. Also farmer can deliver the product by himself only so that he can get more profit rather than deliver by the agent.

SL. No.	TITLE	PAGE No.
1	INTRODUCTION	1-2
2	OBJECTIVES	3-5
3	TOOLS/ENVIRONMENT USED	6-7
4	SYSTEM ANALYSIS	8-38
5	SYSTEM DESIGN	39-48
6	PROGRAM CODE	49-118
7	TESTING	119-124
8	INPUT & OUTPUT SCREENS	125-159
9	FUTURE APPLICATION	160
10	LIMITATIONS OF THE PROJECT	160
11	REFERENCE	161

1

INTRODUCTION

INTRODUCTION

Direct marketing is the sale of agricultural goods and products from the farm straight to the consumer, without intervening distributors or retailers. Direct marketing can benefit farmers by allowing them to earn a greater portion of the consumer food dollar by eliminating intermediary brokers, buyers, and distributors. Examples of direct marketing include community supported agriculture (CSA) ventures, farmers' markets and farm stands, and direct sale to businesses and institutions, such as restaurants, schools or hospitals. Direct marketing can contribute toward sustainable agriculture and food systems by increasing farmer profitability, promoting the local economy, and providing consumers with higher quality and healthier products.

Prior to the widespread availability of refrigeration, many farmers were responsible for their own marketing. Sales were generally made directly to retail stores, or to customers at farm stands and street markets. The development of refrigeration, efficient transportation, and large supermarkets in the 1940's caused many farmers to shift away from the responsibility of marketing and selling their own products. Farmers began developing contracts with wholesale buyers so they could focus on increasing the scale of production. Farmers also began to specialize in just one or a limited number of products so as to be able to supply enough quantity for wholesale level demand. The wholesale contract model generally involves an intermediary customer who re-sells or distributes produce to retail outlets or other wholesale markets. This introduces an additional transaction point before the product reaches an end consumer.

The project entitled farmers direct marketing portal is a web application to reduce the gap between farmers and end customers.

2

OBJECTIVES

OBJECTIVES

The main objective of the development of farmers direct marketing web application is to reduce the gap between farmers and customers. Also make sure that the products will reaches at consumers with higher quality and healthy.

- Make user-friendly application
- Easy platform for farmers and consumers

Primary objective

The primary objective of this project to help farmers to sell their product directly without a middleman .By directly selling their product famers can get their product at the right price they decide. The customers can also get fresh product without delay. The customer and farmers can contact directly. So the process of selling and buying will become more effortless.

Secondary objective

This project helps the farmers to sell their agricultural product to business and institutions such as restaurants, schools or hospitals without a middle man. Here a person can trade agricultural products using one account. Here the farmer who sells product can decide the agricultural product prize .So that a person can use the account conveniently. Through this web application farmers and customers can have the advantage of getting more money and fresh product within less time. The farmers and customers can communicate directly. The trading can also be done between two farmers. Only with the approval of the admin the farmer can trade the product without any error.

Purpose

FARMERS DIRECT SELLING PORTAL is a web application where a farmer can sell crops from the farm directly to customers. Farm direct marketing is a long felt need of the farmers and consumers of the country as it goes a long way in ensuring higher remuneration to the farmers and meeting the satisfaction level of the consumers through direct sale of the agricultural commodity by the farmers to the consumer at affordable prices. The farmer sets the price or is more control of the price. Good products and services can get attractive prices and therefore, small farms can be profitable. direct marketing the agricultural products allows better potential profit margins compared to selling wholesale.

Scope

Direct marketing can benefit farmers by allowing them to earn a greater portion of the consumer food dollar by eliminating intermediary brokers, buyers, and distributors so this web application will help them Direct marketing can contribute toward sustainable agriculture and food systems by increasing farmer profitability, promoting the local economy, and providing consumers with higher quality and healthier products.

Since the web application is available in the smartphone it is easily accessible and always available. It is easy to extend the system that we have proposed. In future we can implement some features like online payment also there is a scope for an android/IOS application.

3

TOOLS/ENVIRONMENT USED

SYSTEM REQUIREMENTS

HARDWARE:-

- Processor : Pentium III
- Speed : 1.1 Ghz
- Hard Disk : 20 GB
- RAM : 4 GB
- Monitor : 15 VGA Color
- Keyboard : Standard Windows Keyboard

SOFTWARE:-

- Operating System : Windows 7 and above
- Front End : Python
- Database : Firebase
- Tools : Pycharm or python IDE

4

SYSTEM ANALYSIS

System analysis is the process of gathering and interpreting facts, diagnosing problems the facts to improve the system. System analysis deals with a detailed study of the various performed by the system and their relationship with in and outside of the system. System analysis is the heart of the process. Analysis helps us to understand the present system. System specifies what system should do. A system is a set of components that interact to accomplish purpose.

1.SOFTWARE REQUIREMENT SPECIFICATION

In this field all the works are done manually. The main problem of manual entry is, it is time consuming and error prone than the computerized system. There exists wastage of the data retrieval time as in the case of searching and further processing. There was no efficient system to manipulate the existing records

INTRODUCTION:-

The project entitled **FARMERS DIRECT MARKETING PORTAL** is a web application to reduce the gap between farmers and end customers.

Direct marketing is the sale of agricultural goods and products from the farm straight to the consumer, without intervening distributors or retailers. Direct marketing can benefit farmers by allowing them to earn a greater portion of the consumer food dollar by eliminating intermediary brokers, buyers, and distributors. Examples of direct marketing include

community supported agriculture (CSA) ventures, farmers' markets and farm stands, and direct sale to businesses and institutions, such as restaurants, schools or hospitals. Direct marketing can contribute toward sustainable agriculture and food systems by increasing farmer profitability, promoting the local economy, and providing consumers with higher quality and healthier products

PURPOSE

FARMERS DIRECT SELLING PORTAL is a web application where a farmer can sell crops from the farm directly to customers. Farm direct marketing is a long felt need of the farmers and consumers of the country as it goes a long way in ensuring higher remuneration to the farmers and meeting the satisfaction level of the consumers through direct sale of the agricultural commodity by the farmers to the consumer at affordable prices. The farmer sets the price or is more control of the price. Good products and services can get attractive prices and therefore, small farms can be profitable. direct marketing the agricultural products allows better potential profit margins compared to selling wholesale.

SCOPE

Direct marketing can benefit farmers by allowing them to earn a greater portion of the consumer food dollar by eliminating intermediary brokers, buyers, and distributors so this web application will help them Direct marketing can contribute toward sustainable agriculture and food systems by increasing farmer profitability, promoting the local economy, and providing consumers with higher quality and healthier products.

Since the web application is available in the smartphone it is easily accessible and always available. It is easy to extend the system that we have proposed. In future we can implement some features like online payment also there is a scope for an android/IOS application.

Features

- The fruits and vegetable images and details can directly uploads to application by registering di
- consumers can view and buy according to their wish.
- Farmers can decide the price
- One can buy and sell using one account

GENERAL CONSTRAINTS

A constraint is something that affects the way that requirements are met. It imposes restrictions on the design of the system that do not affect the external behavior of the system, but must be fulfilled to meet technical or project obligations. Typical constraints are time, money, technology, interaction with already existing systems.

PROPOSED SYSTEM

The proposed system is a very flexible online marketing management application for farmers and consumers. This is a web based application that offers a wide range of operation such as online sales of fruits and vegetables. The fruits and vegetable images and details can directly uploads to application by registering and consumers can view and buy according to their wish. This leads farmers to get more profit. We can easily search and order product. It required less time for registration for farmers and searching of products. The system is user friendly and anyone having computer knowledge can handle it easily. Easy to use and less hardware requirements

SPECIFIC REQUIREMENTS

This section requires the functional and non-functional requirements.

SOFTWARE REQUIREMENTS:-

- Operating System : Windows 7 and above
- Front End : Python
- Database : Firebase
- Tools : Pycharm or python IDE

HARDWARE REQUIREMENTS:-

- Processor : Pentium III
- Speed : 1.1 Ghz
- Hard Disk : 20 GB
- RAM : 4 GB
- Monitor : 15 VGA Color

Keyboard : Standard Windows Keyboard

2.SOFTWARE DESCRIPTION

In this section a brief introduction of the technologies of the software used in the proposed system are discussed.

2.1 PYTHON:-

Python is an open source, high-level programming language developed by Guido van Rossum in the late 1980s and presently administered by Python Software Foundation. It came from the ABC language that he helped create early on in his career. Python is a powerful language that you can use to create games, write GUIs, and develop web applications. It is a high-level language. Reading and writing codes in Python is much like reading and writing regular English statements. Because they are not written in machine-readable language, Python programs need to be processed before machines can run them. Python is an interpreted language. This means that every time a program is run, its interpreter runs through the code and translates it into machine-readable byte code. Python is an object-oriented language that allows users to manage and control data structures or objects to create and run programs. Everything in Python is, in fact, first class. All objects, data types, functions, methods, and classes take equal position in Python. Programming languages are created to satisfy the needs of programmers and users for an effective tool to develop applications that impact lives, lifestyles, economy, and society. They help make lives better by increasing productivity, enhancing communication, and improving efficiency. Languages die and become obsolete when they fail to live up to expectations and are replaced 9 and superseded by languages that are more powerful. Python is a programming language that has stood the test of time and has remained relevant across industries and businesses and among programmers, and individual users. It is a living, thriving, and highly useful language that is highly recommended as a first programming language for

those who want to dive into and experience programming. Scripting language: A script is a program that controls other programs. Scripting languages are good for quick development and prototyping because they're good at passing messages from one component to another and at handling fiddly stuff like memory management so that the programmer doesn't have to. Python has grown beyond scripting languages, which are used mostly for small applications. The Python community prefers to call Python a dynamic programming language.

Using Python:

- Web applications and frameworks
- Game development
- Data analysis
- Machine learning and artificial intelligence
- GUI based desktop applications
- Enterprise and business applications

ADVANTAGES AND DIS ADVANTAGES OF PYTHON

Advantages of Python

1. **Easy to use and learn:** For beginners, Python is straightforward to use. It is a high-level programming language, and its syntax is like the English language. These reasons make the language easy to learn and adapt to. Compared to Java and C, in Python, the same task can be performed using fewer lines of code. As a result of its easy learning, the principles in Python can be executed faster compared to other languages.
2. **Increased productivity:** Python is a very productive language. The simple nature of Python helps the developers to concentrate on solving the issues in it. To understand the syntax and behavior of the programming language, the users do not have to spend hours, so more work is done.

3. **Flexibility:** This language is very flexible, and hence it allows the user to try new things. The users can develop new sorts of the application using Python programming language. The language does not restrict the user from trying something different. Other programming languages do not provide this type of flexibility and freedom, and hence Python is more preferred in these matters.
4. **Extensive library:** Python provides the user with a vast library. Python's standard library is massive, and just about every function one needs to perform is available in its library. This is because it has a hugely supportive community and corporate sponsorship. External libraries are not used by users while working with Python.
5. **Supportive community:** The Python language was created years ago, and hence it has a mature community that can support every type of developer, starting from beginners' level to expert level. There are enough guides, tutorials, and documentation available on the Python programming language, which helps the developers to understand the language faster and better. Because of its supportive community, Python has rapid growth compared to other languages.

Disadvantages of Python

1. **Speed:** Compared to Java or C, the rate of Python is slower. Python is an interpreted language that is dynamically typed. For the execution of a code, each line of the code needs to be explicitly ordered since the language gets interpreted. This is time-consuming, and hence it slows down the process of execution. The dynamic structure of Python also slows its speed because while executing the code, the excess work also needs to be completed. Therefore, in cases where fast acceleration is required, Python is not used there very commonly.
2. **Memory consumption:** Python has a very high memory consumption. This is because it is flexible to the data types. It uses large amounts of memory. Python is not a good choice for tasks where the user wants to optimize memory, i.e., a memory-intensive language.

3. **Mobile development:** Python is strong in server platforms and desktops, and hence it is a fantastic server-side programming language. But it is not appropriate for mobile development. For mobile development, Python is a fragile language. Since it is not memory efficient and has a prolonged power for processing, due to these reasons, Python does not have many built-in mobile applications. Carbonnelle is a built-in application present in Python.
4. **Database access:** Python provides easy programming. However, when it interacts with the database, some issues arise. Compared to technologies like JDBC and ODBC, which are pretty famous, the database access layer of the Python programming language is primitive and underdeveloped. Large enterprises that usually need smooth interaction with complex legacy data do not prefer the usage of Python.
5. **Runtime errors:** The users of Python mentioned various issues they faced with the language design. Since the language of Python is dynamically typed, there can be changes in the data type of a variable at any time. Therefore, it needs to be tested more often, and also, there are errors in the language displayed during runtime.
6. **Simplicity:** Python is a straightforward and easy-to-use programming language which is also a disadvantage of the language. The users of Python get so accustomed to its easy syntax and extensive library feature that they face issues while learning other programming languages. Some users also feel that the Java codes are unnecessary due to their complexity. Therefore, Python has a very vulnerable nature, and the users start taking everything lightly.

Some of the general IDEs which have Python support:

- Sublime Text
- Atom
- Eclipse + PyDev

Some of the best Python-specific IDEs are:

- PyCharm
- Thonny
- Spyder

There are more, but these are a few of the best ones.

2.2 FIREBASE

Firebase is a product of Google which helps developers to build, manage, and grow their apps easily. It helps developers to build their apps faster and in a more secure way. No programming is required on the firebase side which makes it easy to use its features more efficiently. It provides services to android, ios, web, and unity. It provides cloud storage. It uses NoSQL for the database for the storage of data.

Firebase is not only a database but is a feature-rich platform for mobile app development. According to Google. “firebase is Google’s mobile application development platform that helps you build, improve and grow your app”. In the time during which an application is being developed, there are a lot of things on which a developer has to work. It not only costs them a lot of time but also a lot of hard work and still it is difficult for beginners to do it.

So, most of the necessary tools that help an app to work properly and grow are provided by firebase which is developed by Google. There is a lot of stuff that is included in firebase but here in this article, we’ll go for a basic understanding of it only.

Google also says that **“firebase helps you build and run successful apps”** so, if we summarize it in simple words we can say that firebase is a platform for mobile apps

development that provides all the necessary tools to build, improve, and grow apps efficiently and also helps the developers to save a lot of time.

It allows the developer to add advanced functionalities to his app without spending any extra hours or days as one can add such functionalities via Firebase

Firebase Pros and Cons

Pros:

- JSON (Javascript Object Notation) storage means no barrier between data and objects
- simple serialization of app state
- 3-way data binding via Angularfire
- minimal setup
- easy access to data, files, auth, and more
- no server infrastructure needed to power apps with data
- massive storage size potential
- real time
- pretty much the most advanced hosted BaaS solution
- highly secure
- serverless

Cons:

- not widely used or battle tested for Enterprises
- very limited querying and indexing
- no aggregation
- no map reduce
- can’t query or list users or stored files

2.3 PYTHON IDE:-

An integrated development environment is a software that helps you write the code. IDE makes developing applications, programs easier. It acts as a text editor, debugger, and compiler. It contains all the tools and features required by a programmer. Hence, it is easier to develop any application or program using an IDE.

An IDE (or Integrated Development Environment) is a program dedicated to software development. As the name implies, IDEs integrate several tools specifically designed for software development. These tools usually include:

An editor designed to handle code (with, for example, syntax highlighting and auto-completion)

Build, execution, and debugging tools

Some form of source control

Most IDEs support many different programming languages and contain many more features. They can, therefore, be large and take time to download and install. You may also need advanced knowledge to use them properly.

In contrast, a dedicated code editor can be as simple as a text editor with syntax highlighting and code formatting capabilities. Most good code editors can execute code and control a debugger. The very best ones interact with source control systems as well. Compared to an IDE, a good dedicated code editor is usually smaller and quicker, but often less feature rich.

There are a core set of features that makes coding easier:

- Save and reload code files

If an IDE or editor won't let you save your work and reopen everything later, in the same state it was in when you left, it's not much of an IDE.

- Run code from within the environment

  Similarly, if you have to drop out of the editor to run your Python code, then it's not much more than a simple text editor.

- Debugging support

  Being able to step through your code as it runs is a core feature of all IDEs and most good code editors.

- Syntax highlighting

  Being able to quickly spot keywords, variables, and symbols in your code makes reading and understanding code much easier.

- Automatic code formatting

  Any editor or IDE worth it's salt will recognize the colon at the end of a while or for statement, and know the next line should be indented.

2.4 PYCHARM

PyCharm is the most popular IDE used for Python scripting language. This chapter will give you an introduction to PyCharm and explains its features. PyCharm enables smoother code completion whether it is for built in or for an external package. SQLAlchemy as Debugger You can set a breakpoint, pause in the debugger and can see the SQL representation of the user

expression for SQL Language code. Git Visualization in Editor When coding in Python, queries are normal for a developer.

Features:-

•Coding assistance and analysis, with code completion, syntax and error highlighting, linter integration, and quick fixes

• Project and code navigation: specialized project views, file structure views and quick jumping between files, classes, methods and usages

• Python refactoring: includes rename, extract method, introduce variable, introduce constant, pull up, push down and others

• Support for web frameworks: Django, web2py and Flask.

• Integrated Python debugger

• Integrated unit testing, with line-by-line code coverage

• Google App Engine Python development [professional edition only]

• Support for scientific tools like matplotlib, numpy and scipy.

Overview of Important Features and Tools Provided by PyCharm

• Code Editor

The intelligent code editor provided by PyCharm enables programmers to write high quality Python code. The editor enables programmers to read code easily 14 through colour schemes, insert indents on new lines automatically, pick the appropriate coding style, and avail context-aware code completion suggestions. At the same time, the programmers can also use the editor to expand a code block to an expression or logical block, avail code snippets, format the code base, identify errors and misspellings, detect duplicate code, and auto-generate code.Also, the editor makes it easier for developers to analyze the code and identify the errors while writing code.

• Code Navigation

The smart code navigation options provided by PyCharm help programmers to edit and improve code without putting extra time and effort. The IDE makes it easier for programmers to go to a class, file and symbols, along with the go to declarations invoked from a reference. The user can even find an item in the source code, code snippet, UI element, or user action almost immediately. They can further locate usage of various symbols, and set bookmarks in the code ·

•Refactoring

PyCharm makes it easier for developers to implement both local and global changes quickly and efficiently. The developers can even take advantage of the refactoring options provided by the IDE while writing plain Python code and working with Python frameworks. They can avail the rename and move refactoring for files, classes, functions, methods, properties, parameters, and local/global variables. Likewise, they can improve code quality by extracting 15 variables, fields, constants, and parameters. Also, PyCharm allows programmers to break up longer classes and methods through extract method

Support for Popular Web Technologies

PyCharm makes it easier for programmers to write various web applications in Python supporting widely used web technologies like HTML, CSS, JavaScript, TypeScript and CoffeeScript. The web developers can use the live editing preview option provided by the IDE to view a single web page simultaneously in the editor and browser. ·

Support for Popular Python Web Frameworks

In addition to supporting commonly used web technologies, PyCharm also provides first-class support for a robust Python web framework like Django. The developers can use the IDE to avail code completion suggestions for Django tags, filters, parameters, and template variables. Also, they can gather additional information about tags and filters by referring to the quick documentation. The Python IDE even helps web developers to debug Django templates, format the code, verify the code, and manage .py consoles.

· **Support for Python Scientific Libraries**

PyCharm further helps programmers to use Python more efficiently in big data and data science projects. It supports some of the widely used scientific libraries for Python — NumPy, Anaconda and Metplotlib. The developers can work efficiently 16 with these scientific libraries by availing the interactive graphs, deep code insight, and array viewers provided by the IDE. ·

Database Tools

In addition to supporting various Python libraries and frameworks, PyCharm allows developers to work with a number of relational databases including Oracle, SQL Server, MySQL and PostgreSQL. The developers can further use the IDE to run queries, edit SQL code, browse data, alter table data, and alter/analyze schemas. PyCharm further supports SQLAlchemy library and inject SQL code into code written in various programming languages. ·

Visual Debugger

The visual debugger provided by the IDE helps programmers to debug Python, JavaScript, and Django code. The developers can use the inline debugger to see live debugging data directly on the editor. Likewise, they can debug multiple Python processes simultaneously and step through the code bypassing libraries.

Built-in Terminal

PyCharm comes with local terminals for Windows, Linux, and macOS. The builtin terminal enables programmers to continue coding and testing without leaving the IDE. Also, the programmers can use the IDE to run Python files and configure custom Python environments according to precise project requirements. At the same time, they can run interactive Python or Django consoled directly in the IDE.

Support for Major Version Control Systems

PyCharm allows developers to work with widely used version control systems like Git, Mercurial, Perforce and SVN. It even performs complex tasks like adding, removing, and deleting files automatically. The developers even have option to avail a number of features provided by the IDE regardless of their choice of version control system — grouping

individual changes into multiple change lists, setting aside the changes to be restored, monitor changes made to the code repository by various users, and check the changes made to the code before being integrated into the local copy.

· **Remote Development Capabilities**

PyCharm allows developers to connect with various machines and build software applications remotely. The programmers can avail the built-in SSH console provided by the IDE to connect to machines and perform various development tasks remotely through SSH. They can even run, debug, and profile the Python applications in a remote environment by replacing the local interpreter with a remote interpreter. PyCharm enables programmers to create reproducible development environments through a robust tool like Vagrant, and simplify distributed application development through Docker. The users even have option to integrate PyCharm seamlessly with issue tracking systems.

2.5 HTML

HTML stands for Hyper Text Markup Language, which is the most widely used language on Web to develop web pages. HTML was created by Berners-Lee in late 1991 but "HTML 2.0" was the first standard HTML specification which was published in 1995. HTML 4.01 was a major version of HTML and it was published in late 1999. Though HTML 4.01 version is widely used but currently we are having HTML-5 version which is an extension to HTML 4.01, and this version was published in 2012.

• Hypertext refers to the way in which Web pages (HTML documents) are linked together. Thus the link available on a webpage is called Hypertext.

• As its name suggests, HTML is a Markup Language which means you use HTML to simply "mark up" a text document with tags that tell a Web browser how to structure it to display.

Originally, HTML was developed with the intent of defining the structure of documents like headings, paragraphs, lists, and so forth to facilitate the sharing of scientific information between researchers. These are some basic tags of html Tag Description

Tag	**Description**
<! DOCTYPE>	This tag defines the document type and HTML version.
<html>	This tag encloses the complete HTML document and mainly comprises of document header which is represented by **<head>...</head>** and document body which is represented by **<body>...</body>** tags.
<head>	This tag represents the document's header which can keep other HTML tags like <title>, <link> etc.
<title>	The **<title>** tag is used inside the <head> tag to mention the document title.
<body>	This tag represents the document's body which keeps other HTML tags like <h1>, , <p> etc.
<h1>	This tag represents the heading.
<p>	This tag represents a paragraph.

2.6 ABOUT OPERATING SYSTEM

An Operating System may be viewed as an organized collection of software extensions of hardware, consisting of control routines for operating a computer and for providing an environment for execution of programs. Programs rely on facilities provided by the operating system to gain access to computer-system resources such as files and input/output devices. i.e., the operating system acts as interface between users and the hardware of a computer system. Windows XP is an operating system that uses pre-emptive, multitasking and multi-threading to perform several actions at the same time. The advantage are powerful mobile computing features using the Microsoft Network, a Recycle Bin for storing the deleted files which can be cleared later and the inbox which can be used for communications like FAX, E-MAIL etc. Windows 2000 includes a suite of programs designed to optimize our computer's efficiency, especially when used together. It improves computer reliability by introducing new wizards, utilities, and resources that help keep our system running smoothly. The key features are: • A very advanced set of multimedia programs are shipped along with the operating system that just doesn't sing or run animations but can create and edit these applications.

• The Operating System automatically detects network cards and user has to just specify the name of the network and the computer can be logged on to the specified network.

• The computer can be connected to the internet if the user has access to it and the operating system needed to log on and run applications on the internet.

2.7 SUBLIME TEXT EDITOR

Sublime is a graphical text editor that's extensible with Python. It sports a mini-map scroll bar and multiple-cursor editing, but its most important feature is how fast it is despite the

number and size of files that are open. Features like jumping to a function definition are built-in, despite it not being an IDE or having innate knowledge of how to compile a project. It has three features built-in that I added to The nvi text editor with Modern nvi mappings but which feel more seamless in Sublime Text:

⌘P opens a file in the currently open project based on fuzzy searching.

⌘F searches across multiple files in the project.

⌘R jumps to a definition based on fuzzy searching.

But it feels like Sublime Text's moment in the sun is passing. Many of the most popular plugins haven't seen updates recently and it looks like their authors have moved on.

Sublime Text has some headlining features that I don't particularly care for:

The mini-map doesn't offer much and takes up valuable screen space. I do like the larger grab area for the scroll bar, though

Multiple cursor editing doesn't make much sense to me unless you're very fluent in moving the cursor using the keyboard, like with vim. Using ⌘D to select the next occurence of selected text helps, but I have trouble remembering to use it.

And others that I haven't given a fair shot yet:

One of my favorite features from vim is the macro recording system, where you start recording, do a series of actions, and then can replay those actions on other lines. It feels like a natural way to build batch editing operations. Sublime Text has this feature, but I haven't tried it.

It's possible to set up a JSON file that instructs Sublime Text to build a project and get file and line information from any errors or warnings. I've tried to use this for Xcode projects, but didn't spend time finding out why it wasn't working.

Sublime Text doesn't support italic or bold with proportional fonts so it's difficult to recommend it for writing prose.

I've augmented the built-in features with some Sublime Text plugins I use and fortunately didn't have to spend much time Configuring the defaults of Sublime Text.

Linked to by Configuring the defaults of Sublime Text, Future macOS software setup, Text editing software, The Nova text editor, The nvi text editor, macOS software setup.

2.8 VISUAL STUDIO

Visual Studio is an Integrated Development Environment(IDE) developed by Microsoft to develop GUI(Graphical User Interface), console, Web applications, web apps, mobile apps, cloud, and web services, etc. With the help of this IDE, you can create managed code as well as native code. It uses the various platforms of Microsoft software development software like Windows store, Microsoft Silverlight, and Windows API, etc. It is not a language-specific IDE as you can use this to write code in C#, C++, VB(Visual Basic), Python, JavaScript, and many more languages. It provides support for 36 different programming languages. It is available for Windows as well as for macOS.

Evolution of Visual Studio: The first version of VS(Visual Studio) was released in 1997, named as Visual Studio 97 having version number 5.0. The latest version of Visual Studio is 15.0 which was released on March 7, 2017. It is also termed as Visual Studio 2017. The supported .Net Framework Versions in latest Visual Studio is 3.5 to 4.7. Java was supported in old versions of Visual Studio but in the latest version doesn’t provide any support for Java language.

Visual studio edition:-

1. Community: It is a free version which is announced in 2014. All other editions are paid. This contains the features similar to Professional edition. Using this edition, any individual developer can develop their own free or paid apps like .Net applications, Web applications and many more. In an enterprise organization, this edition has some limitations. For example, if your organization have more than 250 PCs and having annual revenue greater than $1 Million(US Dollars) then you are not permitted to use this edition. In a non-

enterprise organization, up to five users can use this edition. Its main purpose is to provide the Ecosystem(Access to thousands of extensions) and Languages(You can code in C#, VB, F#, C++, HTML, JavaScript, Python, etc.) support.

2. Professional: It is the commercial edition of Visual Studio. It comes in Visual Studio 2010 and later versions. It provides the support for XML and XSLT editing and includes the tool like Server Explorer and integration with Microsoft SQL Server. Microsoft provides a free trial of this edition and after the trial period, the user has to pay to continue using it. Its main purpose is to provide Flexibility(Professional developer tools for building any application type), Productivity(Powerful features such as CodeLens improve your team's productivity), Collaboration(Agile project planning tools, charts, etc.) and Subscriber benefits like Microsoft software, plus Azure, Pluralsight, etc.

3. Enterprise: It is an integrated, end to end solution for teams of any size with the demanding quality and scale needs. Microsoft provides a 90-days free trial of this edition and after the trial period, the user has to pay to continue using it. The main benefit of this edition is that it is highly scalable and deliver high-quality software.

2.9 DJANGO:-

Django is a python-based free and open-source web framework that follows the model–template–views (MTV) architectural pattern. It is maintained by the Django Software Foundation (DSF), an American independent organization established as a 501©(3) non-profit. Django's primary goal is to ease the creation of complex, database-driven websites. The framework emphasizes reusability and "plug ability" of components, less code, low coupling, rapid development, and the principle of don't repeat yourself. Python is used throughout, even for settings, files, and data models. Django also provides an optional

administrative create, read, update and delete interface that is generated dynamically through introspection and configured via admin models. Included in the core framework are:

- a lightweight and standalone web server for development and testing
- a form serialization and validation system that can translate between HTML forms and values suitable for storage in the database
- a template system that utilizes the concept of inheritance borrowed from object-oriented programming
- a caching framework that can use any of several cache methods

ANALYSIS TOOLS

DATA FLOW DIAGRAM:-

DFD is the abbreviation for **Data Flow Diagram**. The flow of data of a system or a process is represented by DFD. It also gives insight into the inputs and outputs of each entity and the process itself. DFD does not have control flow and no loops or decision rules are present. Specific operations depending on the type of data can be explained by a flowchart. Data Flow Diagram can be represented in several ways. The DFD belongs to structured-analysis modeling tools. Data Flow diagrams are very popular because they help us to visualize the major steps and data involved in software-system processes.

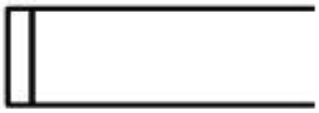
COMPONENTS OF DFD

The Data Flow Diagram has 4 components:

1. **External entity**: an outside system that sends or receives data, communicating with the system being diagrammed. They are the sources and destinations of information entering or leaving the system. They might be an outside organization or person, a computer system or a business system. They are also known as terminators, sources and sinks or actors. They are typically drawn on the edges of the diagram.

2. Process: any process that changes the data, producing an output. It might perform computations, or sort data based on logic, or direct the data flow based on business rules. A short label is used to describe the process, such as ―Submit payment.‖

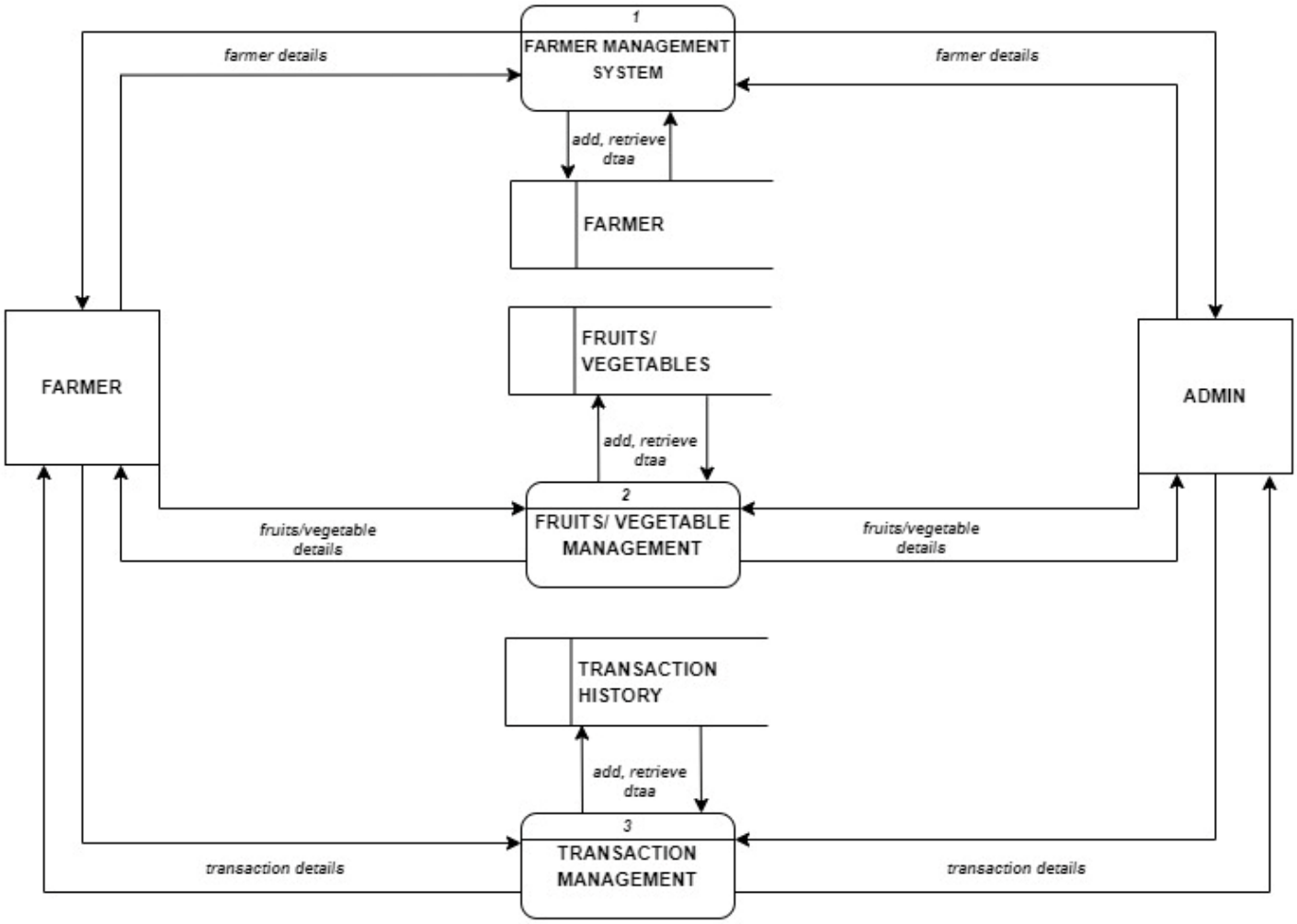
3 Data store: files or repositories that hold information for later use, such as a database table or a membership form. Each data store receives a simple label, such as ―Orders.‖ 3. Data store: files or repositories that hold information for later use, such as a database table or a membership form. Each data store receives a simple label, such as ―Orders.‖



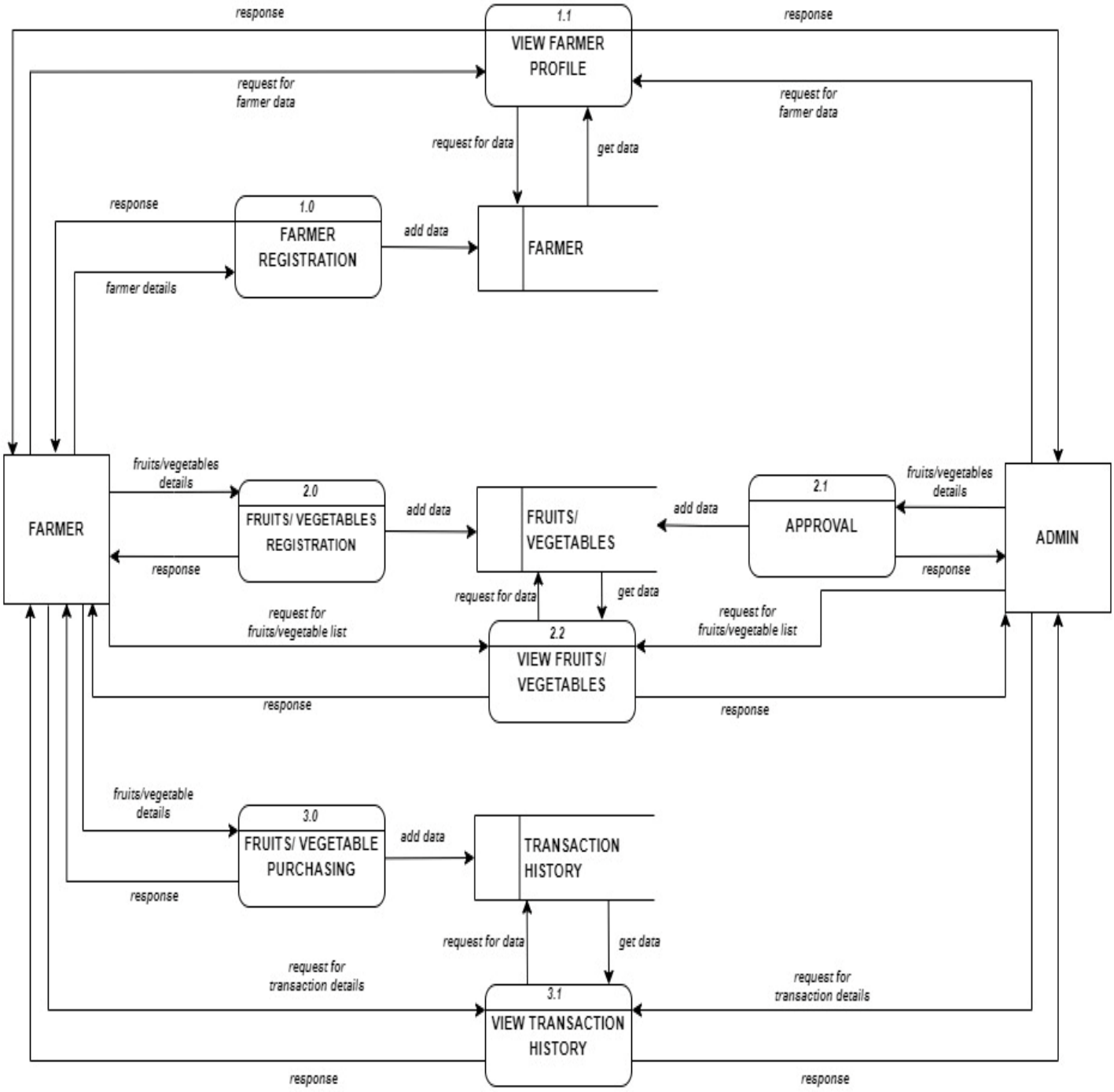
4. Data flow: the route that data takes between the external entities, processes and data stores. It portrays the interface between the other components and is shown with arrows, typically labeled with a short data name, like ―Billing details.

Level 0 DFD:

Level 1 DFD:


1
FARMER MANAGEMENT SYSTEM
farmer details
farmer details
add, retrieve dtaa
FARMER
FRUITS/ VEGETABLES
FARMER
ADMIN
add, retrieve dtaa
2
FRUITS/ VEGETABLE MANAGEMENT
fruits/vegetable details
fruits/vegetable details
TRANSACTION HISTORY
add, retrieve dtaa
3
TRANSACTION MANAGEMENT
transaction details
transaction details

Level 2 DFD:


1.1
VIEW FARMER PROFILE
response
response
request for farmer data
request for farmer data
request for data
get data
1.0
FARMER REGISTRATION
response
add data
FARMER
farmer details
FARMER
fruits/vegetables details
2.0
FRUITS/ VEGETABLES REGISTRATION
add data
FRUITS/ VEGETABLES
add data
2.1
APPROVAL
fruits/vegetables details
ADMIN
response
response
request for data
get data
request for fruits/vegetable list
request for fruits/vegetable list
2.2
VIEW FRUITS/ VEGETABLES
response
response
fruits/vegetable details
3.0
FRUITS/ VEGETABLE PURCHASING
add data
TRANSACTION HISTORY
response
request for data
get data
request for transaction details
request for transaction details
3.1
VIEW TRANSACTION HISTORY
response
response

ENTITY RELATIONSHIP DIAGRAM

An entity relationship diagram (ERD) shows the relationships of entity sets stored in a database. An entity in this context is an object, a component of data. An entity set is a collection of similar entities. These entities can have attributes that define its properties. By defining the entities, their attributes, and showing the relationships between them, an ER diagram illustrates the logical structure of databases. ER diagrams are used to sketch out the design of a database.

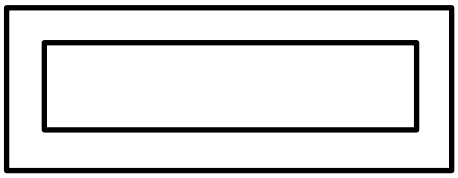
ENTITY RELATIONSHIP DIAGRAM SYMBOLS

An ER diagram is a means of visualizing how the information a system produces is related. There are five main components of an ERD:

Entities:-

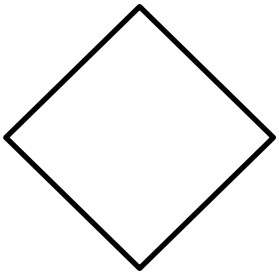
Which are represented by rectangles An entity is an object or concept about which you want to store information.

A weak entity is an entity that must defined by a foreign key relationship with another entity as it cannot be uniquely identified by its own attributes alone.



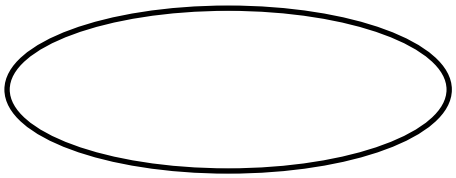
Actions,

which are represented by diamond shapes, show how two entities share information in the database. In some cases, entities can be self-linked.



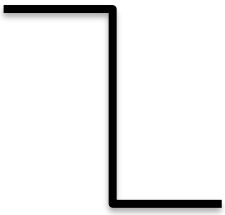
Attributes:-

Which are represented by ovals. A key attribute is the unique, distinguishing characteristic of the entity. For example, an employee's social security number might be the employee's key attribute.



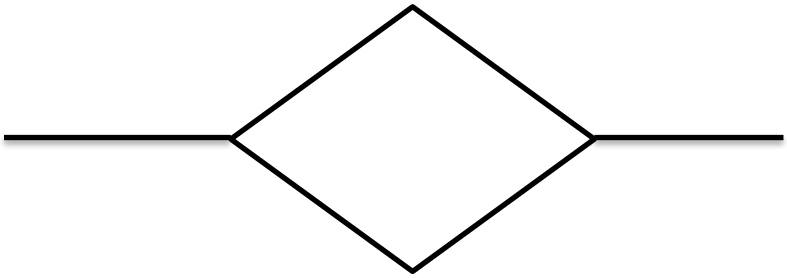
Connecting lines,

solid lines that connect attributes to show the relationships of entities in the diagram.

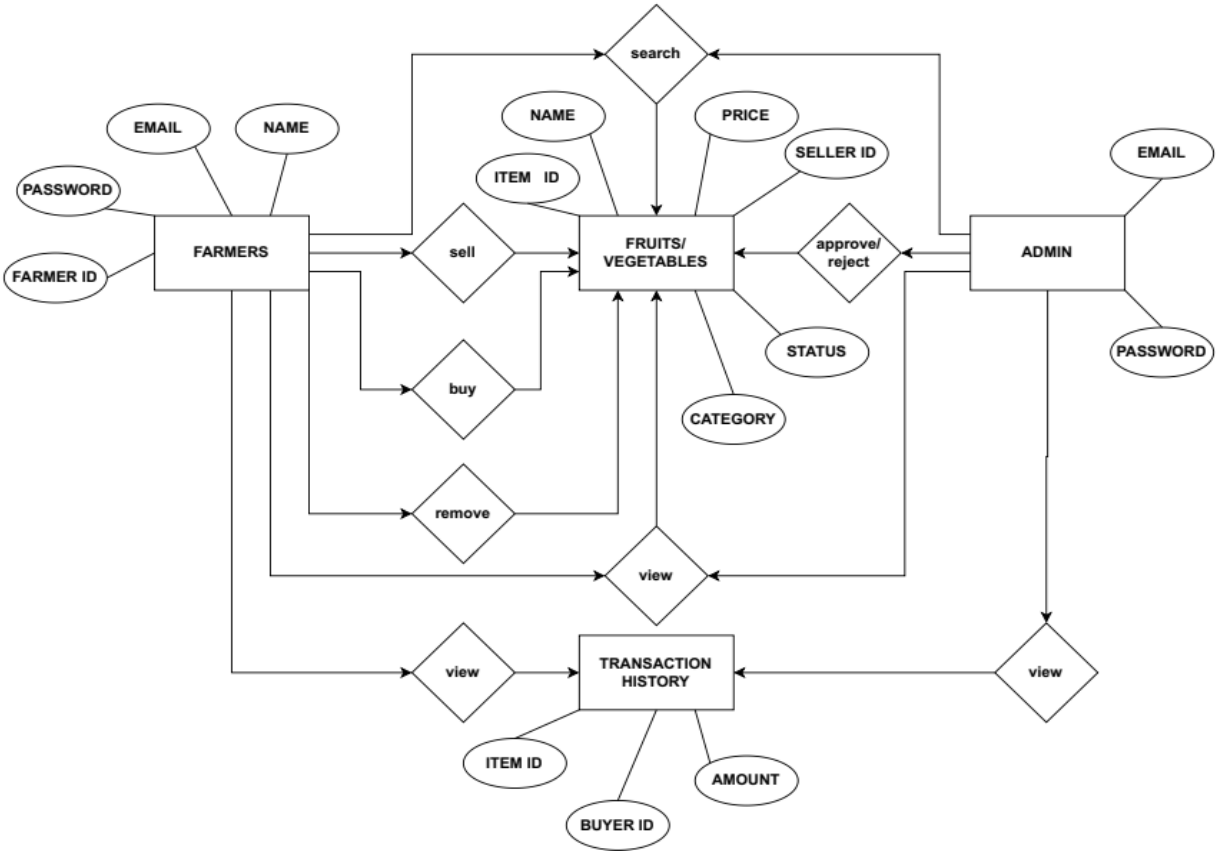


Cardinality

specifies how many instances of an entity relate to one instance of another entity. Ordinarily is also closely linked to cardinality. While cardinality specifies the occurrences of a relationship, ordinarily describes the relationship as either mandatory or optional. In other words, cardinality specifies the maximum number of relationships and ordinarily specifies the absolute minimum number of relationships.



ER Diagram:-



5

SYSTEM DESIGN

SYSTEM DESIGN

MODULE DESCRIPTION:-

- Admin
- Farmers

ADMIN

• **APPROVE / REJECT FRUITS/VEGETABLES:** admin will approve the fruits/vegetables before it goes for sale

• **VIEW FARMERS LIST:** admin can view the list of farmers registered into the system

• **VIEW FARMERS PROFILE:** admin can view the profile details of the farmers

• **VIEW FRUITS/VEGETABLES LIST:** admin can view the list fruits and vegetables that are in sales

• **VIEW FRUITS/VEGETABLES DETAILS:** admin can view complete details of fruits and vegetables

• **VIEW TRANSACTION HISTORY:** admin can view whole transaction details of the sale that done in that system

FARMERS

• **FARMERS REGISTRATION:** farmers itself register into the system with essential details

• **EDIT PROFILE:** farmers can edit his/her profile of farmers anytime

• **SELL FRUITS/VEGETABLES:** farmers can sell his fruits/vegetables in this system. But farmers want to wait for the approval from admin to sale that fruits/vegetables

• **BUY FRUITS/VEGETABLES:** farmers can also buy the fruits/vegetables that others farmers put on sales

• **VIEW SELL HISTORY:** farmers can view the history of fruits/vegetables and its details that he/she sold

• **VIEW BUY HISTORY:** farmers can view the history of fruits/vegetables and its details

that he/she bought from other farmers

- **SEARCH FRUITS/VEGETABLES:** farmers can fruits/vegetables on the basis of name and location also
- **VIEW FRUITS/VEGETABLES LIST:** farmers can view the list of fruits/vegetables and also filtered fruits/vegetables list on the basis of location
- **VIEW OWN FRUITS/VEGETABLES LIST:** farmers can view list of fruits/vegetables that he/she put on sale

MODULE DESCRIPTION

A module is logically separate part of a program. It is a program unit that is discreet and identifiable with respect to compiling and loading. A system is considered to modular if it consists of discrete components so that each component can be implemented separately and a change to one component has minimal impaction other components. Modularity is clearly a desirable property in a system. Modularity helps in system debugging

Isolating the system problems to component is easier if the system is modular. In system repair or changing a part of the system is easy as it affect few other parts and system building. A modular system can easily built by putting its modules together.

The main modules are:

ADMIN

- ❖ Admin can approve the product
- ❖ Admin can view farmers list
- ❖ Admin can view fruits and vegetables list that are in for sale
- ❖ Admin can view transaction history

FARMERS

- ❖ Farmers can register itself into the system with essential details.
- ❖ Frmers can sell fruits/vegetables
- ❖ Farmers can view history of the vegetables/fruits and details that they sold

- Farmers can view history of the vegetables/fruits and details that they sold

TABLE DESIGN

TABLE NAME: ADMIN LOGIN

FIELD	TYPE	CONSTRAINS	DESCRIPTION
email	Varchar(30)	Not null	admin email id
password	Varchar(10)	Not null	admin password

TABLE NAME: FARMER

FIELD	TYPE	CONSTRAINS	DESCRIPTION
farmer_id	Varchar(10)	Primary key	farmer id
name	Varchar(25)	Not null	name of farmer
email	Varchar(30)	Not null	farmer email
password	Varchar(10)	Not null	farmer password
mobile_number	Varchar(10)	Not null	farmer mobile number

TABLE NAME: FRUITS/VEGETABLES

FIELD	TYPE	CONSTRAINS	DESCRIPTION
item_id	Varchar(10)	Primary key	item id
seller_farmer_id	Varchar(10)	Foreign key	seller farmer id
item _name	Varchar(25)	Not null	item name
item _image	Varchar(100)	Not null	item image
item _desc	Varchar(100)	Not null	item description
item _stock	int	Not null	item stock
item _amount	int	Not null	item amount
item _rating	float	Not null	item rating
item _location	Varchar(50)	Not null	item location

TABLE NAME: TRANSACTION HISTORY

FIELD	TYPE	CONSTRAINS	DESCRIPTION
trans_id	Varchar(10)	Primary key	transaction id
item_id	Varchar(10)	Foreign key	item id
buyer_farmer_id	Varchar(10)	Foreign key	buyer farmer id
quanity	int	Not null	quantity
amount	int	Not null	amount

DATA INTEGRITY AND CONSTRAINTS

Data integrity is referring to maintaining and assuring the accuracy and consistency of data over its entire life-cycle. Database Systems ensure data integrity through constraints which are used to restrict data that can be entered or modified in the database. In this project data security, data validation checking methods are applied using a password authentication Generally the access to the whole system can be provided only if given user id and password as authorized otherwise, an error message box is displayed once user gives correct password he is authenticated and hence permitted to get in to the master form. Data integrity refers to the process of ensuring that a database remains an accurate reflection of the universe of discourse it is modeling or representing In other words there is a close correspondence between the facts stored in the database and the real world it models data integrity is normally enforced in a database system by a series of integrity constraints or rules. Three types of integrity constraints are an inherent part of the relational data model, entity integrity, referential integrity and domain integrity. Entity integrity the concept of a primary key . Entity integrity is integrity rule which status that every table must have a primary key and that the column or columns chosen to be the primary key should be unique and not null. Referential integrity concerns the concept of a foreign key. The referential integrity rules states that any foreign key value can only be inone of two states Domain integrity specifies that all columns in relational database must be declared upon a defined domain.

A constraint is a property assigned to a column or the set of columns in a table that prevents certain type of inconsistent data values from being placed in the column Constraints are used to enforce the data integrity. This ensures the accuracy and reliability of the data in the database.

A PRIMARY KEY constraint is a unique identifier for a row within database. Every a table should have a constraint to uniquely identify each row and only one primary key constraint can be created for each table. A UNIQUE constraint enforces the uniqueness of the value in a set of

columns, So no duplicate values are entered. The unique key constraints are used to enforce entity integrity as the primary key constraint.

A FOREIGN KEY constraint prevents any action that would destroy link between tables with the corresponding data values. A CHECK constraint is used to limit the values that can be placed in a column. The check constraint are used to enforce domain integrity A NOT NULL constraint enforces that column will not accept null value. The not null constraint are used to enforce domain integrity, as the check constraints

NORMALIZATION

The process of normalization is concerned with the transformation of the conceptual scheme to a computer represent able form. Normalization reduces the redundancies and anomalies. If a database design is not perfect, it may contain anomalies, which are like a bad dream for any database administrator. Managing a database with anomalies is next to impossible

. Update Anomalies: If data items are scattered and are not linked to each other properly, then it could lead to strange situations.

Deletion anomalies: We tried to delete a record, but parts of it was left undeleted because of unawareness, the data is also saved somewhere else.

Insert anomalies: We tried to insert data in a record that does not exist at all. Normalization is a method to remove all these anomalies and bring the database to a consistent state.

First Normal Form

First Normal Form is defined in the definition of relations (tables) itself. This rule defines that all the attributes in a relation Must have atomic domains. The values in an atomic domain are indivisible units.

Second Normal Form

In second normal form, for relation where primary key contains multiple attributes, non-key attributes should be functionally dependent on a part of the primary key.

Third Normal Form:

In third normal form, relation should not have a non-key attribute functionally determined by another non-key attribute. That is there should be no transitive dependency of a non-key attribute on the primary key. DE design is required to manage large bodies of information. The management involves both the definition of the structure of storage of information and provisions of mechanism for the manipulation of information.

USER INTERFACE DESIGN

User gets a friendly environment to interact with the software fields. As we are doing the program in PHP and Android the user can view different forms and data access as well as entering is easy. The user's interaction with system is through forms. Forms are related to databases, So that it becomes easy to enter the data in to the system. Once the input forms are designed the actual data must be entered in to forms, the data entry must be done carefully to avoid any incorrect data. Farms are designed in such a way that during the processing the cursor is placed in the position where the data must be entered the designed forms must be user friendly.

INPUT DESIGN

The goal of input design is to capture data and get that data into a format suitable for the computer. Wien we computerize a system, the user-oriented program is to be converted into programmer-oriented specification for the computer-based system. All systems need some

input data for processing. In order to achieve the objectives of the system, input should be given accurately at the right time. Any ambiguity concerning the inputs will lead to total faults in the functioning of the System.

The most common source of inaccurate data processing errors is the inaccurate input data, The errors made by users can be controlled by input design. The aim of input design is to make operation as easy and error free as possible.

OUTPUT DESIGN

The output design has been done so that the results of processing should be communicated to the user. Effective output design will improve the clarity and performance of outputs. Output is the main reason for developing the system and the basis on which they will evaluate the usefulness of the application. Output design phase of the system is concerned with the convergence of information to the end user-friendly manner. The output design should be efficient, Intelligible so that system relationship with the end user is improved and they're by enhancing the process of decision-making.

6

PROGRAM CODE

```
from flask import Flask, request
from flask_restful import Resource, Api, abort, reqparse
from flask_cors import CORS
from firebase import Firebase
import werkzeug
from datetime import datetime
from pytz import timezone

app = Flask(__name__)
api = Api(app)
CORS(app)

config = {
  "apiKey": "AIzaSyCjM4iy43XcmqJXXm1vLSXiVDKcIJazMCM",
  "authDomain": "farmersportal.firebaseapp.com",
  "databaseURL": "https://farmersportal.firebaseio.com",
  "storageBucket": "farmersportal.appspot.com",
}

firebase = Firebase(config)
db = firebase.database()
storage = firebase.storage()

ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in
ALLOWED_EXTENSIONS
```

```
farmerRegParser = reqparse.RequestParser()
farmerRegParser.add_argument('name', type=str, required=True)
farmerRegParser.add_argument('mobileNumber', type=int, required=True)
farmerRegParser.add_argument('address', type=str, required=True)
farmerRegParser.add_argument('city', type=str, required=True)
farmerRegParser.add_argument('email', type=str, required=True)
farmerRegParser.add_argument('password', type=str, required=True)
farmerRegParser.add_argument('photo', type=werkzeug.datastructures.FileStorage,
location='files')

farmerUpdateParser = reqparse.RequestParser()
farmerUpdateParser.add_argument('name', type=str)
farmerUpdateParser.add_argument('mobileNumber', type=int)
farmerUpdateParser.add_argument('address', type=str)
farmerUpdateParser.add_argument('city', type=str)
farmerUpdateParser.add_argument('email', type=str)
farmerUpdateParser.add_argument('password', type=str)
farmerUpdateParser.add_argument('photo', type=werkzeug.datastructures.FileStorage,
location='files')

farmerLoginParser = reqparse.RequestParser()
farmerLoginParser.add_argument('email', type=str, required=True)
farmerLoginParser.add_argument('password', type=str, required=True)

adminLoginParser = reqparse.RequestParser()
adminLoginParser.add_argument('email', type=str, required=True)
adminLoginParser.add_argument('password', type=str, required=True)
```

```
catRegParser = reqparse.RequestParser()
catRegParser.add_argument('name', type=str, required=True)

productRegParser = reqparse.RequestParser()
productRegParser.add_argument('name', type=str, required=True)
producutRegParser.add_argument('catID', type=str, required=True)
productRegParser.add_argument('farmerID', type=str, required=True)
productRegParser.add_argument('rate', type=int, required=True)
productRegParser.add_argument('stock', type=int, required=True)
productRegParser.add_argument('description', type=str, required=True)
productRegParser.add_argument('photo', type=werkzeug.datastructures.FileStorage,
location='files', action='append', required=True)

productDataUpdateParser = reqparse.RequestParser()
productDataUpdateParser.add_argument('name', type=str)
productDataUpdateParser.add_argument('catID', type=str)
productDataUpdateParser.add_argument('rate', type=int)
productDataUpdateParser.add_argument('stock', type=int)
productDataUpdateParser.add_argument('description', type=str)

productImgUpdateParser = reqparse.RequestParser()
productImgUpdateParser.add_argument('photo',
type=werkzeug.datastructures.FileStorage, location='files', action='append')
productImgUpdateParser.add_argument('imgName', type=str)

filterProductItemParser = reqparse.RequestParser()
filterProductItemParser.add_argument('farmerID', type=str, required=True)
```

```
filterProductParser = reqparse.RequestParser()
filterProductParser.add_argument('farmerID', type=str, required=True)
filterProductParser.add_argument('catID', type=str, required=True)
filterProductParser.add_argument('catName', type=str, required=True)

searchProductParser = reqparse.RequestParser()
searchProductParser.add_argument('farmerID', type=str, required=True)
searchProductParser.add_argument('catID', type=str, required=True)
searchProductParser.add_argument('catName', type=str)
searchProductParser.add_argument('searchItem', type=str, required=True)

productCartParser = reqparse.RequestParser()
productCartParser.add_argument('productID', type=str, required=True)
productCartParser.add_argument('qty', type=int, required=True)

productCartUpdateParser = reqparse.RequestParser()
productCartUpdateParser.add_argument('qty', type=int, required=True)

productApproveParser = reqparse.RequestParser()
productApproveParser.add_argument('action', type=str, required=True)

def locationlist(productlist):
    locationList = []
    for i in productlist:
        locationList.append(productlist[i]['sellerDetails']['city'].lower())
    locationList = set(locationList)
    locationList = list(locationList)
```

```
    return locationList

def categorylist(productlist):
    tempCategorylist = []
    for i in productlist:
        tempCategorylist.append(productlist[i]['catID'])
    tempCategorylist = set(tempCategorylist)
    tempCategorylist = list(tempCategorylist)
    categoryList = db.child('FarmersPortal').child('categoryList').get().val()
    if categoryList == None:
        categoryList = {}
    category_list = {}
    for i in tempCategorylist:
        if i == 'CATOTH':
            category_list[i] = {'name' : 'others'}
        else:
            category_list[i] = categoryList[i]
    return category_list

class FarmerReg(Resource):
    def post(self):
        args = farmerRegParser.parse_args()
        if args['photo']:
            if not allowed_file(args['photo'].filename):
                abort(400, message='unsupported file format')
        farmerCnt = db.child('FarmersPortal').child('farmerCnt').get().val()
        if farmerCnt == None:
            farmerCnt = 0
```

```
            farmerCnt += 1
            db.child('FarmersPortal').child('farmerCnt').set(farmerCnt)
            farmerID = 'FARM' + str(100 + farmerCnt)
            if args['photo']:
                f = args['photo']
                del args['photo']

        storage.child('FarmersPortal').child('farmerImage').child(farmerID).child('pic.jpg')
.put(f)
            else:

        storage.child('FarmersPortal').child('farmerImage').child(farmerID).child('pic.jpg')
.put('static/user.png')
            farmerList = db.child('FarmersPortal').child('farmerList').get().val()
            if farmerList == None:
                farmerList = {}
            args['imgUrl'] =
storage.child('FarmersPortal').child('farmerImage').child(farmerID).child('pic.jpg').get_u
rl(None)
            farmerList[farmerID] = args
            db.child('FarmersPortal').child('farmerList').set(farmerList)
            return args

        def get(self):
            farmerList = db.child('FarmersPortal').child('farmerList').get().val()
            if farmerList == None:
                farmerList = {}
            return farmerList
```

```
class FarmerUpdate(Resource):
    def get(self, farmerID):
        farmerID = farmerID.upper()
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
            farmerList = {}
        if farmerID in farmerList:
            farmerDetails = farmerList[farmerID]
        else:
            abort(400, message='farmer not found')
        return farmerDetails

    def put(self, farmerID):
        farmerID = farmerID.upper()
        args = farmerUpdateParser.parse_args()
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
            farmerList = {}
        if farmerID in farmerList:
                if args['name']:
                    farmerList[farmerID]['name'] = args['name']
                if args['mobileNumber']:
                    farmerList[farmerID]['mobileNumber'] =
args['mobileNumber']
                if args['address']:
                    farmerList[farmerID]['address'] = args['address']
                if args['city']:
```

```
                    farmerList[farmerID]['city'] = args['city']
                if args['email']:
                    farmerList[farmerID]['email'] = args['email']
                if args['password']:
                    farmerList[farmerID]['password'] = args['password']
                if args['photo']:

        storage.child('FarmersPortal').child('farmerImage').child(farmerID).child('pic.jpg')
.put(args['photo'])
                    farmerList[farmerID]['imgUrl'] =
storage.child('FarmersPortal').child('farmerImage').child(farmerID).child('pic.jpg').get_u
rl(None)
                db.child('FarmersPortal').child('farmerList').set(farmerList)
                farmerDetails= farmerList[farmerID]
            else:
                abort(400, message='farmer not found')
            return farmerDetails

        def delete(self, farmerID):
            farmerID = farmerID.upper()
            farmerList = db.child('FarmersPortal').child('farmerList').get().val()
            if farmerList == None:
                farmerList = {}
            if farmerID in farmerList:
                del farmerList[farmerID]
                db.child('FarmersPortal').child('farmerList').set(farmerList)
            else:
                abort(400, message='farmer not found')
```

```
        return farmerList

class FarmerLogin(Resource):
    def post(self):
        args = farmerLoginParser.parse_args()
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
            farmerList = {}
        farmerID = ''
        for i in farmerList:
            if farmerList[i]['email'] == args['email'] and farmerList[i]['password']
== args['password']:
                farmerID = i
        return farmerID

class AdminLogin(Resource):
    def post(self):
        args = adminLoginParser.parse_args()
        adminPassword =
db.child('FarmersPortal').child('adminPassword').get().val()
        if adminPassword == None:
            adminPassword = 'admin123'
            db.child('FarmersPortal').child('adminPassword').set(adminPassword)
        if 'admin@gmail.com' == args['email'] and adminPassword ==
args['password']:
            return 'admin'
        else:
            return ''
```

```
class CategoryReg(Resource):
    def post(self):
        args = catRegParser.parse_args()
        args['deleteStatus'] = 0
        catCnt = db.child('FarmersPortal').child('catCnt').get().val()
        if catCnt == None:
            catCnt = 0
        catCnt += 1
        db.child('FarmersPortal').child('catCnt').set(catCnt)
        catID = 'CAT' + str(100 + catCnt)
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
            categoryList = {}
        categoryList[catID] = args
        db.child('FarmersPortal').child('categoryList').set(categoryList)
        return args

    def get(self):
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
            categoryList = {}
        categoryList['CATOTH'] = {'name':'others'}
        return categoryList

class CategoryUpdate(Resource):
    def get(self, catID):
        catID = catID.upper()
```

```
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
                categoryList = {}
        if catID in categoryList:
                catDetails = categoryList[catID]
        else:
                abort(400, message='category not found')
        return catDetails

    def put(self, catID):
        catID = catID.upper()
        args = catRegParser.parse_args()
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
                categoryList = {}
        if catID in categoryList:
                        if args['name']:
                                categoryList[catID]['name'] = args['name']
                        db.child('FarmersPortal').child('categoryList').set(categoryList)
                        catDetails= categoryList[catID]
        else:
                abort(400, message='category not found')
        return catDetails

    def delete(self, catID):
        catID = catID.upper()
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
```

```
            categoryList = {}
        if catID in categoryList:
            if categoryList[catID]['deleteStatus'] == 0:
                del categoryList[catID]
                db.child('FarmersPortal').child('categoryList').set(categoryList)
            else:
                abort(400, message='category cant be deleted')
        else:
            abort(400, message='category not found')
        return categoryList

class ProductReg(Resource):
    def post(self):
        args = productRegParser.parse_args()
        farmerID = args['farmerID'].upper()
        args['farmerID'] = farmerID
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
            farmerList = {}
        if not farmerID in farmerList:
            abort(400, message='farmer not found')
        catID = args['catID'].upper()
        args['catID'] = catID
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
            categoryList = {}
        if catID != 'CATOTH':
            if not catID in categoryList:
```

```
                abort(400, message='category not found')
        images = args['photo']
        del args['photo']
        for img in images:
            if not allowed_file(img.filename):
                abort(400, message='unsupported file format')
        productCnt = db.child('FarmersPortal').child('productCnt').get().val()
        if productCnt == None:
            productCnt = 0
        productCnt += 1
        db.child('FarmersPortal').child('productCnt').set(productCnt)
        productID = 'CRFT' + str(100 + productCnt)
        args['imgList'] = []
        for img in images:
            imgCnt = db.child('FarmersPortal').child('imgCnt').get().val()
            if imgCnt == None:
                imgCnt = 0
            imgCnt += 1
            db.child('FarmersPortal').child('imgCnt').set(imgCnt)
            imgName = 'IMG' + str(100 + imgCnt) + '.jpg'

    storage.child('FarmersPortal').child('productImage').child(productID).child(imgNa
me).put(img)
            imgUrl =
storage.child('FarmersPortal').child('productImage').child(productID).child(imgName).ge
t_url(None)
            args['imgList'].append({'imgName': imgName,'imgUrl' : imgUrl})
        args['totalRating'] = 0
```

```
        args['averageRating'] = 0
        args['ratedFarmerCnt'] = 0
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
            productList = {}
        productList[productID] = args
        db.child('FarmersPortal').child('productList').set(productList)
        if catID != 'CATOTH':
            categoryList[catID]['deleteStatus'] = 1
            db.child('FarmersPortal').child('categoryList').set(categoryList)
        return args

class ProductDataUpdate(Resource):
    def get(self, productID):
        productID = productID.upper()
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
            productList = {}
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
            farmerList = {}
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
            categoryList = {}
        if productID in productList:
            productDetails = productList[productID]
            if productDetails['catID'] == 'CATOTH':
                productDetails['categoryName'] = 'others'
```

```
                else:
                        productDetails['categoryName'] =
categoryList[productDetails['catID']]['name']
                productDetails['sellerDetails'] = farmerList[productDetails['farmerID']]
                productDetails['averageRating'] =
round(productDetails['averageRating'])
        else:
                abort(400, message='product not found')
        return productDetails

    def put(self, productID):
        productID = productID.upper()
        args = productDataUpdateParser.parse_args()
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
                productList = {}
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
                categoryList = {}
        if productID in productList:
                        if args['name']:
                                productList[productID]['name'] = args['name']
                        if args['rate']:
                                productList[productID]['rate'] = args['rate']
                        if args['stock']:
                                productList[productID]['stock'] = args['stock']
                        if args['description']:
                                productList[productID]['description'] = args['description']
```

```
                if args['catID']:
                        catID = args['catID'].upper()
                        if catID != 'CATOTH':
                                if not catID in categoryList:
                                        abort(400, message='category doesnt exist')
                        oldCatID = productList[productID]['catID']
                        if oldCatID != catID:
                                productList[productID]['catID'] = catID
                                if catID != 'CATOTH':
                                        categoryList[catID]['deleteStatus'] = 1

db.child('FarmersPortal').child('categoryList').set(categoryList)
                                if oldCatID != 'CATOTH':
                                        oldFlag = 0
                                        for i in productList:
                                                if productList[i]['catID'] == oldCatID:
                                                        oldFlag = 1
                                                        break
                                        if oldFlag == 0:
                                                categoryList[oldCatID]['deleteStatus'] = 0

db.child('FarmersPortal').child('categoryList').set(categoryList)
                db.child('FarmersPortal').child('productList').set(productList)
                productDetails= productList[productID]
        else:
                abort(400, message='product not found')
        return productDetails
```

```
    def delete(self, productID):
        productID = productID.upper()
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
            productList = {}
        if not productID in productList:
            abort(400, message='product not found')
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
            categoryList = {}
        catID = productList[productID]['catID']
        del productList[productID]
        db.child('FarmersPortal').child('productList').set(productList)
        if catID != 'CATOTH':
            flag = 0
            for i in productList:
                if productList[i]['catID'] == catID:
                    flag = 1
            if flag == 0:
                categoryList[catID]['deleteStatus'] = 0
                db.child('FarmersPortal').child('categoryList').set(categoryList)
        return productList

class ProductImgUpdate(Resource):
    def put(self,productID):
        productID = productID.upper()
        args = productImgUpdateParser.parse_args()
        if args['photo']:
```

```
productList = db.child('FarmersPortal').child('productList').get().val()
if productList == None:
    productList = {}
if productID in productList:
    images = args['photo']
    for img in images:
        if not allowed_file(img.filename):
            abort(400, message='unsupported file format')
    for img in images:
        imgCnt =
db.child('FarmersPortal').child('imgCnt').get().val()
        if imgCnt == None:
            imgCnt = 0
        imgCnt += 1
        db.child('FarmersPortal').child('imgCnt').set(imgCnt)
        imgName = 'IMG' + str(100 + imgCnt) + '.jpg'

    storage.child('FarmersPortal').child('productImage').child(productID).child(imgNa
me).put(img)
        imgUrl =
storage.child('FarmersPortal').child('productImage').child(productID).child(imgName).ge
t_url(None)
        productList[productID]['imgList'].append({'imgName' :
imgName, 'imgUrl' : imgUrl})
    db.child('FarmersPortal').child('productList').set(productList)
else:
    abort(400, message='product not found')
else:
```

```
            abort(400, message='photo required')
        return productList[productID]

    def delete(self,productID):
        productID = productID.upper()
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
            productList = {}
        args = productImgUpdateParser.parse_args()
        if args['imgName']:
            imgName = args['imgName'].upper()
            if productID in productList:
                if len(productList[productID]['imgList']) > 1:
                    flag = 0
                    for i in productList[productID]['imgList']:
                        if i['imgName'].upper() == imgName:
                            flag = 1
                            productList[productID]['imgList'].remove(i)

db.child('FarmersPortal').child('productList').set(productList)
                    if flag == 0:
                        abort(400, message='image not found')
                else:
                    abort(400, message='minimum one image required')
            else:
                abort(400, message='product not found')
        else:
            abort(400, message='image name required')
```

```
        return productList[productID]

class ProductList(Resource):
    def get(self, farmerID, cat):
        farmerID = farmerID.upper()
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
            productList = {}
        tempProductList = {}
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
            farmerList = {}
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
            categoryList = {}
        if farmerID == 'ADMIN':
            tempProductList = productList
        else:
            if farmerID in farmerList:
                if cat == 'own':
                    for i in productList:
                        if productList[i]['farmerID'] == farmerID:
                            tempProductList[i] = productList[i]
                elif cat == 'oth':
                    for i in productList:
                        if productList[i]['farmerID'] != farmerID:
                            if productList[i]['stock'] > 0:
                                tempProductList[i] = productList[i]
```

```
                        else:
                                abort(400, message='invalid category')
                else:
                        abort(400, message='invalid farmer')
        for i in tempProductList:
                if tempProductList[i]['catID'] == 'CATOTH':
                        tempProductList[i]['categoryName'] = 'others'
                else:
                        tempProductList[i]['categoryName'] =
categoryList[tempProductList[i]['catID']]['name']
                tempProductList[i]['sellerDetails'] =
farmerList[tempProductList[i]['farmerID']]
                tempProductList[i]['averageRating'] =
round(tempProductList[i]['averageRating'])
        return tempProductList

class FilterProductItemList(Resource):
    def post(self):
        args = filterProductItemParser.parse_args()
        farmerID = args['farmerID'].upper()
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
                productList = {}
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
                farmerList = {}
        tempProductList = {}
        if farmerID == 'FARMER':
```

```
                tempProductList = productList
                for i in tempProductList:
                        tempProductList[i]['sellerDetails'] =
farmerList[tempProductList[i]['farmerID']]
        else:
                if farmerID in farmerList:
                        for i in productList:
                                if productList[i]['farmerID'] != farmerID:
                                        tempProductList[i] = productList[i]
                                        tempProductList[i]['sellerDetails'] =
farmerList[productList[i]['farmerID']]
                else:
                        abort(400, message='farmer not found')
        filterItemList = {}
        filterItemList['location'] = locationlist(tempProductList)
        filterItemList['category'] = categorylist(tempProductList)
        filterItemList['rating'] = [5,4,3,2,1,0]
        return filterItemList

class FilterProduct(Resource):
    def post(self):
        args = filterProductParser.parse_args()
        farmerID = args['farmerID'].upper()
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
                productList = {}
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
```

```
        categoryList = {}
    farmerList = db.child('FarmersPortal').child('farmerList').get().val()
    if farmerList == None:
        farmerList = {}
    tempProductList = {}
    if farmerID == 'FARMER':
        tempProductList = productList
        for i in tempProductList:
            tempProductList[i]['sellerDetails'] =
farmerList[tempProductList[i]['farmerID']]
    else:
        if farmerID in farmerList:
            for i in productList:
                if productList[i]['farmerID'] != farmerID:
                    tempProductList[i] = productList[i]
                    tempProductList[i]['sellerDetails'] =
farmerList[productList[i]['farmerID']]
        else:
            abort(400, message='farmer not found')
    filterResult = {}
    if args['catID'] == 'location':
        locationList = locationlist(tempProductList)
        if args['catName'].lower() in locationList:
            for i in tempProductList:
                if tempProductList[i]['sellerDetails']['city'].lower() ==
args['catName'].lower():
                    filterResult[i] = tempProductList[i]
        else:
```

```
                abort(400, message='invalid category name')
        elif args['catID'] == 'category':
            category_list = categorylist(tempProductList)
            if args['catName'].upper() in category_list:
                for i in tempProductList:
                    if tempProductList[i]['catID'] == args['catName'].upper():
                        filterResult[i] = tempProductList[i]
            else:
                abort(400, message='invalid category name')
        elif args['catID'] == 'rate':
            if args['catName'].count('-') == 1:
                minLimit = args['catName'].split('-')[0]
                maxLimit = args['catName'].split('-')[1]
                if minLimit.isnumeric() and maxLimit.isnumeric():
                    minLimit = int(minLimit)
                    maxLimit = int(maxLimit)
                    if minLimit != maxLimit:
                        if minLimit < maxLimit:
                            for i in tempProductList:
                                if tempProductList[i]['rate'] >= minLimit
and tempProductList[i]['rate'] <= maxLimit:
                                    filterResult[i] = tempProductList[i]
                        else:
                            abort(400, message='minLimit should be less
than maxLimit')
                    else:
                        abort(400, message='minLimit and maxLimit should
not be equal')
```

```
                else:
                        abort(400, message='limit should be a integer')
            else:
                abort(400, message='invalid format')
        else:
            abort(400, message='invalid category id')
        for i in filterResult:
            filterResult[i]['averageRating'] = round(filterResult[i]['averageRating'])
            if filterResult[i]['catID'] == 'CATOTH':
                filterResult[i]['categoryName'] = 'others'
            else:
                filterResult[i]['categoryName'] =
categoryList[filterResult[i]['catID']]['name']
        return filterResult

class SearchProduct(Resource):
    def post(self):
        args = searchProductParser.parse_args()
        farmerID = args['farmerID'].upper()
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
            productList = {}
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
            categoryList = {}
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
            farmerList = {}
```

```
        tempProductList = {}
        if farmerID == 'FARMER':
            tempProductList = productList
            for i in tempProductList:
                tempProductList[i]['sellerDetails'] =
farmerList[tempProductList[i]['farmerID']]
        else:
            if farmerID in farmerList:
                for i in productList:
                    if productList[i]['farmerID'] != farmerID:
                        tempProductList[i] = productList[i]
                        tempProductList[i]['sellerDetails'] =
farmerList[productList[i]['farmerID']]
            else:
                abort(400, message='farmer not found')
        searchResult = {}
        if args['searchItem'] == '':
            abort(400, message='search item should not be empty')
        else:
            searchItem = args['searchItem'].lower()
        filterResult = {}
        if args['catID'] == 'nill':
            for i in tempProductList:
                productName = tempProductList[i]['name'].lower()
                if productName.startswith(searchItem):
                    searchResult[i] = tempProductList[i]
        elif args['catID'] == 'location':
            if args['catName']:
```

```
                        locationList = locationlist(tempProductList)
                        if args['catName'].lower() in locationList:
                                for i in tempProductList:
                                        if tempProductList[i]['sellerDetails']['city'].lower() ==
args['catName'].lower():
                                                filterResult[i] = tempProductList[i]
                                for i in filterResult:
                                        productName = filterResult[i]['name'].lower()
                                        if productName.startswith(searchItem):
                                                searchResult[i] = filterResult[i]
                        else:
                                abort(400, message='invalid category name')
                else:
                        abort(400, message='category name required')
        elif args['catID'] == 'category':
                if args['catName']:
                        category_list = categorylist(tempProductList)
                        if args['catName'].upper() in category_list:
                                for i in tempProductList:
                                        if tempProductList[i]['catID'] ==
args['catName'].upper():
                                                filterResult[i] = tempProductList[i]
                                for i in filterResult:
                                        productName = filterResult[i]['name'].lower()
                                        if productName.startswith(searchItem):
                                                searchResult[i] = filterResult[i]
                        else:
                                abort(400, message='invalid category name')
```

```
            else:
                abort(400, message='category name required')
        elif args['catID'] == 'rate':
            if args['catName']:
                if args['catName'].count('-') == 1:
                    minLimit = args['catName'].split('-')[0]
                    maxLimit = args['catName'].split('-')[1]
                    if minLimit.isnumeric() and maxLimit.isnumeric():
                        minLimit = int(minLimit)
                        maxLimit = int(maxLimit)
                        if minLimit != maxLimit:
                            if minLimit < maxLimit:
                                for i in tempProductList:
                                    if tempProductList[i]['rate'] >=
minLimit and tempProductList[i]['rate'] <= maxLimit:
                                        filterResult[i] =
tempProductList[i]
                                for i in filterResult:
                                    productName =
filterResult[i]['name'].lower()
                                    if
productName.startswith(searchItem):
                                        searchResult[i] =
filterResult[i]
                            else:
                                abort(400, message='minLimit should be
less than maxLimit')
                        else:
```

```
                                        abort(400, message='minLimit and maxLimit
should not be equal')
                                else:
                                        abort(400, message='limit should be a integer')
                        else:
                                abort(400, message='invalid format')
                else:
                        abort(400, message='category name required')
        else:
                abort(400, message= 'invalid category')
        for i in searchResult:
                searchResult[i]['averageRating'] =
round(searchResult[i]['averageRating'])
                if searchResult[i]['catID'] == 'CATOTH':
                        searchResult[i]['categoryName'] = 'others'
                else:
                        searchResult[i]['categoryName'] =
categoryList[searchResult[i]['catID']]['name']
        return searchResult

class ProductCart(Resource):
    def post(self, farmerID):
        farmerID = farmerID.upper()
        args = productCartParser.parse_args()
        productID = args['productID'].upper()
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
                productList = {}
```

```
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
            farmerList = {}
        productCart = db.child('FarmersPortal').child('productCart').get().val()
        if productCart == None:
            productCart = {}
        if not farmerID in farmerList:
            abort(400, message='farmer not found')
        if not productID in productList:
            abort(400, message='product not found')
        if productList[productID]['farmerID'] == farmerID:
            abort(400, message='product owner')
        if farmerID in productCart:
            if productID in productCart[farmerID]:
                productCart[farmerID][productID]['qty'] =
productCart[farmerID][productID]['qty'] + args['qty']
            else:
                productCart[farmerID][productID] = { 'qty' : args['qty'] }
        else:
            productCart[farmerID] = {
                productID : { 'qty' : args['qty'] }
                }
        db.child('FarmersPortal').child('productCart').set(productCart)
        return productCart[farmerID]

    def get(self, farmerID):
        farmerID = farmerID.upper()
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
```

```
        if farmerList == None:
            farmerList = {}
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
            categoryList = {}
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
            productList = {}
        productCart = db.child('FarmersPortal').child('productCart').get().val()
        if productCart == None:
            productCart = {}
        if not farmerID in farmerList:
            abort(400, message='farmer not found')
        farmerCart = {}
        if farmerID in productCart:
            farmerCart = productCart[farmerID]
            for i in farmerCart:
                farmerCart[i]['productDetails'] = productList[i]
                farmerCart[i]['productDetails']['averageRating'] =
round(farmerCart[i]['productDetails']['averageRating'])
                if farmerCart[i]['productDetails']['catID'] == 'CATOTH':
                    farmerCart[i]['productDetails']['categoryName'] = 'others'
                else:
                    farmerCart[i]['productDetails']['categoryName'] =
categoryList[farmerCart[i]['productDetails']['catID']]['name']
                farmerCart[i]['sellerDetails'] =
farmerList[farmerCart[i]['productDetails']['farmerID']]
        return farmerCart
```

```
class ProductCartUpdate(Resource):
    def get(self, farmerID, productID):
        farmerID = farmerID.upper()
        productID = productID.upper()
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
            farmerList = {}
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
            categoryList = {}
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
            productList = {}
        productCart = db.child('FarmersPortal').child('productCart').get().val()
        if productCart == None:
            productCart = {}
        if not farmerID in farmerList:
            abort(400, message='farmer not found')
        if not productID in productList:
            abort(400, message='product not found')
        cartItem = {}
        if farmerID in productCart:
            farmerCart = productCart[farmerID]
            if productID in farmerCart:
                cartItem = farmerCart[productID]
                cartItem['productDetails'] = productList[productID]
```

```
                        cartItem['productDetails']['averageRating'] =
round(cartItem['productDetails']['averageRating'])
                        if cartItem['productDetails']['catID'] == 'CATOTH':
                                cartItem['productDetails']['categoryName'] = 'others'
                        else:
                                cartItem['productDetails']['categoryName'] =
categoryList[cartItem['productDetails']['catID']]['name']
                        cartItem['sellerDeatials'] =
farmerList[cartItem['productDetails']['farmerID']]
            return cartItem

        def put(self, farmerID, productID):
            farmerID = farmerID.upper()
            productID = productID.upper()
            args = productCartUpdateParser.parse_args()
            farmerList = db.child('FarmersPortal').child('farmerList').get().val()
            if farmerList == None:
                farmerList = {}
            categoryList = db.child('FarmersPortal').child('categoryList').get().val()
            if categoryList == None:
                categoryList = {}
            productList = db.child('FarmersPortal').child('productList').get().val()
            if productList == None:
                productList = {}
            productCart = db.child('FarmersPortal').child('productCart').get().val()
            if productCart == None:
                productCart = {}
            if not farmerID in farmerList:
```

```
            abort(400, message='farmer not found')
        if not productID in productList:
            abort(400, message='product not found')
        flag = 0
        if farmerID in productCart:
            farmerCart = productCart[farmerID]
            if productID in farmerCart:
                flag = 1
                productCart[farmerID][productID]['qty'] = args['qty']
                cartItem = productCart[farmerID][productID]
                db.child('FarmersPortal').child('productCart').set(productCart)
                cartItem['productDetails'] = productList[productID]
                cartItem['productDetails']['averageRating'] =
round(cartItem['productDetails']['averageRating'])
                if cartItem['productDetails']['catID'] == 'CATOTH':
                    cartItem['productDetails']['categoryName'] = 'others'
                else:
                    cartItem['productDetails']['categoryName'] =
categoryList[cartItem['productDetails']['catID']]['name']
                cartItem['sellerDeatials'] =
farmerList[cartItem['productDetails']['farmerID']]
        if flag == 0:
            abort(400, message='item not found')
        return cartItem

    def delete(self, farmerID, productID):
        farmerID = farmerID.upper()
        productID = productID.upper()
```

```
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
            farmerList = {}
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
            categoryList = {}
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
            productList = {}
        productCart = db.child('FarmersPortal').child('productCart').get().val()
        if productCart == None:
            productCart = {}
        if not farmerID in farmerList:
            abort(400, message='farmer not found')
        if not productID in productList:
            abort(400, message='product not found')
        flag = 0
        if farmerID in productCart:
            farmerCart = productCart[farmerID]
            if productID in farmerCart:
                flag = 1
                del productCart[farmerID][productID]
                productCart =
db.child('FarmersPortal').child('productCart').set(productCart)
        if flag == 0:
            abort(400, message='item not found')
        farmerCart = {}
        if farmerID in productCart:
```

```
                farmerCart = productCart[farmerID]
                for i in farmerCart:
                        farmerCart[i]['productDetails'] = productList[i]
                        farmerCart[i]['productDetails']['averageRating'] =
round(farmerCart[i]['productDetails']['averageRating'])
                        if farmerCart[i]['productDetails']['catID'] == 'CATOTH':
                                farmerCart[i]['productDetails']['categoryName'] = 'others'
                        else:
                                farmerCart[i]['productDetails']['categoryName'] =
categoryList[farmerCart[i]['productDetails']['catID']]['name']
                        farmerCart[i]['sellerDeatials'] =
farmerList[farmerCart[i]['productDetails']['farmerID']]
        return farmerCart

class CostEstimation(Resource):
    def post(self, farmerID):
        farmerID = farmerID.upper()
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
                farmerList = {}
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
                categoryList = {}
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
                productList = {}
        productCart = db.child('FarmersPortal').child('productCart').get().val()
        if productCart == None:
```

```
            productCart = {}
        if not farmerID in farmerList:
            abort(400, message= 'farmer not found')
        farmerCart = {}
        grandTotal = 0
        if farmerID in productCart:
            farmerCart = productCart[farmerID]
            for i in farmerCart:
                farmerCart[i]['productDetails'] = productList[i]
                farmerCart[i]['productDetails']['averageRating'] =
round(farmerCart[i]['productDetails']['averageRating'])
                if farmerCart[i]['productDetails']['catID'] == 'CATOTH':
                    farmerCart[i]['productDetails']['categoryName'] = 'others'
                else:
                    farmerCart[i]['productDetails']['categoryName'] =
categoryList[farmerCart[i]['productDetails']['catID']]['name']
                farmerCart[i]['sellerDetails'] =
farmerList[farmerCart[i]['productDetails']['farmerID']]
                if farmerCart[i]['qty'] <= productList[i]['stock']:
                    farmerCart[i]['stockStatus'] = 'available'
                    farmerCart[i]['totalPrice'] = productList[i]['rate'] *
farmerCart[i]['qty']
                    grandTotal = grandTotal + farmerCart[i]['totalPrice']
                else:
                    farmerCart[i]['totalPrice'] = productList[i]['rate'] *
farmerCart[i]['qty']
                    farmerCart[i]['stockStatus'] = 'not available'
        cart = {
```

```
                'farmerCart'   : farmerCart,
                'grandTotal' : grandTotal
        }
        return cart

class PurchaseProduct(Resource):
    def post(self, farmerID):
        farmerID = farmerID.upper()
        farmerList = db.child('FarmersPortal').child('farmerList').get().val()
        if farmerList == None:
            farmerList = {}
        categoryList = db.child('FarmersPortal').child('categoryList').get().val()
        if categoryList == None:
            categoryList = {}
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
            productList = {}
        productCart = db.child('FarmersPortal').child('productCart').get().val()
        if productCart == None:
            productCart = {}
        tempProductCart = productCart.copy()
        if not farmerID in farmerList:
            abort(400, message= 'farmer not found')
        if not farmerID in tempProductCart:
            abort(400, message= 'empty cart, cant purchase. please add items to
cart')
        else:
            farmerCart = tempProductCart[farmerID]
```

```
            tempfarmerCart = farmerCart.copy()
        purchaseCnt = db.child('FarmersPortal').child('purchaseCnt').get().val()
        if purchaseCnt == None:
            purchaseCnt = 0
        purchaseCnt += 1
        db.child('FarmersPortal').child('purchaseCnt').set(purchaseCnt)
        purchsID = 'PURCHS' + str(100 + purchaseCnt)
        sampleHistroy = {}
        for i in tempFarmerCart:
            if tempFarmerCart[i]['qty'] <= productList[i]['stock']:
                transDetails = {}
                transactionCnt =
db.child('FarmersPortal').child('transactionCnt').get().val()
                if transactionCnt == None:
                    transactionCnt = 0
                transactionCnt += 1

    db.child('FarmersPortal').child('transactionCnt').set(transactionCnt)
                transID = 'TRANS' + str(100 + transactionCnt)
                x = datetime.now(timezone("Asia/Kolkata"))
                date = x.strftime('%d/%m/%Y')
                time = x.strftime('%X')
                transDetails = {
                    'purchsID'     : purchsID,
                    'productID'      : i,
                    'qty'          : tempFarmerCart[i]['qty'],
                    'totalPrice'    : tempFarmerCart[i]['qty'] *
productList[i]['rate'],
```

```
                    'date'         : date,
                    'time'         : time,
                    'productDetails'  : productList[i],
                    'sellerID'      : productList[i]['farmerID'],
                    'sellerDetails' : farmerList[productList[i]['farmerID']],
                    'buyerID'       : farmerID,
                    'buyerDetails' : farmerList[farmerID]
                }
                transDetails['productDetails']['averageRating'] =
round(transDetails['productDetails']['averageRating'])
                if transDetails['productDetails']['catID'] == 'CATOTH':
                    transDetails['productDetails']['categoryName'] = 'others'
                else:
                    transDetails['productDetails']['categoryName'] =
categoryList[transDetails['productDetails']['catID']]['name']
                transactionHistory =
db.child('FarmersPortal').child('transactionHistory').get().val()
                if transactionHistory == None:
                    transactionHistory = {}
                transactionHistory[transID] = transDetails
                sampleHistroy[transID] = transDetails

        db.child('FarmersPortal').child('transactionHistory').set(transactionHistory)
                productList[i]['stock'] = productList[i]['stock'] -
tempFarmerCart[i]['qty']
                db.child('FarmersPortal').child('productList').set(productList)
                del productCart[farmerID][i]
            db.child('FarmersPortal').child('productCart').set(productCart)
```

```
        return sampleHistroy

class TransactionHistory(Resource):
    def get(self):
        transactionHistory =
db.child('FarmersPortal').child('transactionHistory').get().val()
        if transactionHistory == None:
            transactionHistory = {}
        transKey = list(transactionHistory.keys())
        transKey.reverse()
        transHistory = {}
        for i in transKey:
            transHistory[i] = transactionHistory[i]
        return transHistory

class PurchaseHistory(Resource):
    def get(self, purchsID):
        purchsID = purchsID.upper()
        purchaseHistory = {}
        transactionHistory =
db.child('FarmersPortal').child('transactionHistory').get().val()
        if transactionHistory == None:
            transactionHistory = {}
        for i in transactionHistory:
            if transactionHistory[i]['purchsID'] == purchsID:
                purchaseHistory[i] = transactionHistory[i]
        return purchaseHistory
```

```
class TransactionDetails(Resource):
	def get(self, transID):
		transID = transID.upper()
		transDetails = {}
		transactionHistory =
db.child('FarmersPortal').child('transactionHistory').get().val()
		if transactionHistory == None:
			transactionHistory = {}
		if transID in transactionHistory:
			transDetails = transactionHistory[transID]
		else:
			abort(400, message='transaction not found')
		return transDetails

class FarmerTransactionHistory(Resource):
	def get(self, farmerID, cat):
		farmerID = farmerID.upper()
		farmerList = db.child('FarmersPortal').child('farmerList').get().val()
		if farmerList == None:
			farmerList = {}
		if not farmerID in farmerList:
			abort(400, message='farmer not found')
		transactionHistory =
db.child('FarmersPortal').child('transactionHistory').get().val()
		if transactionHistory == None:
			transactionHistory = {}
		tempTransactionHistory = {}
		for i in transactionHistory:
```

```
                if cat == 'all':
                        if transactionHistory[i]['sellerID'] == farmerID or
transactionHistory[i]['buyerID'] == farmerID:
                                tempTransactionHistory[i] = transactionHistory[i]
                elif cat == 'sell':
                        if transactionHistory[i]['sellerID'] == farmerID:
                                tempTransactionHistory[i] = transactionHistory[i]
                elif cat == 'buy':
                        if transactionHistory[i]['buyerID'] == farmerID:
                                tempTransactionHistory[i] = transactionHistory[i]
                else:
                        abort(400, message='invalid category')
        transKey = list(tempTransactionHistory.keys())
        transKey.reverse()
        transHistory = {}
        for i in transKey:
                transHistory[i] = tempTransactionHistory[i]
        return transHistory

class ProductTransactionHistory(Resource):
    def get(self, productID):
        productID = productID.upper()
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
                productList = {}
        if not productID in productList:
                abort(400, message='product not found')
```

```
        transactionHistory =
db.child('FarmersPortal').child('transactionHistory').get().val()
        if transactionHistory == None:
            transactionHistory = {}
        tempTransactionHistory = {}
        for i in transactionHistory:
            if transactionHistory[i]['productID'] == productID:
                tempTransactionHistory[i] = transactionHistory[i]
        transKey = list(tempTransactionHistory.keys())
        transKey.reverse()
        transHistory = {}
        for i in transKey:
            transHistory[i] = tempTransactionHistory[i]
        return transHistory

class ProductApprove(Resource):
    def post(self, productID):
        args = productApproveParser.parse_args()
        productID = productID.upper()
        productList = db.child('FarmersPortal').child('productList').get().val()
        if productList == None:
            abort(400, message='there is no product registered yet')
        if productID in productList:
            if args['action'] == 'approve':
                if productList[productID]['approvalStatus'] == 0:
                    productList[productID]['approvalStatus'] = 1
                    productList[productID]['activeStatus'] = 1
```

```
        db.child('FarmersPortal').child('productList').set(productList)
                        else:
                                abort(400, message='product is approved already')
                elif args['action'] == 'reject':
                        if productList[productID]['approvalStatus'] == 1:
                                productList[productID]['approvalStatus'] = 0
                                productList[productID]['activeStatus'] = 0

        db.child('FarmersPortal').child('productList').set(productList)
                        else:
                                abort(400, message='product is rejected already')
                else:
                        abort(400, message='invalid action')
            else:
                abort(400, message='product doesnt exist')
            return productList[productID]

api.add_resource(FarmerReg, '/farmer')
api.add_resource(FarmerUpdate, '/farmer/<farmerID>')
api.add_resource(FarmerLogin, '/farmerLogin')
api.add_resource(AdminLogin, '/adminLogin')
api.add_resource(CategoryReg, '/category')
api.add_resource(CategoryUpdate, '/category/<catID>')
api.add_resource(ProductReg, '/product')
api.add_resource(ProductDataUpdate, '/product/<productID>')
api.add_resource(ProductImgUpdate, '/productImg/<productID>')
api.add_resource(ProductList, '/productList/<farmerID>/<cat>')
```

```
api.add_resource(FilterProductItemList, '/filterProductItem')
api.add_resource(FilterProduct, '/filterProduct')
api.add_resource(SearchProduct, '/searchProduct')
api.add_resource(ProductCart, '/productCart/<farmerID>')
api.add_resource(ProductCartUpdate, '/ProductCartUpdate/<farmerID>/<productID>')
api.add_resource(CostEstimation, '/costEstimation/<farmerID>')
api.add_resource(PurchaseProduct, '/purchaseProduct/<farmerID>')
api.add_resource(TransactionHistory, '/transactionHistory')
api.add_resource(PurchaseHistory, '/purchaseHistory/<purchsID>')
api.add_resource(TransactionDetails, '/transDetails/<transID>')
api.add_resource(FarmerTransactionHistory, '/transactionHistory/<farmerID>/<cat>')
api.add_resource(ProductTransactionHistory,
'/productTransactionHistory/<productID>')

if __name__ == '__main__':
        app.run(debug=True, host='0.0.0.0', port=8000)

from flask import Flask, render_template, redirect, url_for, request, make_response,
session
import requests

app = Flask(__name__)
app.secret_key = "abc"

base_url = 'http://127.0.0.1:8000'

@app.route('/')
def homePage():
```

```
    if 'farmer' in session:
        farmerID = session['farmer']
        response = requests.get(base_url + '/farmer/' + farmerID)
        farmerDetails = response.json()
        response = requests.get(base_url + '/productList/' + farmerID + '/oth')
        othProductList = response.json()
        response = requests.post(base_url + '/filterProductItem',data={'farmerID' :
farmerID})
        filterItemList = response.json()
        locationList = filterItemList['location']
        categoryList = filterItemList['category']
        response = requests.post(base_url + '/costEstimation/' + farmerID)
        costEstimation = response.json()
        farmerCart = costEstimation['farmerCart']
        return render_template('farmer_home.html', farmerDetails=farmerDetails,
othProductList=othProductList, locationList=locationList, categoryList=categoryList,
catID='nill', catName='nill', farmerCart=farmerCart)
    else:
        response = requests.get(base_url + '/productList/admin/nil')
        productList = response.json()
        response = requests.post(base_url + '/filterProductItem',data={'farmerID' :
'farmer'})
        filterItemList = response.json()
        locationList = filterItemList['location']
        categoryList = filterItemList['category']
        return render_template('home.html', productList=productList,
locationList=locationList, categoryList=categoryList, catID='nill', catName='nill')
```

```
@app.route('/home1', methods=['POST', 'GET'])
def filterHomePage():
	if request.method == 'GET':
		userInput = request.args.to_dict()
		if userInput['catID'] == 'rate':
			userInput['catName'] = userInput['minLimit'] + '-' +
userInput['maxLimit']
		if 'farmer' in session:
			farmerID = session['farmer']
			userInput['farmerID'] = farmerID
			response = requests.get(base_url + '/farmer/' + farmerID)
			farmerDetails = response.json()
			response = requests.post(base_url + '/filterProduct',data=userInput)
			if response.status_code != 400:
				othProductList = response.json()
			else:
				othProductList = {}
			response = requests.post(base_url +
'/filterProductItem',data={'farmerID' : farmerID})
			filterItemList = response.json()
			locationList = filterItemList['location']
			categoryList = filterItemList['category']
			response = requests.post(base_url + '/costEstimation/' + farmerID)
			costEstimation = response.json()
			farmerCart = costEstimation['farmerCart']
			return render_template('farmer_home.html',
farmerDetails=farmerDetails, othProductList=othProductList, locationList=locationList,
```

```
categoryList=categoryList, catID=userInput['catID'], catName=userInput['catName'],
farmerCart=farmerCart)
            else:
                response = requests.post(base_url + '/filterProduct',data=farmerInput)
                if response.status_code != 400:
                    productList = response.json()
                else:
                    productList = {}
                response = requests.post(base_url +
'/filterProductItem',data={'farmerID' : 'farmer'})
                filterItemList = response.json()
                locationList = filterItemList['location']
                categoryList = filterItemList['category']
                return render_template('home.html', productList=productList,
locationList=locationList, categoryList=categoryList, catID=userInput['catID'],
catName=userInput['catName'])

@app.route('/home2', methods=['POST', 'GET'])
def searchHomePage():
 if request.method == 'GET':
        userInput = request.args.to_dict()
        if 'farmer' in session:
            farmerID = session['farmer']
            userInput['farmerID'] = farmerID
            response = requests.get(base_url + '/farmer/' + farmerID)
            farmerDetails = response.json()
            response = requests.post(base_url + '/searchProduct',data=userInput)
            if response.status_code != 400:
```

```
                othProductList = response.json()
            else:
                othProductList = {}
            response = requests.post(base_url + '/filterProductItem',data={'farmerID' :
farmerID})
            filterItemList = response.json()
            locationList = filterItemList['location']
            categoryList = filterItemList['category']
            response = requests.post(base_url + '/costEstimation/' + farmerID)
            costEstimation = response.json()
            farmerCart = costEstimation['farmerCart']
            return render_template('farmer_home.html', farmerDetails=farmerDetails,
othProductList=othProductList, locationList=locationList, categoryList=categoryList,
catID=farmerInput['catID'], catName=userInput['catName'], farmerCart=farmerCart)
        else:
            response = requests.post(base_url + '/searchProduct',data=userInput)
            if response.status_code != 400:
                productList = response.json()
            else:
                productList = {}
            response = requests.post(base_url + '/filterProductItem',data={'farmerID' :
'farmer'})
            filterItemList = response.json()
            locationList = filterItemList['location']
            categoryList = filterItemList['category']
            return render_template('home.html', productList=productList,
locationList=locationList, categoryList=categoryList, catID=userInput['catID'],
catName=userInput['catName'])
```

```
@app.route('/admin')
def adminHomePage():
	if 'admin' in session:
		response = requests.get(base_url + '/productList/ADMIN/nil')
		productList = response.json()
		return render_template('admin_home.html', productList=productList)
	else:
		return render_template('login.html', session='admin')

@app.route('/othProductProfile/<productID>')
def othProductProfile(productID):
	if 'farmer' in session:
		farmerID = session['farmer']
		response = requests.get(base_url + '/farmer/' + farmerID)
		farmerDetails = response.json()
		response = requests.get(base_url + '/product/' + productID)
		productDetails = response.json()
		response = requests.get(base_url + '/farmer/' + productDetails['farmerID'])
		othFarmerDetails = response.json()
		othID = productDetails['farmerID']
		response = requests.post(base_url + '/costEstimation/' + farmerID)
		costEstimation = response.json()
		farmerCart = costEstimation['farmerCart']
		return render_template('oth_product_profile.html', farmerID=farmerID,
farmerDetails=farmerDetails, productID=productID, productDetails=productDetails,
othFarmerDetails=othFarmerDetails, othID=othID, farmerCart=farmerCart)
	else:
```

```
        response = requests.get(base_url + '/product/' + productID)
        productDetails = response.json()
        return render_template('general_product_profile.html',
productID=productID, productDetails=productDetails)

@app.route('/loginPage/<productID>')
def loginPage(productID):
    return render_template('login.html', session='farmer', productID=productID)

@app.route('/addToCart', methods=['POST', 'GET'])
def addToCart():
    if request.method == 'POST':
        userInput = request.form.to_dict()
        if 'farmer' in session:
            farmerID = session['farmer']
            response = requests.post(base_url + '/productCart/' + farmerID,
data=userInput)
            return
redirect(url_for('othProductProfile',productID=userInput['productID']))
        else:
            return redirect(url_for('loginPage',productID=userInput['productID']))

@app.route('/signup', methods=['POST', 'GET'])
def signup():
    if request.method == 'POST':
        userInput = request.form.to_dict()
        farmerImg = request.files['photo']
        if farmerImg:
```

```
                files = {'photo' : (farmerImg.filename, farmerImg,
farmerImg.content_type)}
                response = requests.post(base_url + '/farmer', data=userInput,
files=files)
        else:
                response=requests.post(base_url + '/farmer', data=userInput)
        return redirect(url_for('homePage'))

@app.route('/login', methods=['POST', 'GET'])
def login():
    if request.method == 'POST':
        userInput = request.form.to_dict()
        if userInput['session'] == 'farmer':
                response = requests.post(base_url + '/farmerLogin', data=userInput)
                if response.json() != '':
                        farmerID = response.json()
                        session['farmer'] = farmerID
                        if userInput['productID'] == 'nill':
                                return redirect(url_for('homePage'))
                        else:
                                response = requests.get(base_url + '/product/' +
userInput['productID'])
                                productDetails = response.json()
                                if productDetails['farmerID'] != farmerID:
                                        return redirect(url_for('othProductProfile',
productID= userInput['productID']))
                                else:
                                        return redirect(url_for('homePage'))
```

```
                else:
                        return redirect(url_for('loginPage',
productID=userInput['productID']))
        elif userInput['session'] == 'admin':
                response = requests.post(base_url + '/adminLogin', data=userInput)
                if response.json() == 'admin':
                        session['admin'] = response.json()
                return redirect(url_for('adminHomePage'))

@app.route('/logout')
def logout():
        if 'farmer' in session:
                session.pop('farmer', None)
        return redirect(url_for('homePage'))

@app.route('/adminLogout')
def adminLogout():
        if 'admin' in session:
                session.pop('admin', None)
        return redirect(url_for('adminHomePage'))

@app.route('/profile/<flag>')
def profile(flag):
        if 'farmer' in session:
                if flag == 'own':
                        farmerID = session['farmer']
                else:
                        farmerID = flag
```

```
            response = requests.get(base_url + '/farmer/' + farmerID)
            farmerDetails = response.json()
            response = requests.post(base_url + '/costEstimation/' + farmerID)
            costEstimation = response.json()
            farmerCart = costEstimation['farmerCart']
            return render_template('farmer_profile.html', farmerDetails=farmerDetails,
flag=flag, farmerCart=farmerCart)
        else:
            return redirect(url_for('homePage'))

@app.route('/editProfile')
def editProfile():
        if 'farmer' in session:
            farmerID = session['farmer']
            response = requests.get(base_url + '/farmer/' + farmerID)
            farmerDetails = response.json()
            response = requests.post(base_url + '/costEstimation/' + farmerID)
            costEstimation = response.json()
            farmerCart = costEstimation['farmerCart']
            return render_template('farmer_edit_profile.html',
farmerDetails=farmerDetails, farmerCart=farmerCart)
        else:
            return redirect(url_for('homePage'))

@app.route('/updateProfile', methods=['POST','GET'])
def updateProfile():
        if 'farmer' in session:
            if request.method == 'POST':
```

```
                farmerID = session['farmer']
                userInput = request.form.to_dict()
                farmerImg = request.files['photo']
                if farmerImg:
                        files = {'photo' : (farmerImg.filename, farmerImg,
farmerImg.content_type)}
                        requests.put(base_url + '/farmer/' + farmerID, files=files)
                requests.put(base_url + '/farmer/' + farmerID, data=userInput)
                return redirect(url_for('profile', flag='own'))
        else:
                return redirect(url_for('homePage'))

@app.route('/farmerList')
def farmerList():
        if 'admin' in session:
                response=requests.get(base_url + '/farmer')
                farmerList = response.json()
                return render_template('admin_farmer_list.html', farmerList=farmerList)
        else:
                return redirect(url_for('adminHomePage'))

@app.route('/addProduct')
def addProduct():
        if 'farmer' in session:
                farmerID = session['farmer']
                response = requests.get(base_url + '/farmer/' + farmerID)
                farmerDetails = response.json()
                response = requests.get(base_url + '/category')
```

```
            categoryList = response.json()
            response = requests.post(base_url + '/costEstimation/' + farmerID)
            costEstimation = response.json()
            farmerCart = costEstimation['farmerCart']
            return render_template('farmer_add_product.html',
farmerDetails=farmerDetails, categoryList=categoryList, farmerCart=farmerCart)
        else:
            return redirect(url_for('homePage'))

@app.route('/productReg', methods=['POST', 'GET'])
def productReg():
        if 'farmer' in session:
            if request.method == 'POST':
                userInput = request.form.to_dict()
                userInput['farmerID'] = session['farmer']
                images = request.files.getlist('photo')
                mulFiles = []
                for img in images:
                    mulFiles.append(('photo', (img.filename, img, img.content_type)))
                response = requests.post(base_url + '/product', data=userInput,
files=mulFiles)
                return redirect(url_for('addProduct'))
        else:
            return redirect(url_for('homePage'))

@app.route('/showProducts')
def showProducts():
        if 'farmer' in session:
```

```
        farmerID = session['farmer']
        response = requests.get(base_url + '/farmer/' + farmerID)
        farmerDetails = response.json()
        response = requests.get(base_url + '/productList/' + farmerID + '/own')
        productList = response.json()
        response = requests.post(base_url + '/costEstimation/' + farmerID)
        costEstimation = response.json()
        farmerCart = costEstimation['farmerCart']
        return render_template('farmer_show_products.html',
farmerDetails=farmerDetails, productList=productList, farmerCart=farmerCart)
    else:
        return redirect(url_for('homePage'))

@app.route('/productProfile/<productID>')
def productProfile(productID):
    if 'farmer' in session:
        farmerID = session['farmer']
        response = requests.get(base_url + '/farmer/' + farmerID)
        farmerDetails = response.json()
        response = requests.get(base_url + '/product/' + productID)
        productDetails = response.json()
        response = requests.get(base_url + '/productTransactionHistory/' +
productID)
        transHistory = response.json()
        response = requests.post(base_url + '/costEstimation/' + farmerID)
        costEstimation = response.json()
        farmerCart = costEstimation['farmerCart']
```

```
        return render_template('product_profile.html', farmerDetails=farmerDetails,
productID=productID, productDetails=productDetails, transHistory=transHistory,
farmerCart=farmerCart)
    else:
        return redirect(url_for('homePage'))

@app.route('/deleteProduct', methods=['POST', 'GET'])
def deleteProduct():
    if 'farmer' in session:
        if request.method == 'POST':
            userInput = request.form.to_dict()
            requests.delete(base_url + '/product/' + userInput['productID'])
            return redirect(url_for('showProducts'))
    else:
        return redirect(url_for('homePage'))

@app.route('/adminDeleteProduct', methods=['POST', 'GET'])
def adminDeleteProduct():
    if 'admin' in session:
        if request.method == 'POST':
            userInput = request.form.to_dict()
            requests.delete(base_url + '/product/' + userInput['productID'])
        return redirect(url_for('adminHomePage'))

@app.route('/editProductDetails/<productID>')
def editProductDetails(productID):
    if 'farmer' in session:
        farmerID = session['farmer']
```

```
        response = requests.get(base_url + '/farmer/' + farmerID)
        farmerDetails = response.json()
        response = requests.get(base_url + '/product/' + productID)
        productDetails = response.json()
        response = requests.get(base_url + '/category')
        categoryList = response.json()
        response = requests.post(base_url + '/costEstimation/' + farmerID)
        costEstimation = response.json()
        farmerCart = costEstimation['farmerCart']
        return render_template('farmer_edit_product.html',
farmerDetails=farmerDetails, productID=productID, productDetails=productDetails,
categoryList=categoryList, farmerCart=farmerCart)
    else:
        return redirect(url_for('homePage'))

@app.route('/updateProductData', methods=['POST', 'GET'])
def updateProductData():
    if 'farmer' in session:
        if request.method == 'POST':
            userInput = request.form.to_dict()
            requests.put(base_url + '/product/' + userInput['productID'],
data=userInput)
            return redirect(url_for('productProfile',
productID=userInput['productID']))
        else:
            return redirect(url_for('homePage'))

@app.route('/deleteProductImg', methods=['POST', 'GET'])
```

```
def deleteProductImg():
	if 'farmer' in session:
		if request.method == 'POST':
			userInput = request.form.to_dict()
			requests.delete(base_url + '/productImg/' + userInput['productID'],
data=userInput)
			return redirect(url_for('editProductDetails',
productID=userInput['productID']))
		else:
			return redirect(url_for('homePage'))

@app.route('/addProductImg', methods=['POST', 'GET'])
def addProductImg():
	if 'farmer' in session:
		if request.method == 'POST':
			userInput = request.form.to_dict()
			images = request.files.getlist('photo')
			mulFiles = []
			for img in images:
				mulFiles.append(('photo', (img.filename, img, img.content_type)))
			requests.put(base_url + '/productImg/' + userInput['productID'],
files=mulFiles)
			return redirect(url_for('editProductDetails',
productID=userInput['productID']))
		else:
			return redirect(url_for('homePage'))

@app.route('/farmerHistory')
```

```
def farmerHistory():
    if 'farmer' in session:
        farmerID = session['farmer']
        response = requests.get(base_url + '/farmer/' + farmerID)
        farmerDetails = response.json()
        response = requests.get(base_url + '/transactionHistory/' + farmerID + '/all')
        allTransHistory = response.json()
        response = requests.get(base_url + '/transactionHistory/' + farmerID + '/sell')
        sellTransHistory = response.json()
        response = requests.get(base_url + '/transactionHistory/' + farmerID + '/buy')
        buyTransHistory = response.json()
        response = requests.post(base_url + '/costEstimation/' + farmerID)
        costEstimation = response.json()
        farmerCart = costEstimation['farmerCart']
        return render_template('farmer_history.html',
farmerDetails=farmerDetails,allTransHistory=allTransHistory,
sellTransHistory=sellTransHistory, buyTransHistory=buyTransHistory,
farmerCart=farmerCart)
    else:
        return redirect(url_for('homePage'))

@app.route('/adminHistory')
def adminHistory():
    if 'admin' in session:
        response = requests.get(base_url + '/transactionHistory')
        transHistory = response.json()
        return render_template('admin_history.html', transHistory=transHistory)
    else:
```

```
        return redirect(url_for('adminHomePage'))

@app.route('/adminTransDetails/<transID>')
def adminTransDetails(transID):
    if 'admin' in session:
        response = requests.get(base_url + '/transDetails/' + transID)
        transDetails = response.json()
        return render_template('trans_details.html', transDetails=transDetails,
session='admin')
    else:
        return redirect(url_for('adminHomePage'))

@app.route('/farmerTransDetails/<transID>')
def farmerTransDetails(transID):
    if 'farmer' in session:
        farmerID = session['farmer']
        response = requests.get(base_url + '/farmer/' + farmerID)
        farmerDetails = response.json()
        response = requests.get(base_url + '/transDetails/' + transID)
        transDetails = response.json()
        response = requests.post(base_url + '/costEstimation/' + farmerID)
        costEstimation = response.json()
        farmerCart = costEstimation['farmerCart']
        return render_template('trans_details.html', farmerID=farmerID,
farmerDetails=farmerDetails, transDetails=transDetails, session='farmer',
farmerCart=farmerCart)
    else:
        return redirect(url_for('adminHomePage'))
```

```
@app.route('/adminProductProfile/<productID>')
def adminProductProfile(productID):
	if 'admin' in session:
		response = requests.get(base_url + '/product/' + productID)
		productDetails = response.json()
		response = requests.get(base_url + '/farmer/' + productDetails['farmerID'])
		sellerDetails = response.json()
		response = requests.get(base_url + '/productTransactionHistory/' +
productID)
		transHistory = response.json()
		return render_template('admin_product_profile.html',
productID=productID, productDetails=productDetails, sellerDetails=sellerDetails,
transHistory=transHistory)
	else:
		return redirect(url_for('adminHomePage'))

@app.route('/adminFarmerProfile/<farmerID>')
def adminFarmerProfile(farmerID):
	if 'admin' in session:
		response = requests.get(base_url + '/farmer/' + farmerID)
		farmerDetails = response.json()
		response = requests.get(base_url + '/transactionHistory/' + farmerID + '/all')
		transHistory = response.json()
		response = requests.get(base_url + '/productList/' + farmerID + '/own')
		productList = response.json()
		response = requests.post(base_url + '/costEstimation/' + farmerID)
		costEstimation = response.json()
```

```
		farmerCart = costEstimation['farmerCart']
		grandTotal = costEstimation['grandTotal']
		return render_template('admin_farmer_profile.html', farmerID=farmerID,
farmerDetails=farmerDetails, transHistory=transHistory, productList=productList,
farmerCart=farmerCart, grandTotal=grandTotal)
	else:
		return redirect(url_for('adminHomePage'))

@app.route('/category')
def viewCategory():
	if 'admin' in session:
		response = requests.get(base_url + '/category', timeout=5)
		categoryList = response.json()
		return render_template('category_list.html', categoryList=categoryList)
	else:
		return redirect(url_for('adminHomePage'))

@app.route('/updateCategory', methods=['POST', 'GET'])
def updateCategory():
	if 'admin' in session:
		if request.method == 'POST':
			userInput = request.form.to_dict()
			response = requests.put(base_url + '/category/' + userInput['catID'],
data={'name':userInput['name']})
			return redirect(url_for('viewCategory'))
	else:
		return redirect(url_for('adminHomePage'))
```

```
@app.route('/deleteCategory', methods=['POST', 'GET'])
def deleteCategory():
	if 'admin' in session:
		if request.method == 'POST':
			userInput = request.form.to_dict()
			response = requests.delete(base_url + '/category/' + userInput['catID'] )
			return redirect(url_for('viewCategory'))
	else:
		return redirect(url_for('adminHomePage'))

@app.route('/addCategory', methods=['POST', 'GET'])
def addCategory():
	if 'admin' in session:
		if request.method == 'POST':
			userInput = request.form.to_dict()
			response = requests.post(base_url + '/category',
data={'name':userInput['name']})
			return redirect(url_for('viewCategory'))
	else:
		return redirect(url_for('adminHomePage'))

@app.route('/detailedCart')
def detailedCart():
	if 'farmer' in session:
		farmerID = session['farmer']
		response = requests.get(base_url + '/farmer/' + farmerID)
		farmerDetails = response.json()
		response = requests.post(base_url + '/costEstimation/' + farmerID)
```

```
            costEstimation = response.json()
            farmerCart = costEstimation['farmerCart']
            grandTotal = costEstimation['grandTotal']
            return render_template('detailed_cart.html', farmerDetails=farmerDetails,
farmerCart=farmerCart, grandTotal=grandTotal)
        else:
            return redirect(url_for('homePage'))

@app.route('/updateCartItem', methods=['POST', 'GET'])
def updateCartItem():
        if 'farmer' in session:
            if request.method == 'POST':
                farmerID = session['farmer']
                userInput = request.form.to_dict()
                response = requests.put(base_url + '/productCartUpdate/' + farmerID +
'/' + userInput['productID'], data=userInput)
                return redirect(url_for('detailedCart'))
        else:
            return redirect(url_for('homePage'))

@app.route('/deleteCartItem', methods=['POST', 'GET'])
def deleteCartItem():
        if 'farmer' in session:
            if request.method == 'POST':
                farmerID = session['farmer']
                userInput = request.form.to_dict()
                response = requests.delete(base_url + '/productCartUpdate/' +
farmerID + '/' + userInput['productID'])
```

```
                return redirect(url_for('detailedCart'))
        else:
                return redirect(url_for('homePage'))

@app.route('/purchaseDetails')
def purchaseDetails():
        if 'farmer' in session:
                farmerID = session['farmer']
                response = requests.get(base_url + '/farmer/' + farmerID)
                farmerDetails = response.json()
                response = requests.post(base_url + '/costEstimation/' + farmerID)
                costEstimation = response.json()
                farmerCart = costEstimation['farmerCart']
                grandTotal = costEstimation['grandTotal']
                return render_template('purchase_details.html',
farmerDetails=farmerDetails, farmerCart=farmerCart, grandTotal=grandTotal)
        else:
                return redirect(url_for('homePage'))

@app.route('/purchaseProduct', methods=['POST', 'GET'])
def purchaseProduct():
        if 'farmer' in session:
                if request.method == 'POST':
                        farmerID = session['farmer']
                        response = requests.post(base_url + '/purchaseProduct/' + farmerID)
                        return redirect(url_for('farmerHistory'))
        else:
                return redirect(url_for('homePage'))
```

```
@app.route('/approveProduct', methods=['POST', 'GET'])
def approveProduct():
	if 'admin' in session:
		if request.method == 'POST':
			userInput = request.form.to_dict()
			requests.post(base_url + '/productApprove/' + userInput['productID'],
data={'action' : 'approve'})
			return redirect(url_for('adminProfileProfile',
productID=userInput['productID'], flag=1))
	else:
		return redirect(url_for('adminHomePage'))

@app.route('/blockProduct', methods=['POST', 'GET'])
def blockProduct():
	if 'admin' in session:
		if request.method == 'POST':
			userInput = request.form.to_dict()
			requests.post(base_url + '/productApprove/' + userInput['productID'],
data={'action' : 'reject'})
			return redirect(url_for('adminProductProfile',
productID=userInput['productID'], flag=1))
	else:
		return redirect(url_for('adminHomePage'))

if __name__ == '__main__':
	app.run(debug=True, host='0.0.0.0')
```

7

TESTING

TESTING

Software testing is a critical element of software quality assurance and represents the ultimatereview of the specification, design and coding. Testing ensure that the system as a whole is bug free. Testing perform a critical role for quality assurance and for ensuring the reliability of the software.

In order to find a highest possible number of errors, tests must be conducted systematically and test cases must be designed using disciplined and techniques. The objective of the software testing is to uncover errors. To fulfill these objectives, a series of testing strategies are used.

They are,

- **U**nit Testing
- **I**ntegration Testing
- **S**ystem Testing
- **A**cceptance Testing
- **V**alidation Testing

Unit Testing:

Unit testing is carried out to screen wise, each screen being identified as an object. Attention is diverted to individual modules, independently to one another to locate errors in coding and logic.

In unit testing,

- Module interface is tested to ensure that information properly flows into and out of the program under test.
- Local data structures are examined to ensure that data stored temporarily maintains its integrity during all steps in algorithm execution.
- Boundary condition is tested to ensure that the module operates properly at boundaries established to limit or restricted processing.
- All independent paths through the control structures are executed to ensure that all statements in the module have been executed at least once.
- Error handling paths are also tested. This test focuses verification effort on the smallest unit of software design, the module. Here the module interface, boundary conditions, and all independent paths were verified by inputting false data. Each single operation is tested individually for its correct functionality

INTEGRATION TESTING

Integration testing is a systematic technique for the constructing the program structure while at the same time conducting tests to uncover errors associated with interfacing. Unit tested module were taken and a single program structure was build that has been dictated by the design. Incremental integration has adopted here. The entire software was developed and tested in a small segment, where errors were easy to locate and rectify. Each database or table manipulation operation was written as a single java class files and separately tested and combined form a single program. After integration, the single program was tested again with numerous test data to check for its functionality.

The integration can be performed in two ways:

- Top down integration.
- Bottom up integration

Here bottom up integration is performed. This strategy is implemented with the following steps.

1. Low-level modules are combined to form clusters that perform a specific software sub function.
2. The cluster is tested.
3. Drivers are removed and clusters are combined moving upward in the program structure.

VALIDATION TESTING

After performing validation testing, the software was completely assembled as package, interfacing errors have been uncovered and corrected, and a final series of software validation testing began. Here we test the system function in a manner that can be reasonable expected by the customer; the system was tested against the system requirement specification.

OUTPUT TESTING

After performing validation test, the next phase is output test of the system, since no system could be useful if it is does not produce the desired output in the desired format. By considered the format is considered in to two ways: one is on the screen and the other is on the printed form.

SYSTEM TESTING

System testing mainly concerned with the software-based systems. This actually a series of different tests whose primary purpose is to fully exercise the computer-based systems. Although each test as different purpose all work should be verify that all system elements have been properly integrated and perform allocated functions.

ACCEPTANCE TESTING

User acceptance test of a system is the key factor for the success of the system. The system under consideration was listed for the user acceptance by keeping constant touch with the perspective user of the system was at the time design, development and making change whenever required. This is done with the regards of the following points:

- Input screen design
- Output design
- Outline message
- Menu driven systems
- Formats of reports

TEST CASES

LOGIN

Test Case Name	Login
Test Case Description	Test whether user can login to the system without giving proper details
Item(s) to be Tested	
1	Mobile number
2	Password

Specification

Input	**Expected output/Result**
1.Valid username and password	1.User is directed to their respective home page
2.Valid username and Invalid password	2.Message: Invalid mobile number or password
3.Invalid username and valid password	3. Message: Invalid mobile number or password
4.Invalid username and Invalid password	4.Message: Invalid mobile number or password
5.Empty field	5.Meassage: please fill the fields

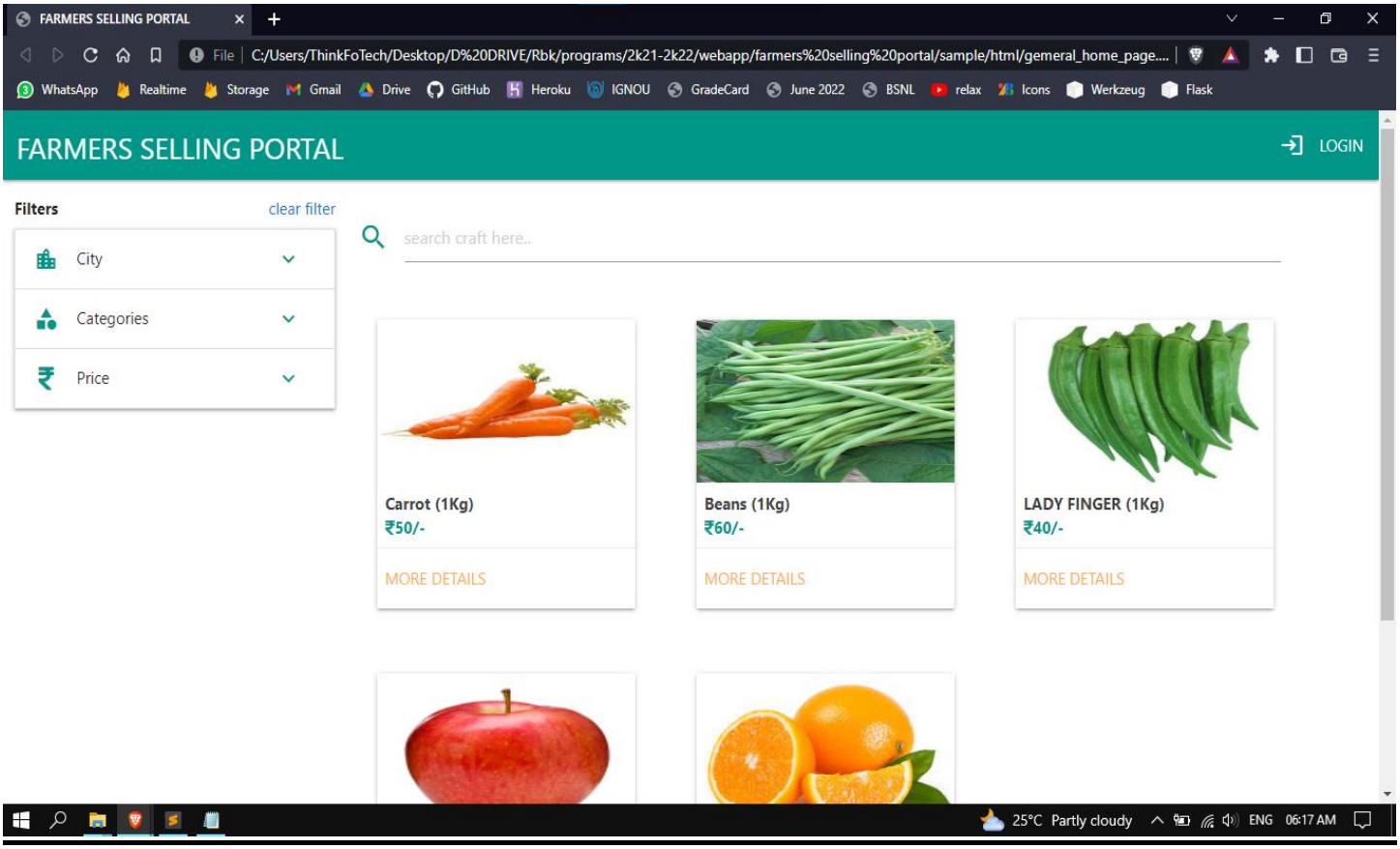
REGISTRATION

Test Case Name	New registration of user
Test Case Description	1.Test whether user details are valid and correct format
Item(s) to be Tested	
1	Inputs are stored correctly
Specification	
Input	**Expected output/Result**
1.All entries are filled	1.Message: Successfully registered
2. User has left certain fields empty.	2.Message: please fill the fields
3.Fields are entered not in correct format	3.Message: please enter value in correct format

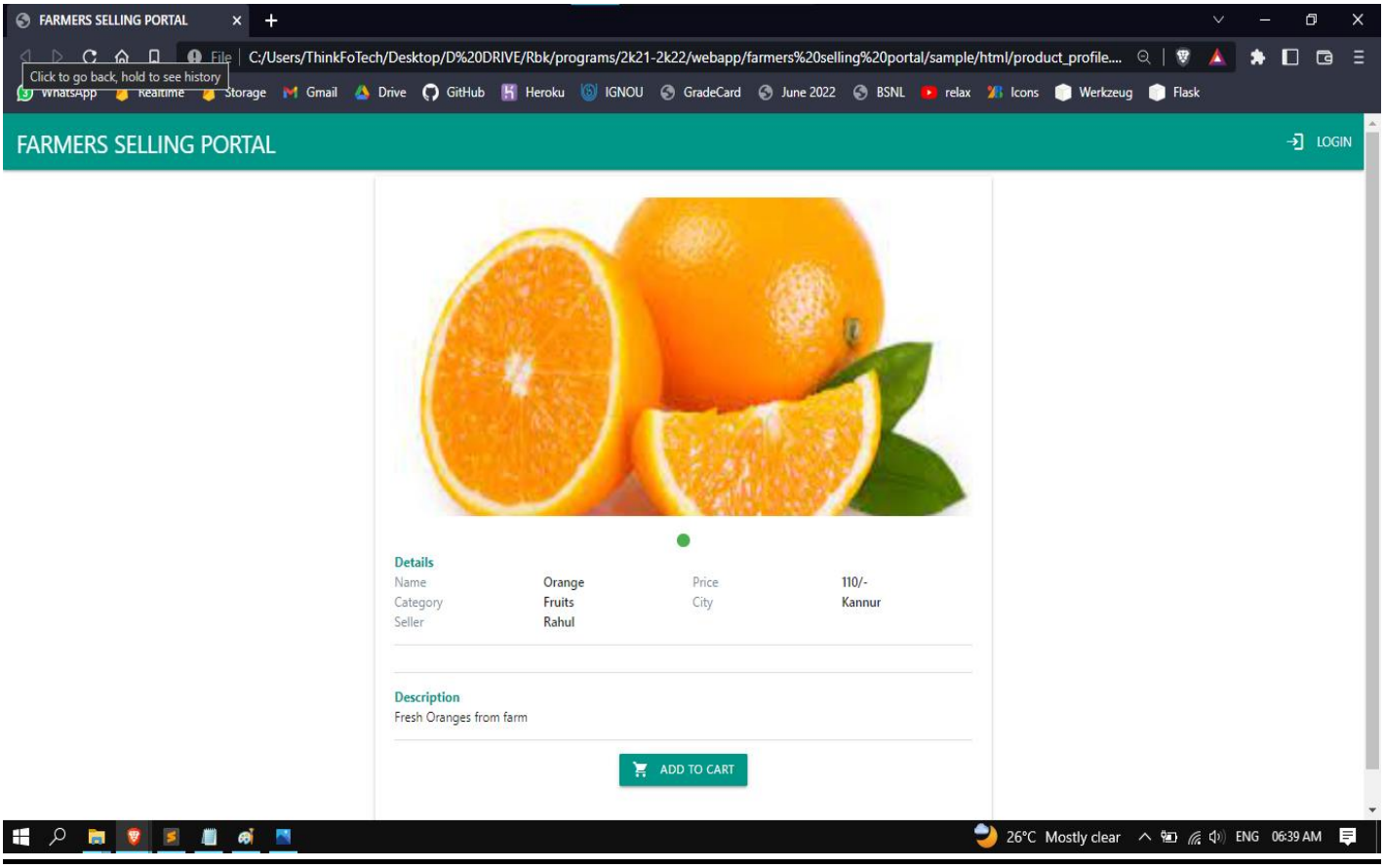
8

INPUT & OUTPUT SCREENS

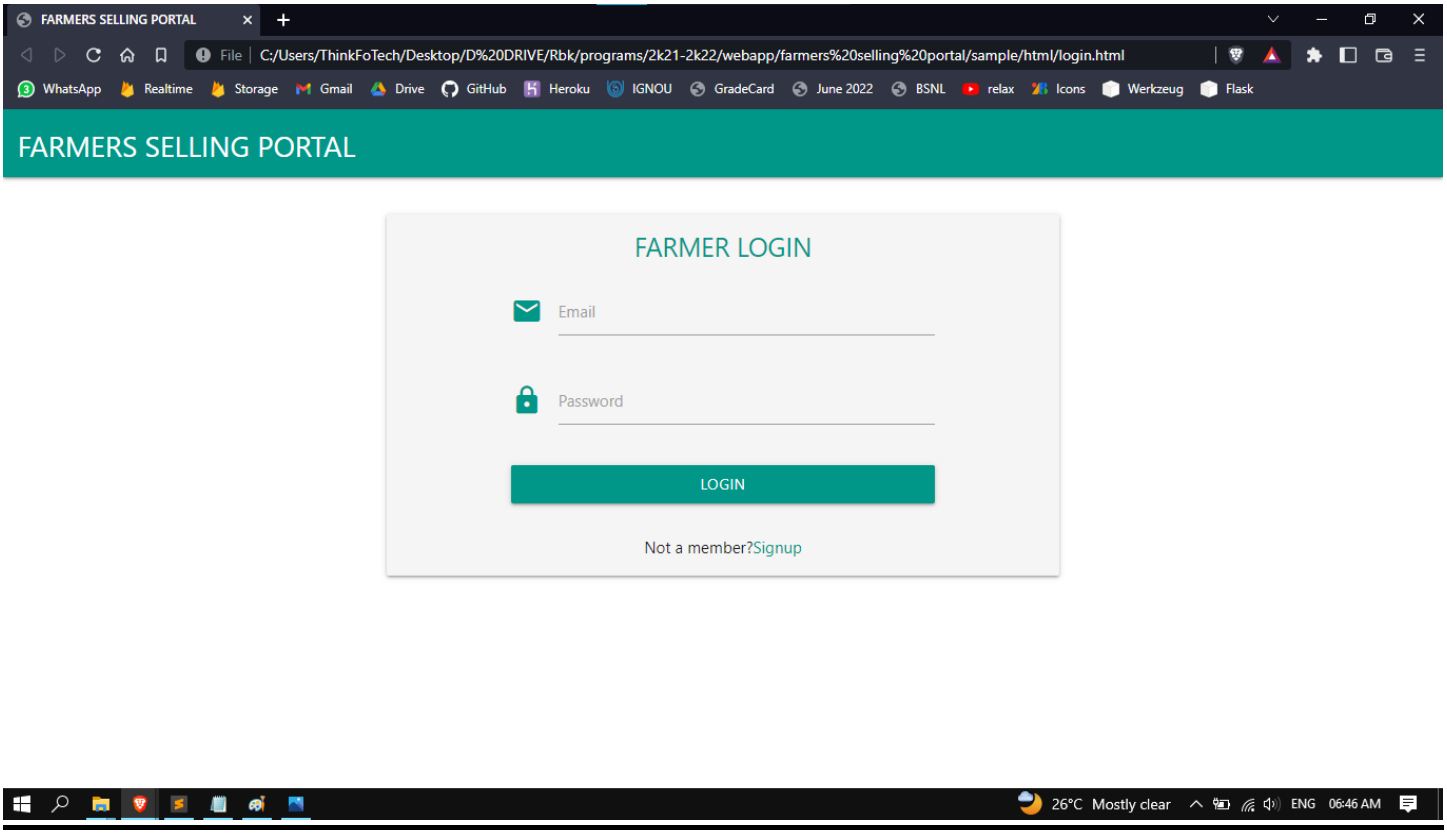
1.GENERAL HOME PAGE



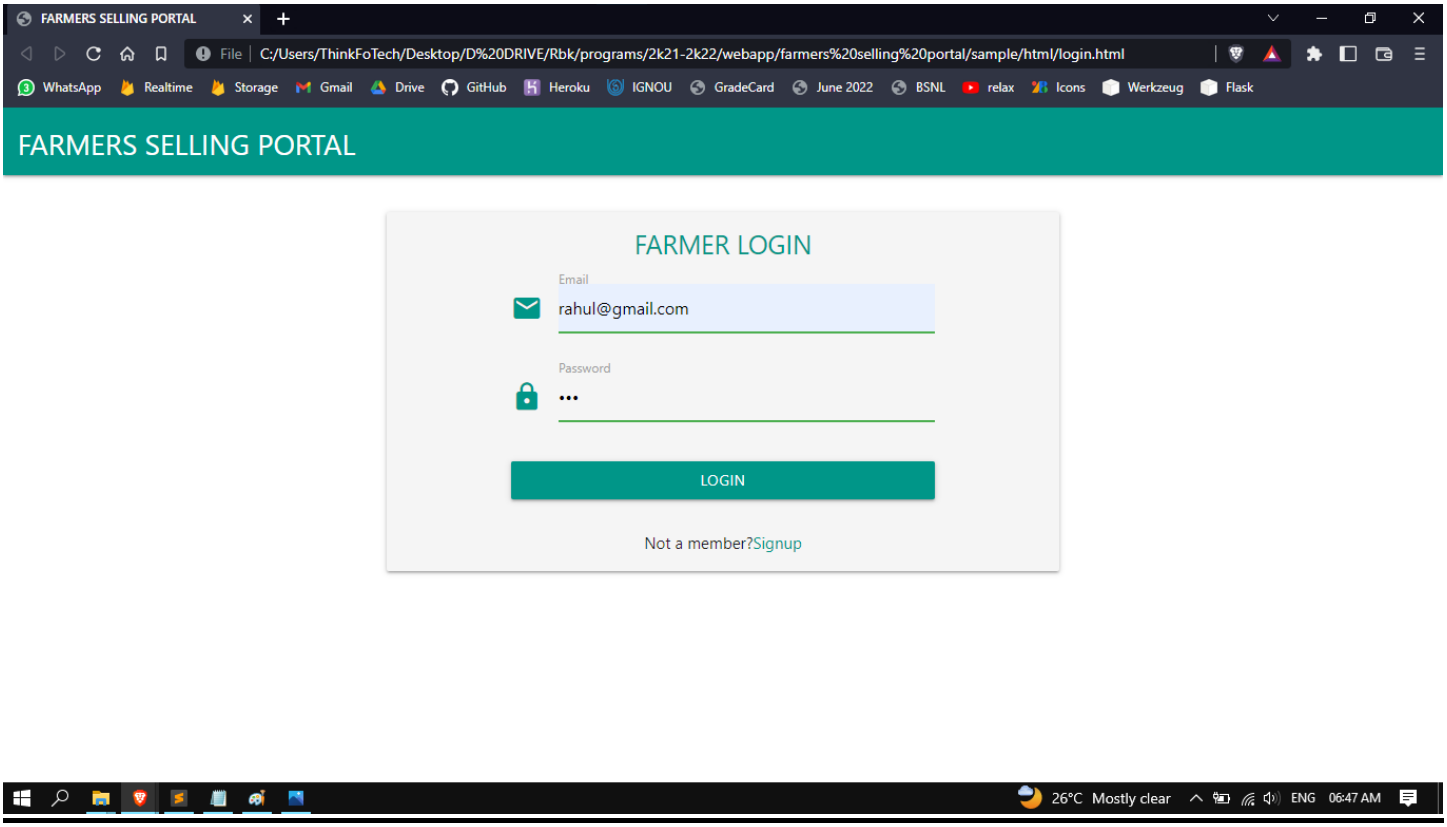
2. GENERAL PRODUCT PROFILE



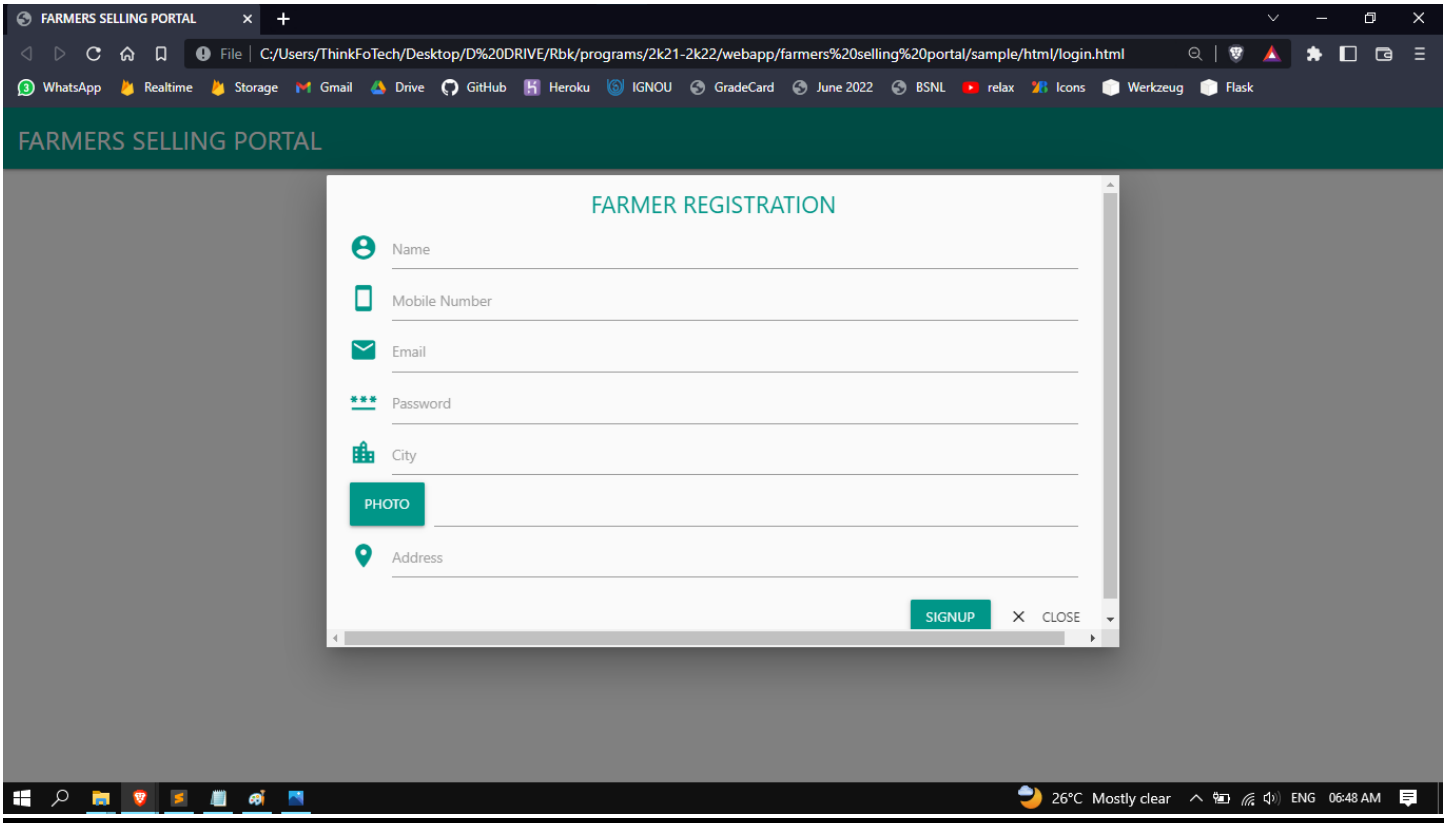
3.FAERMERS LOGIN PAGE



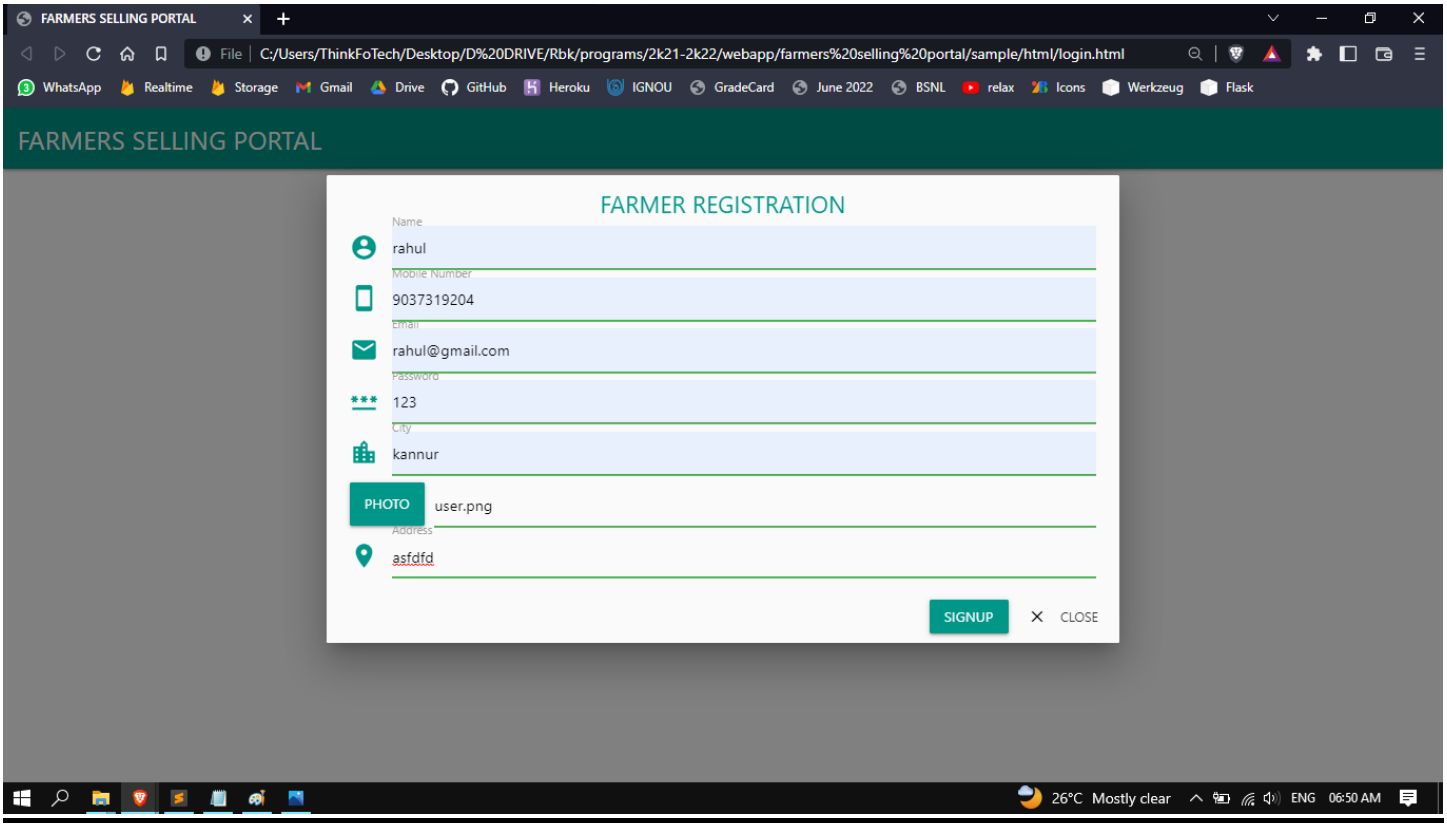
4.FARMERS LOGIN PAGE FIILLED



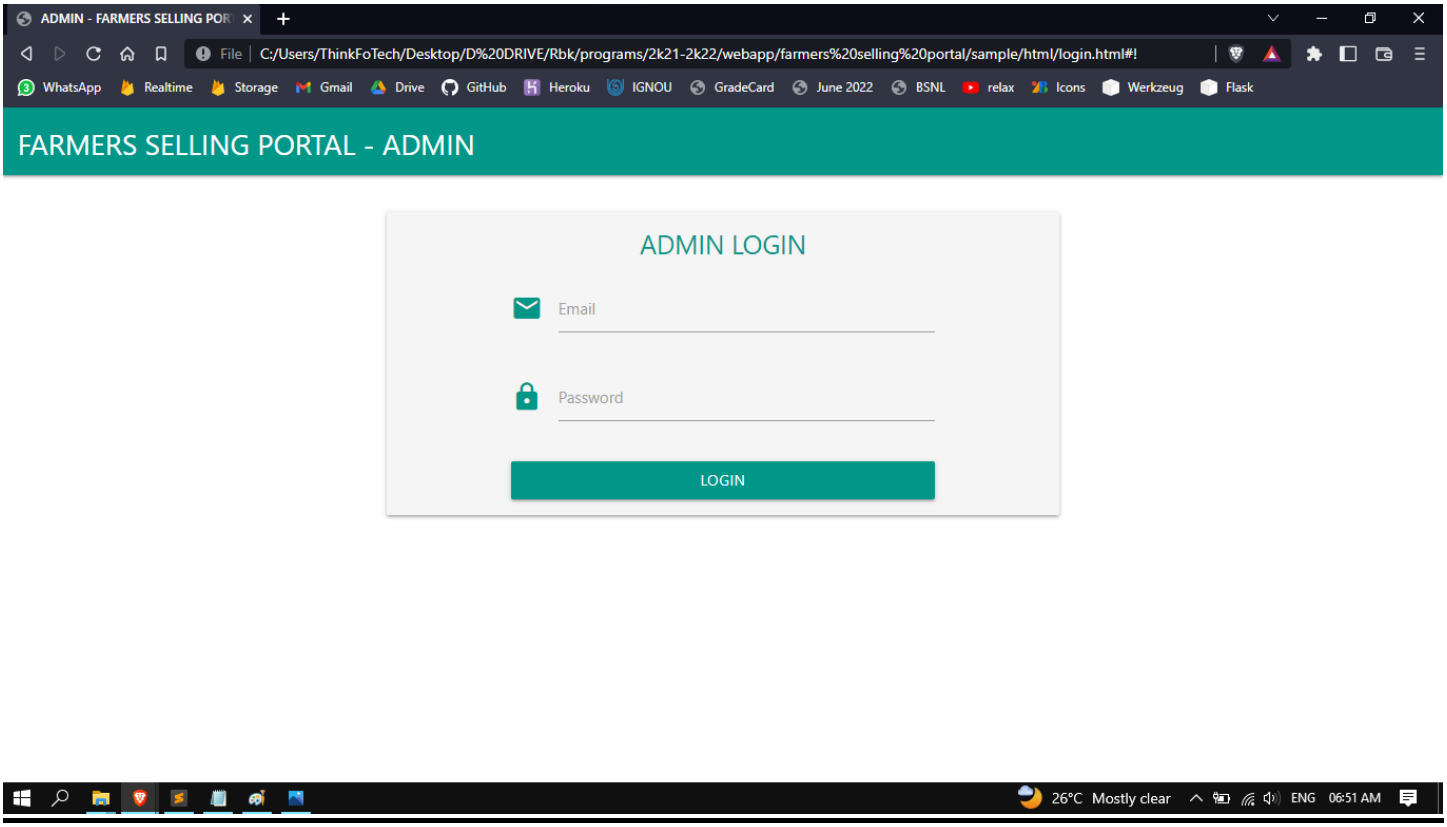
5.FARMERS REGISTRATION PAGE



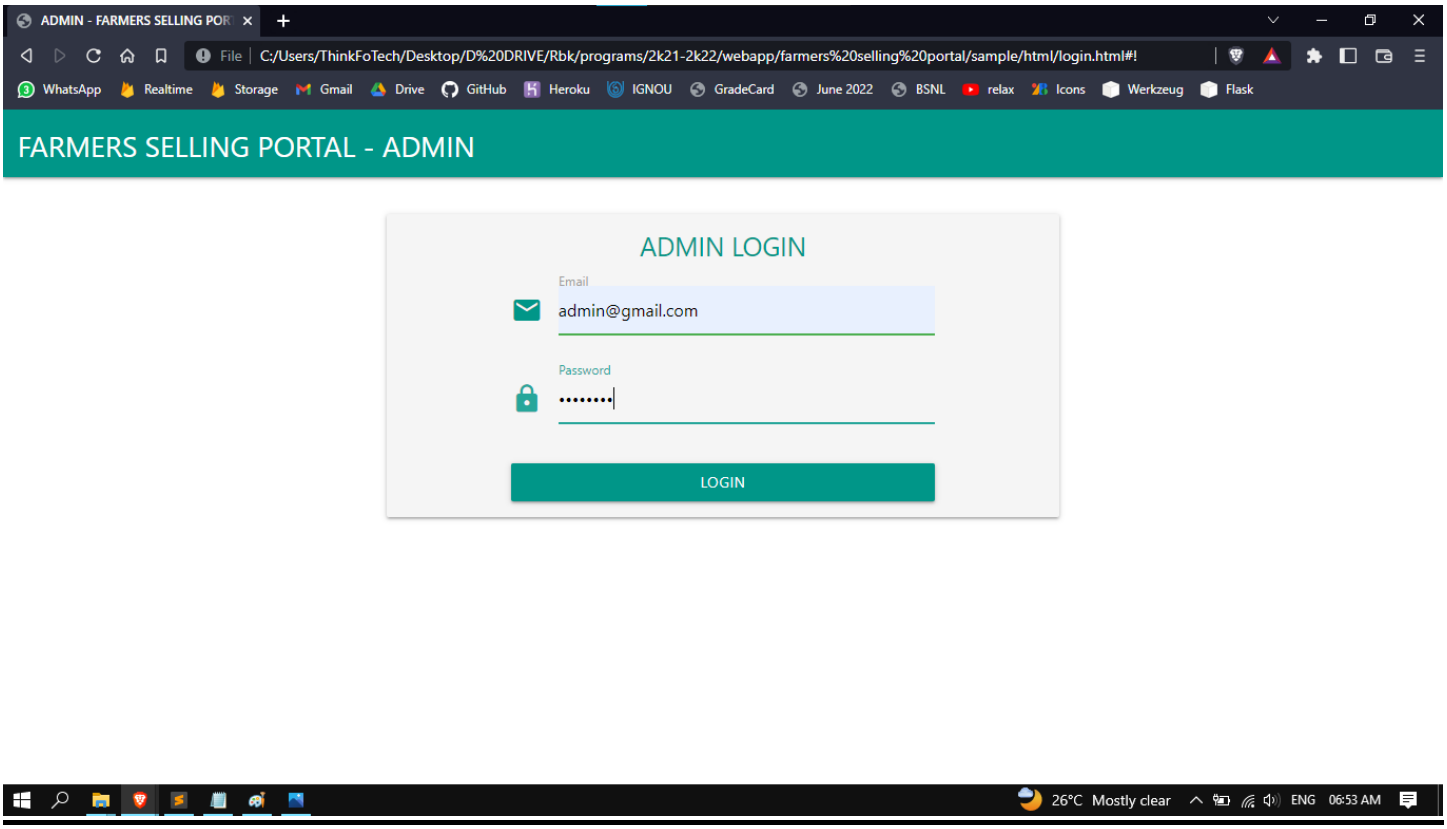
6.FARMERS REGISTRATION PAGE FILLED



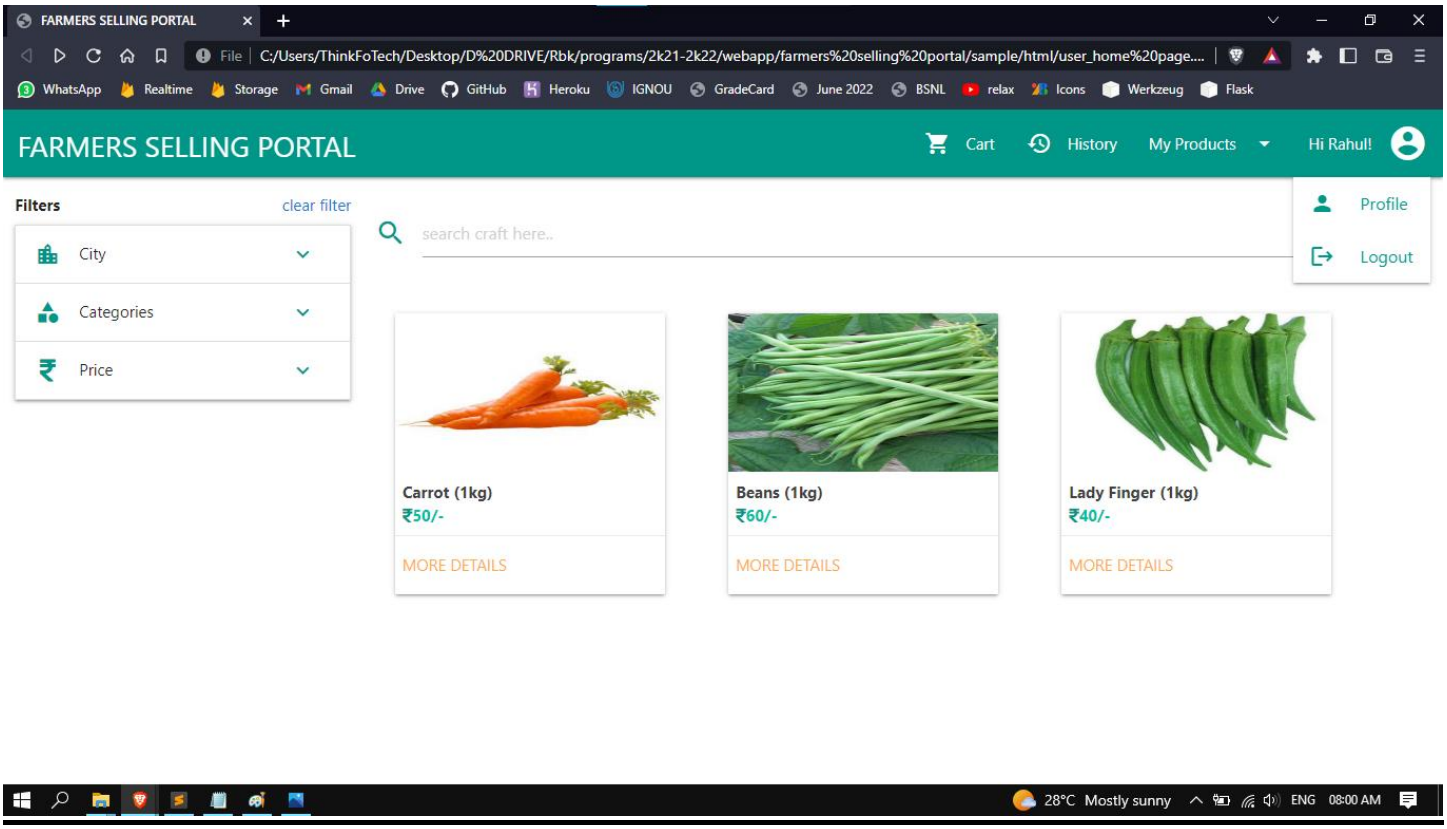
7.ADMIN LOGIN PAGE



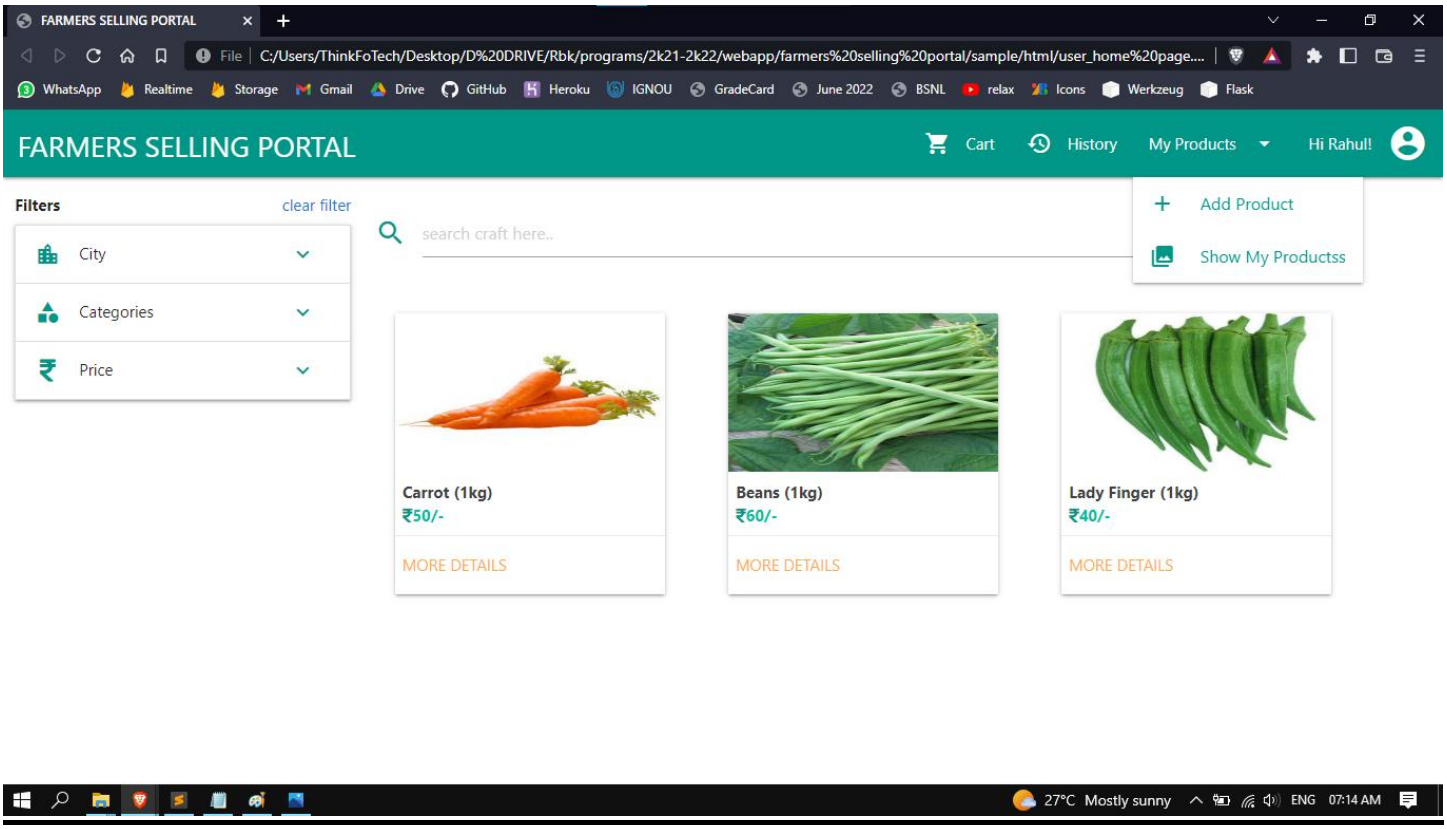
8.ADMIN LOGIN PAGE FILLED



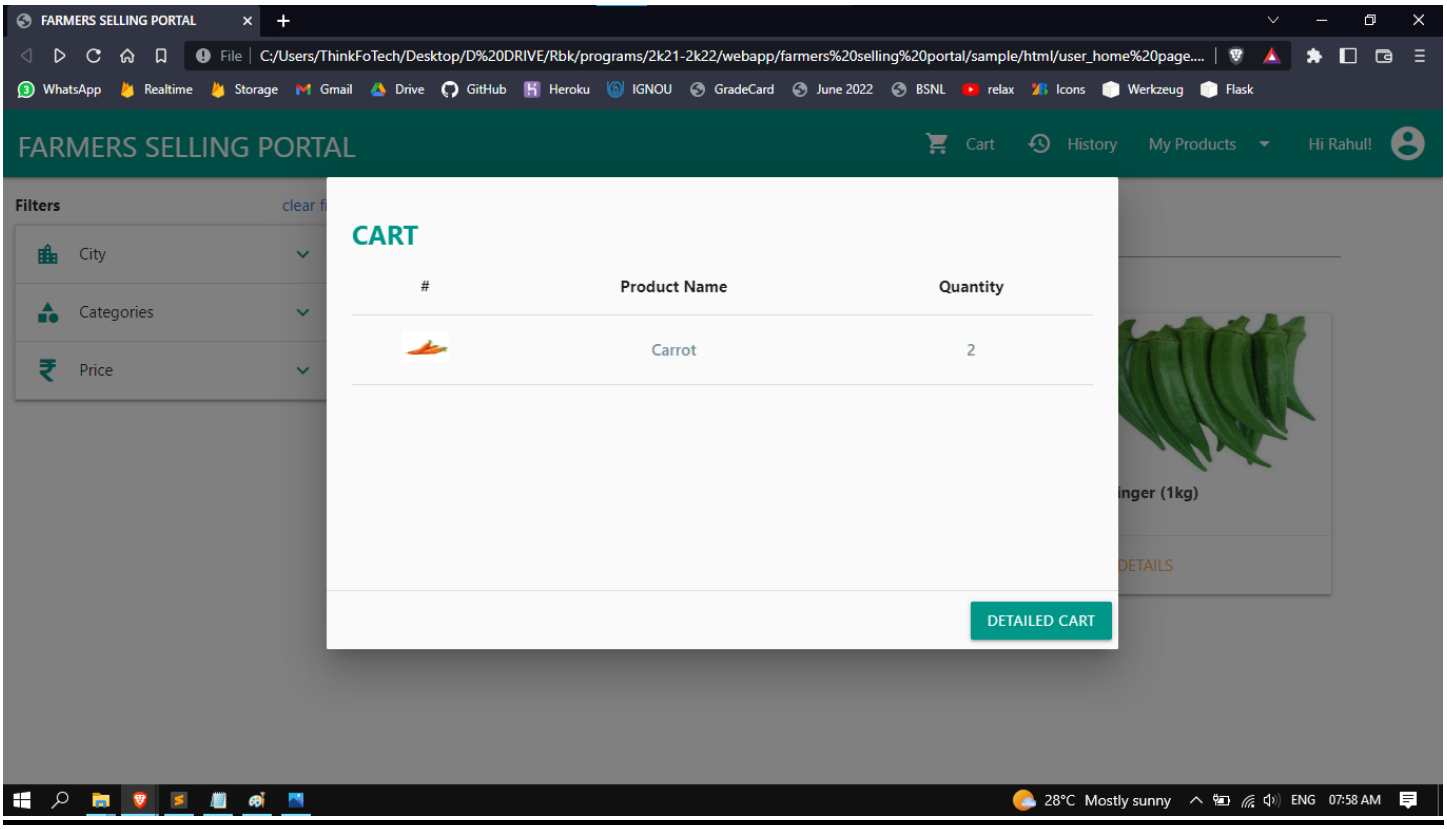
9.FARMER HOME PAGE



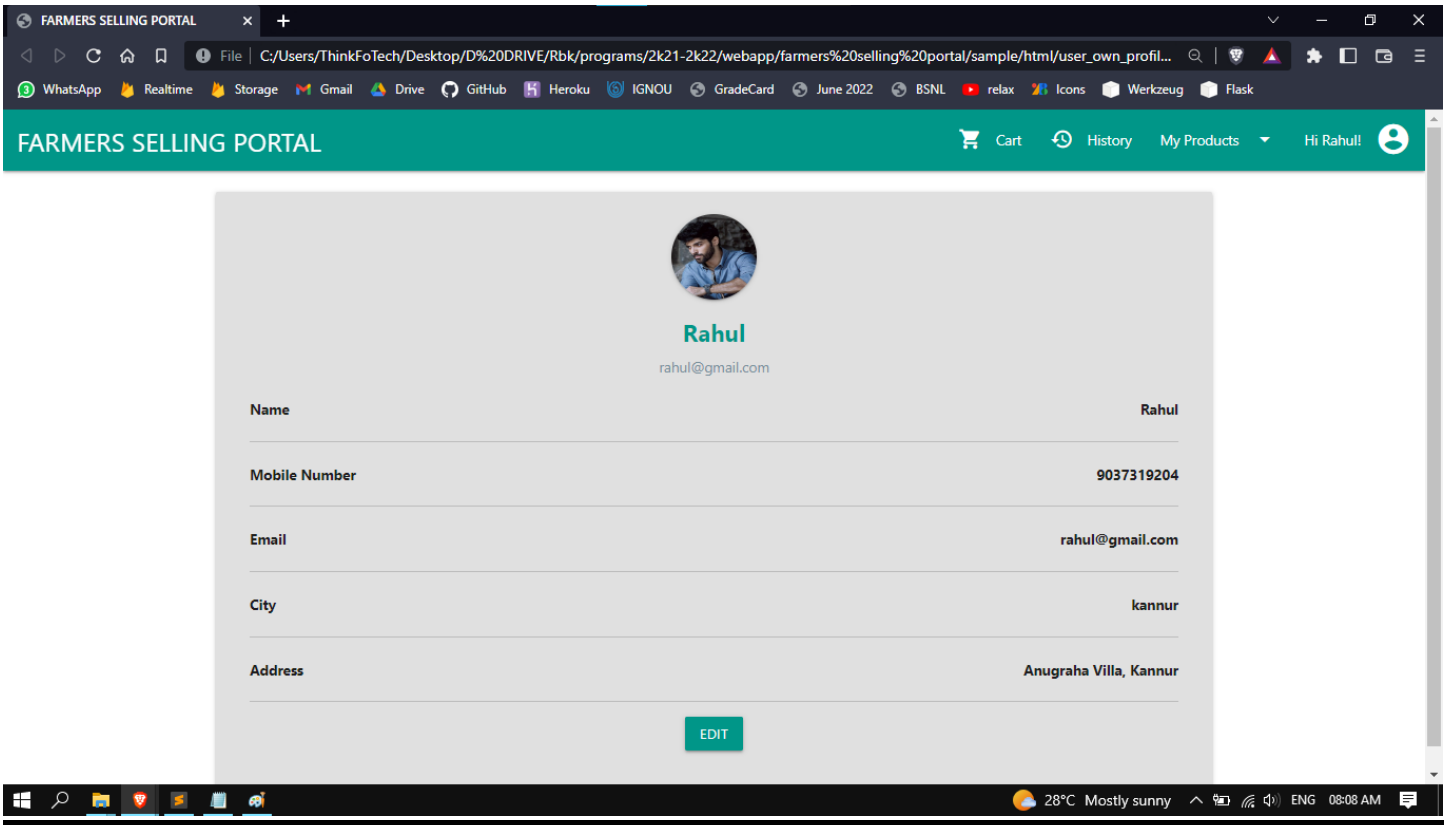
10FARMER HOME PAGE



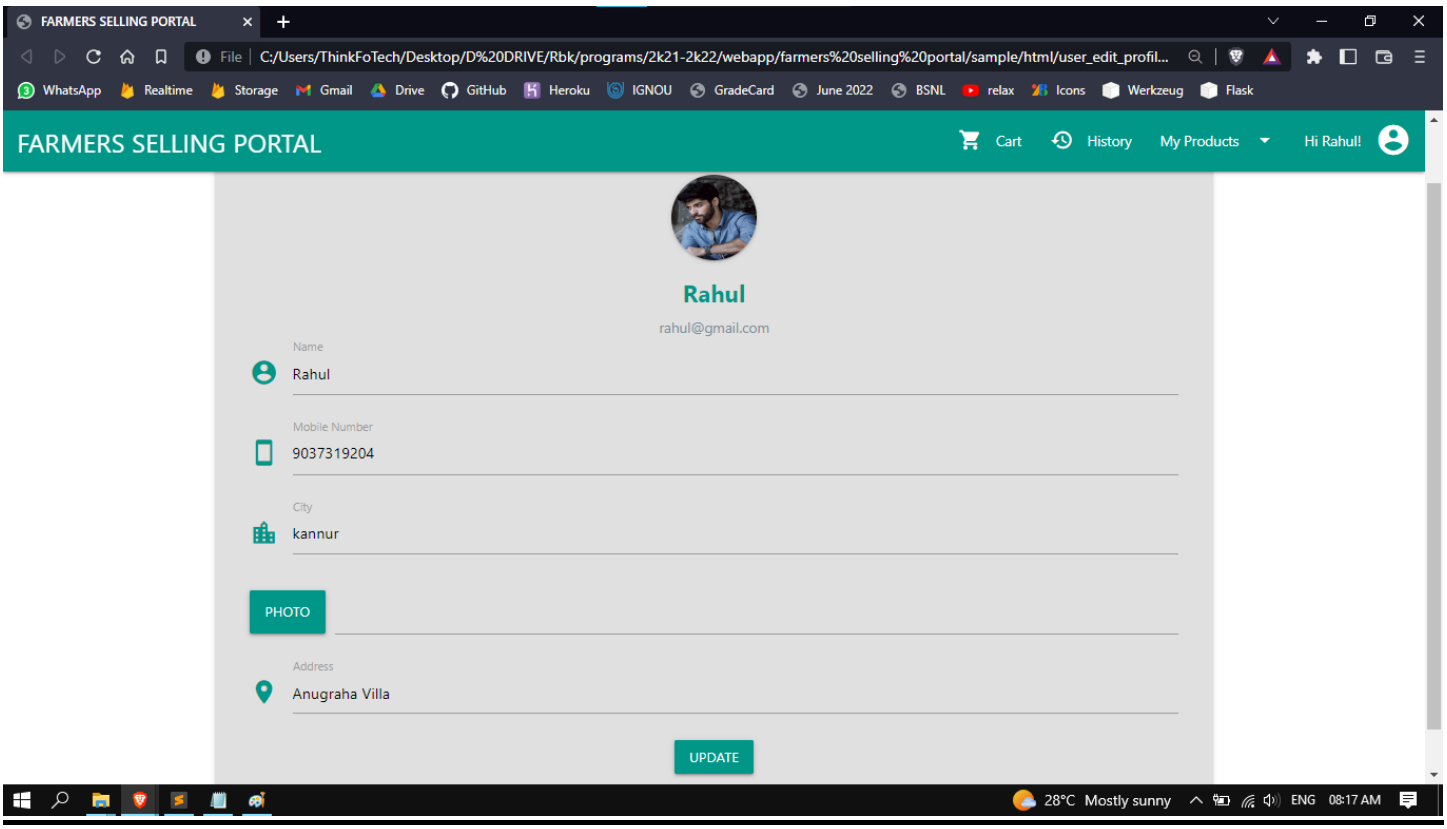
11.FARMER HOME PAGE



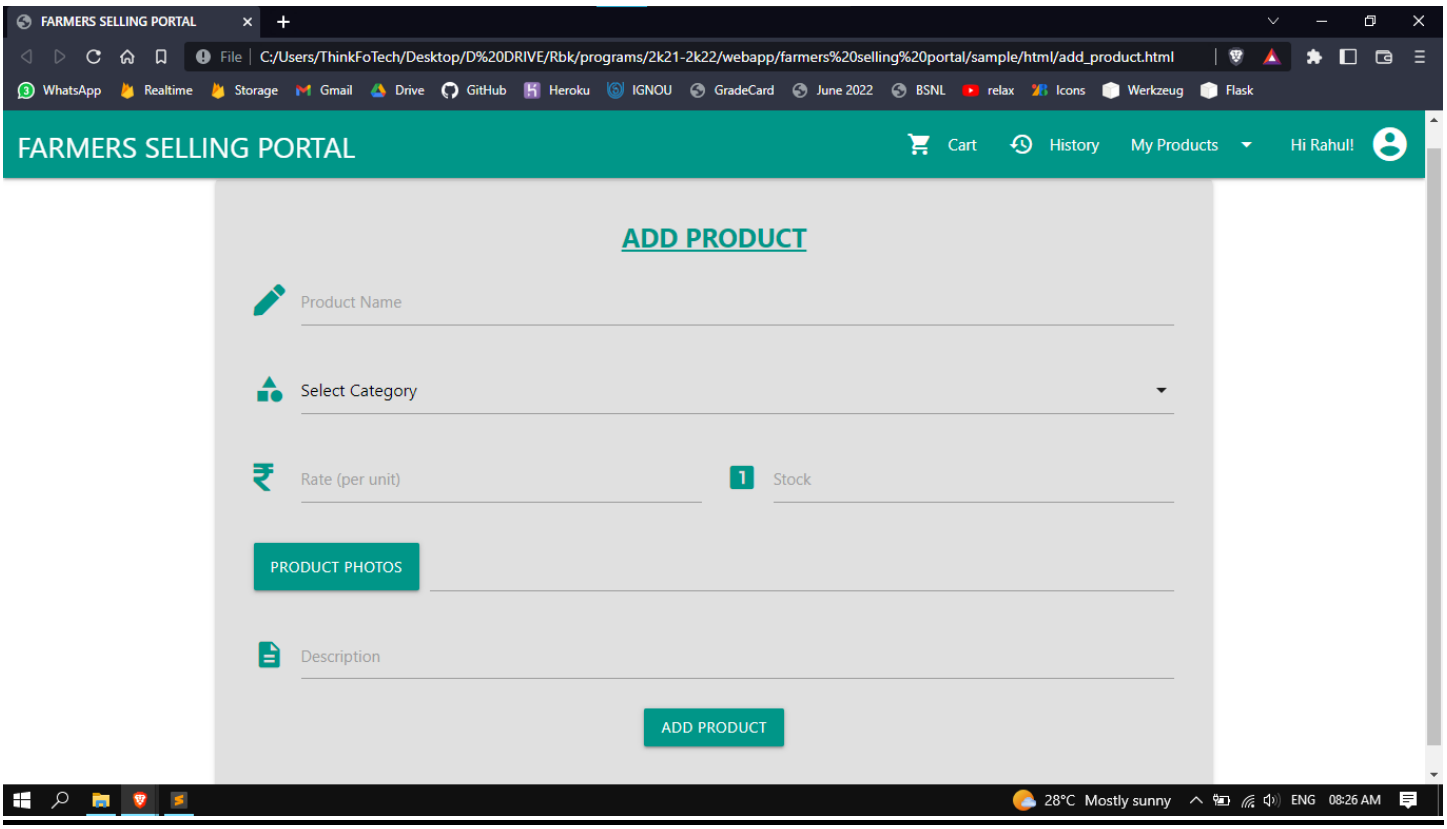
12.FARMER OWN PROFILE



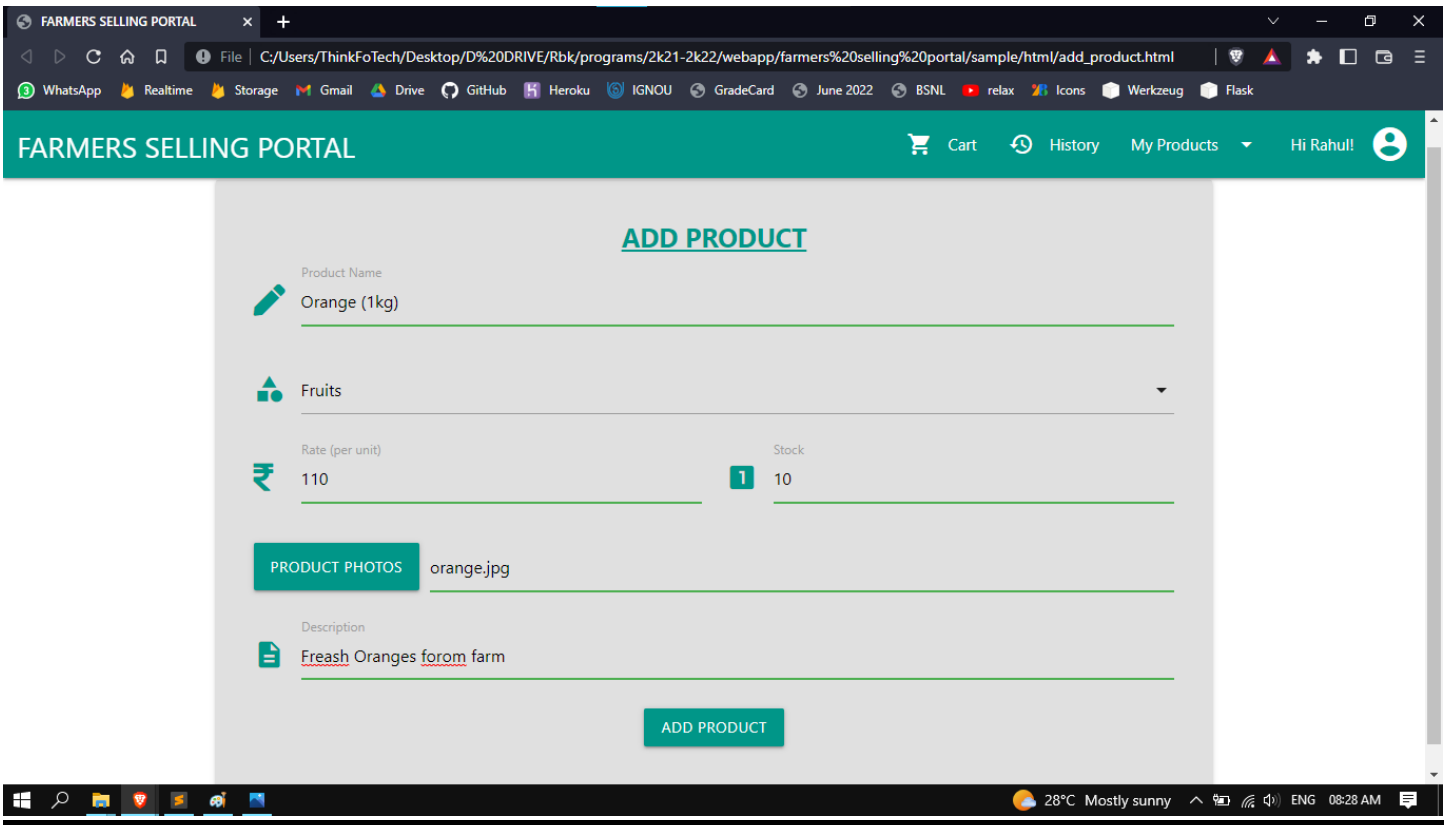
13 .FARMER OWN PROFILE EDIT PAGE



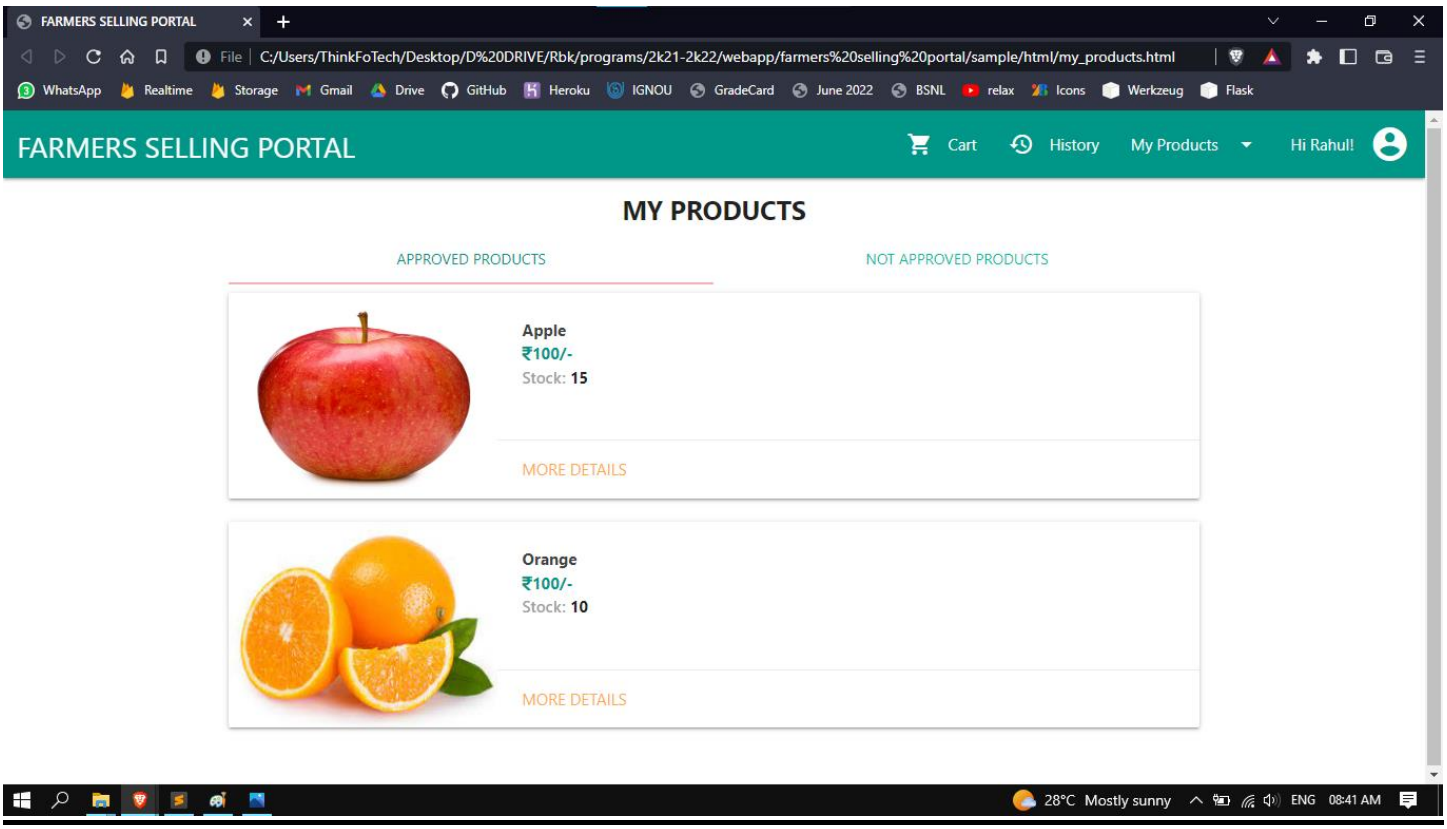
14.FARMER PRODUCT REGISTRATION PAGE



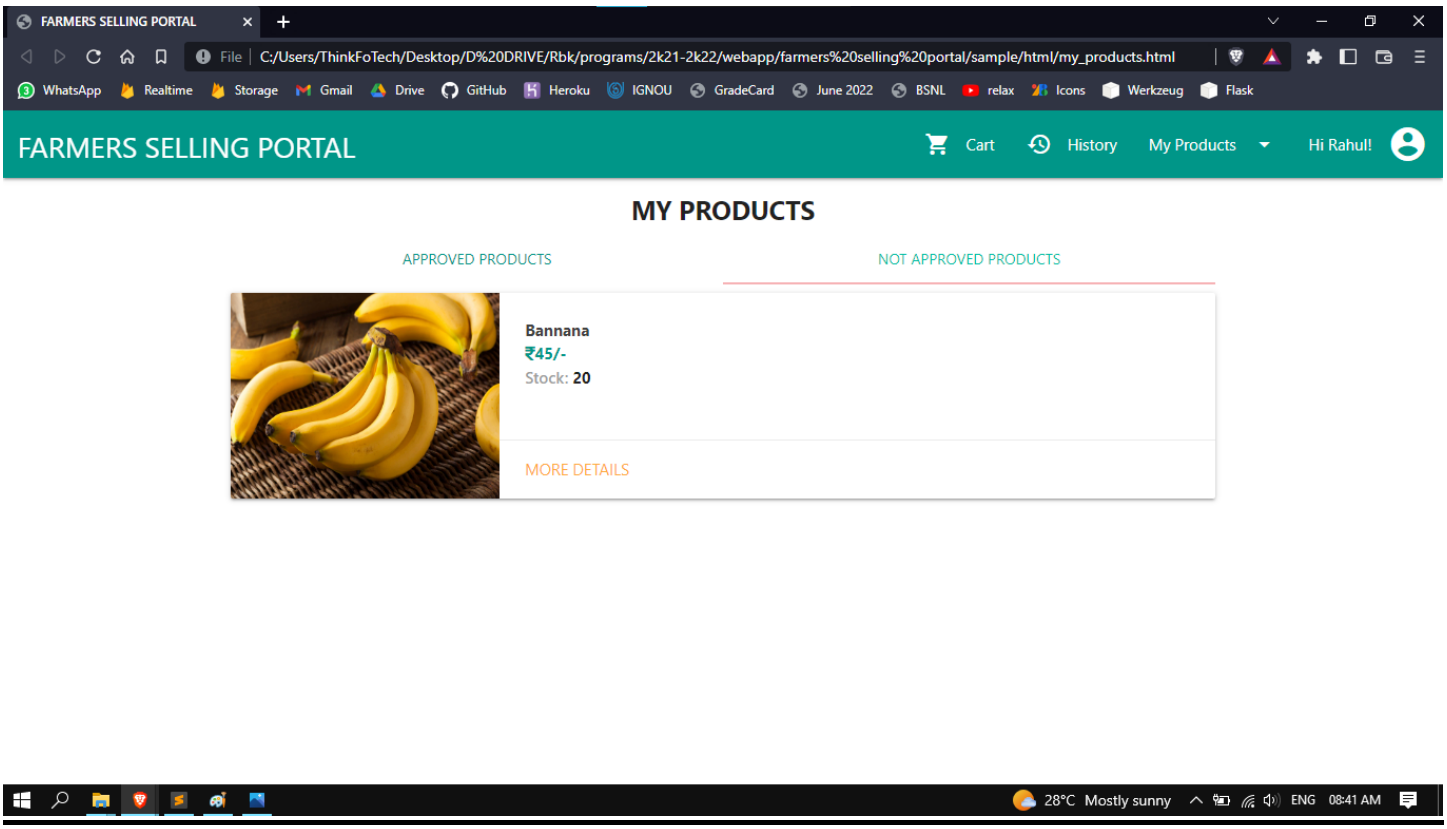
15.FARMER PRODUCT REGISTRATION FILLED



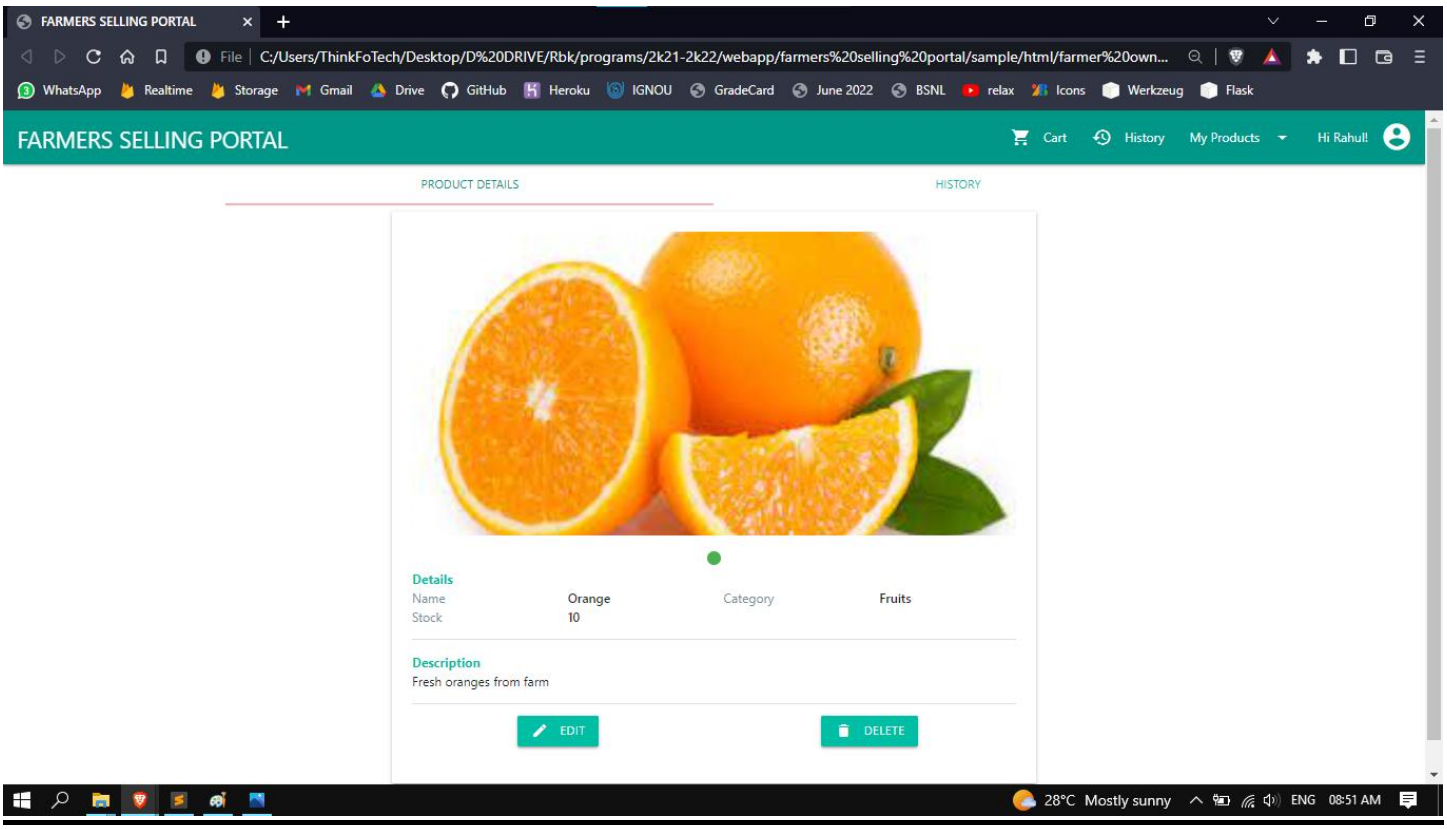
16.FARMER OWN PRODUCT PAGE (APPROVED)



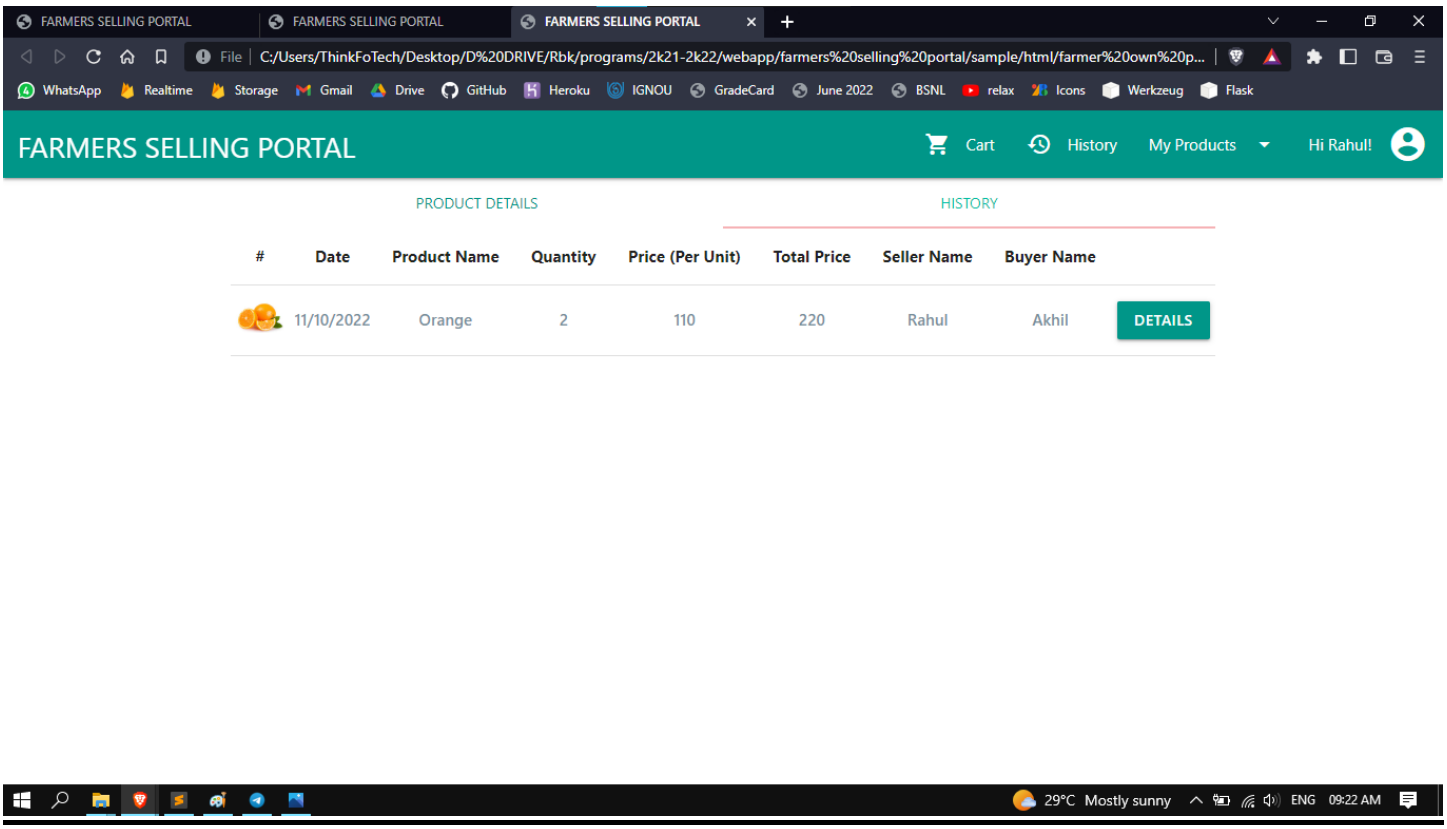
17.FARMER OWN PRODUCT PAGE(NOT APPROVED)



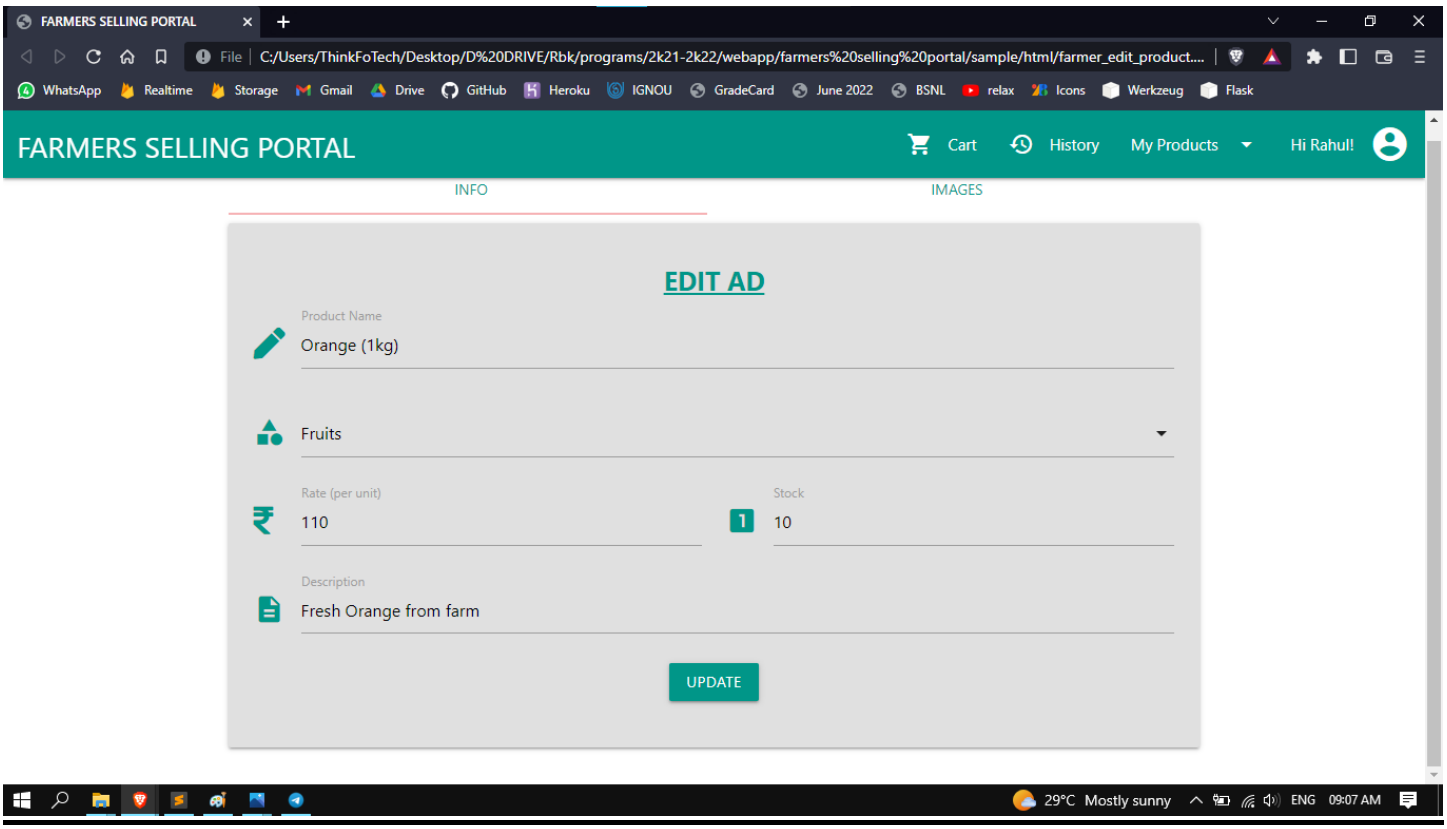
18.FARMER OWN PRODUCT PROFILE PAGE



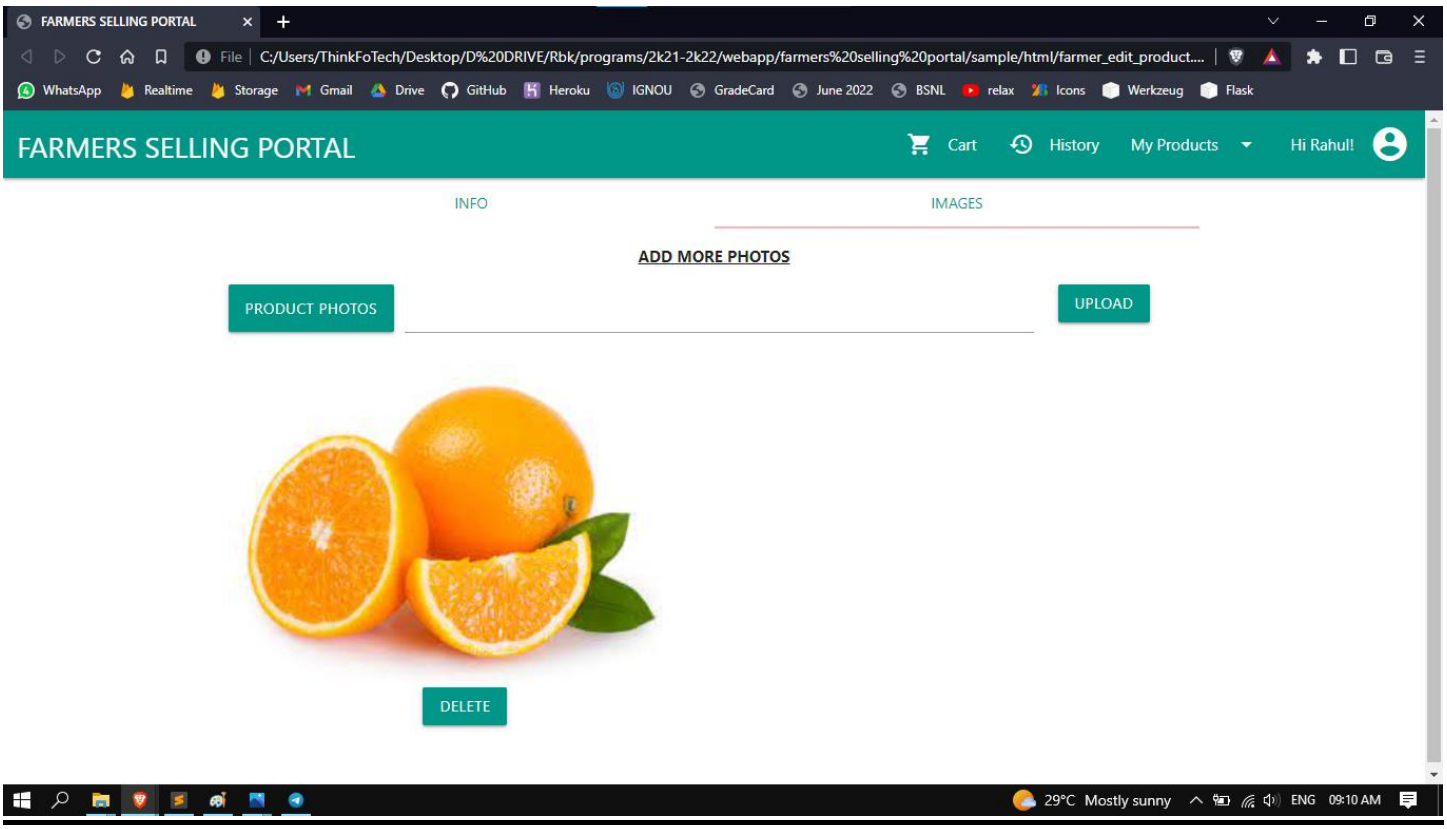
19.FARMER OWN PRODUCT HISTORY



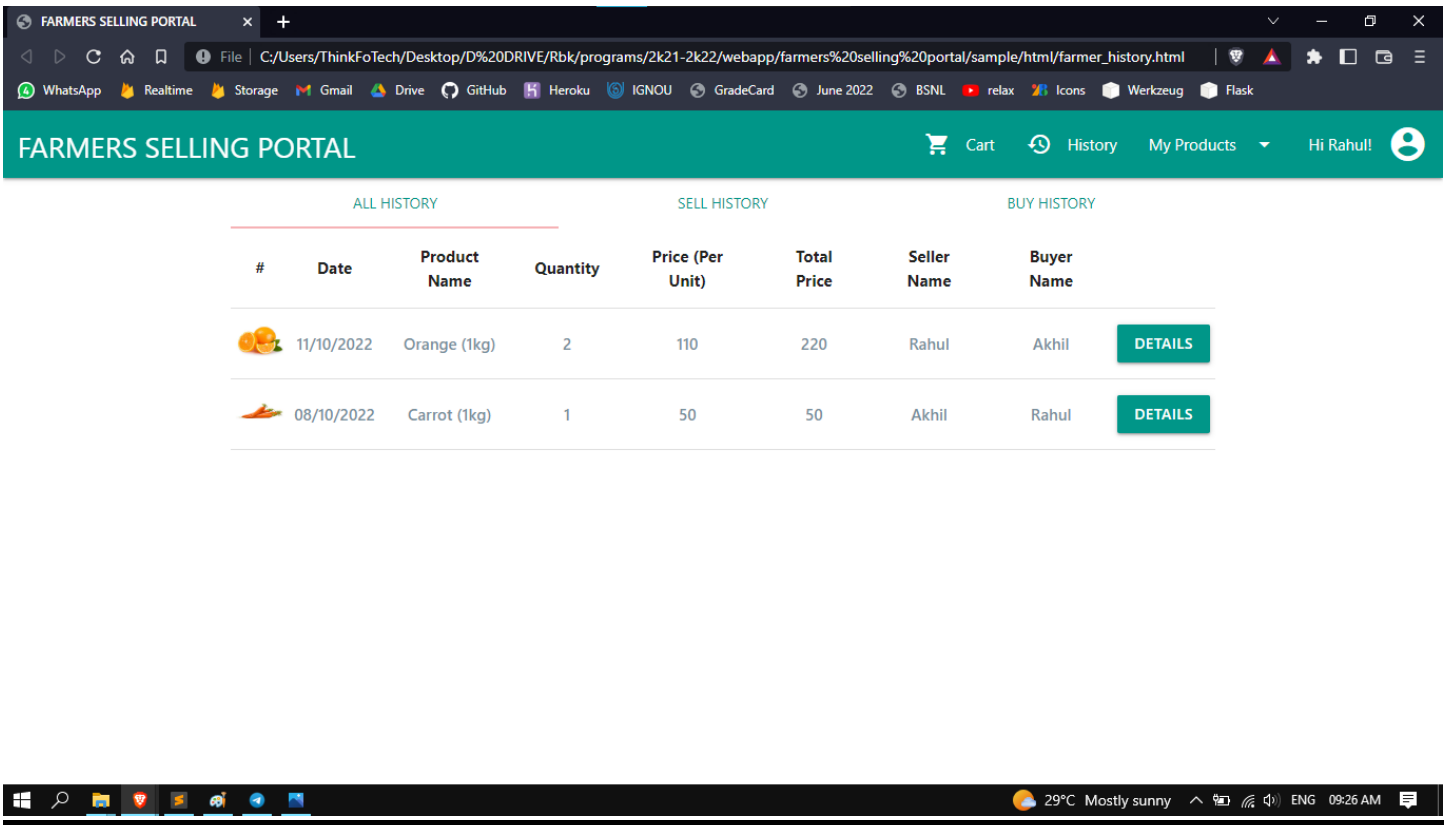
20.FARMER OWN PRODUCT EDIT PAGE



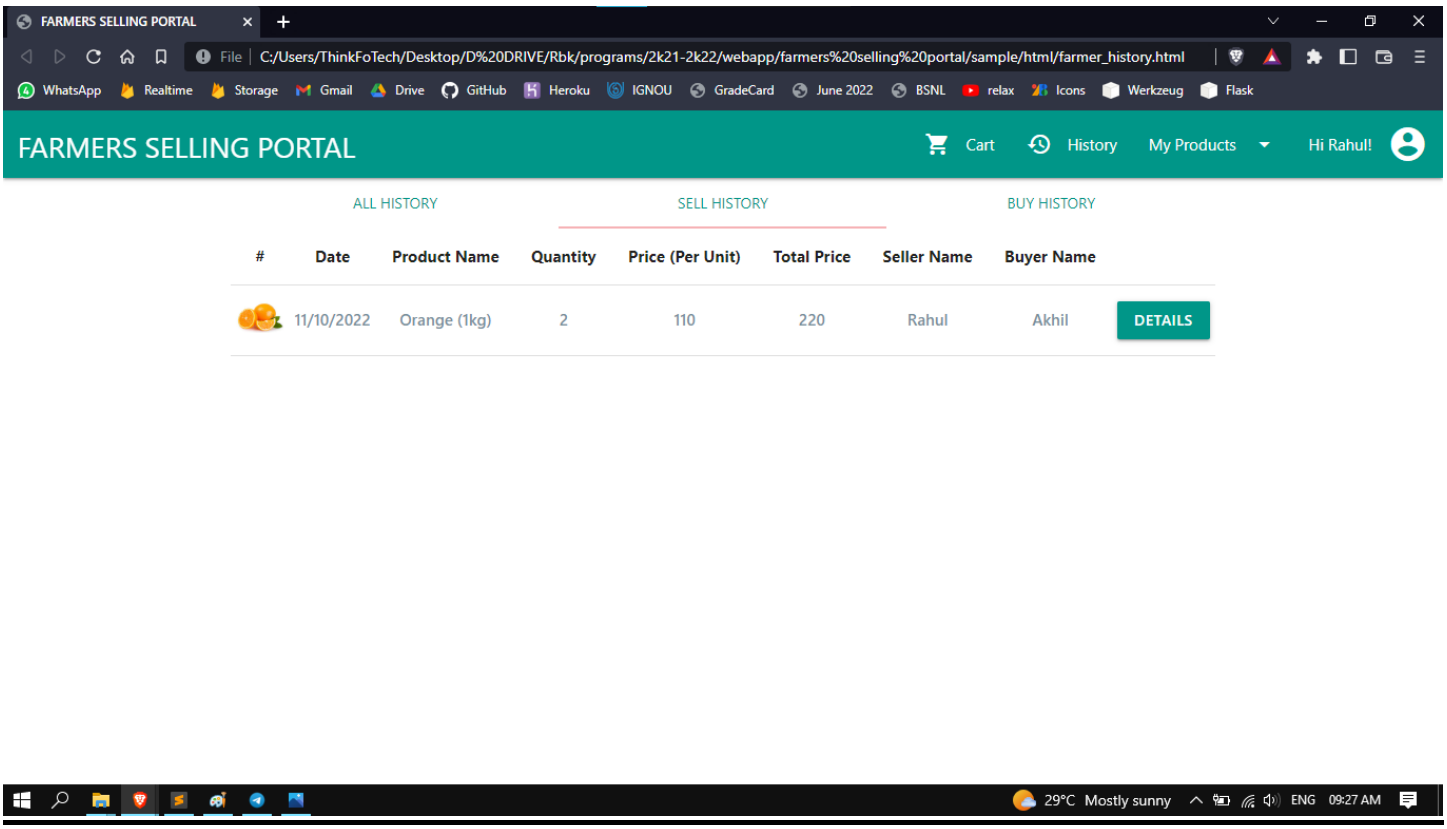
21.FARMER OWN PRODUCT IMAGE EDIT PAGE



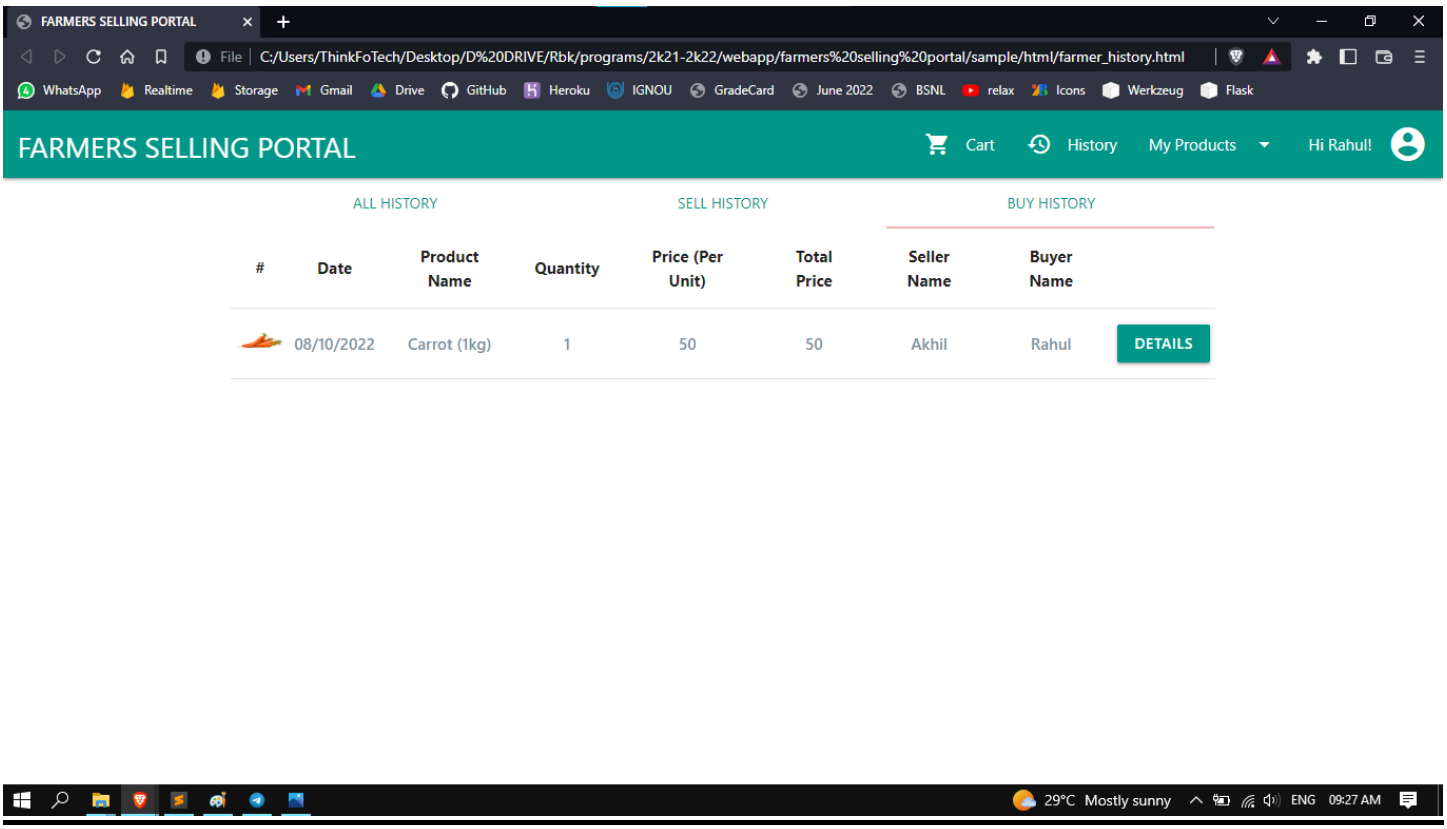
22.FARMER ALL HISTORY



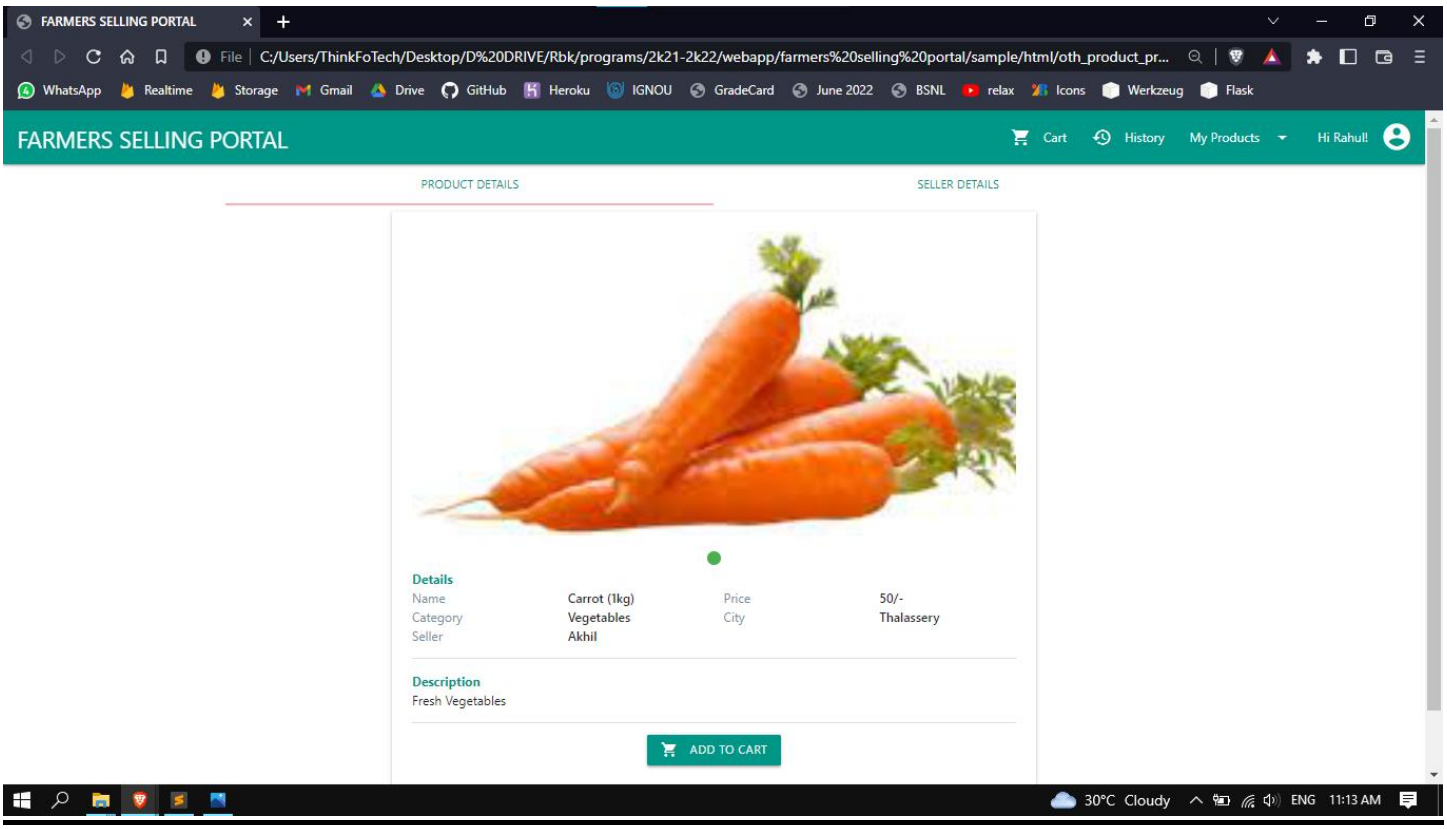
23.FARMER SELL HISTORY



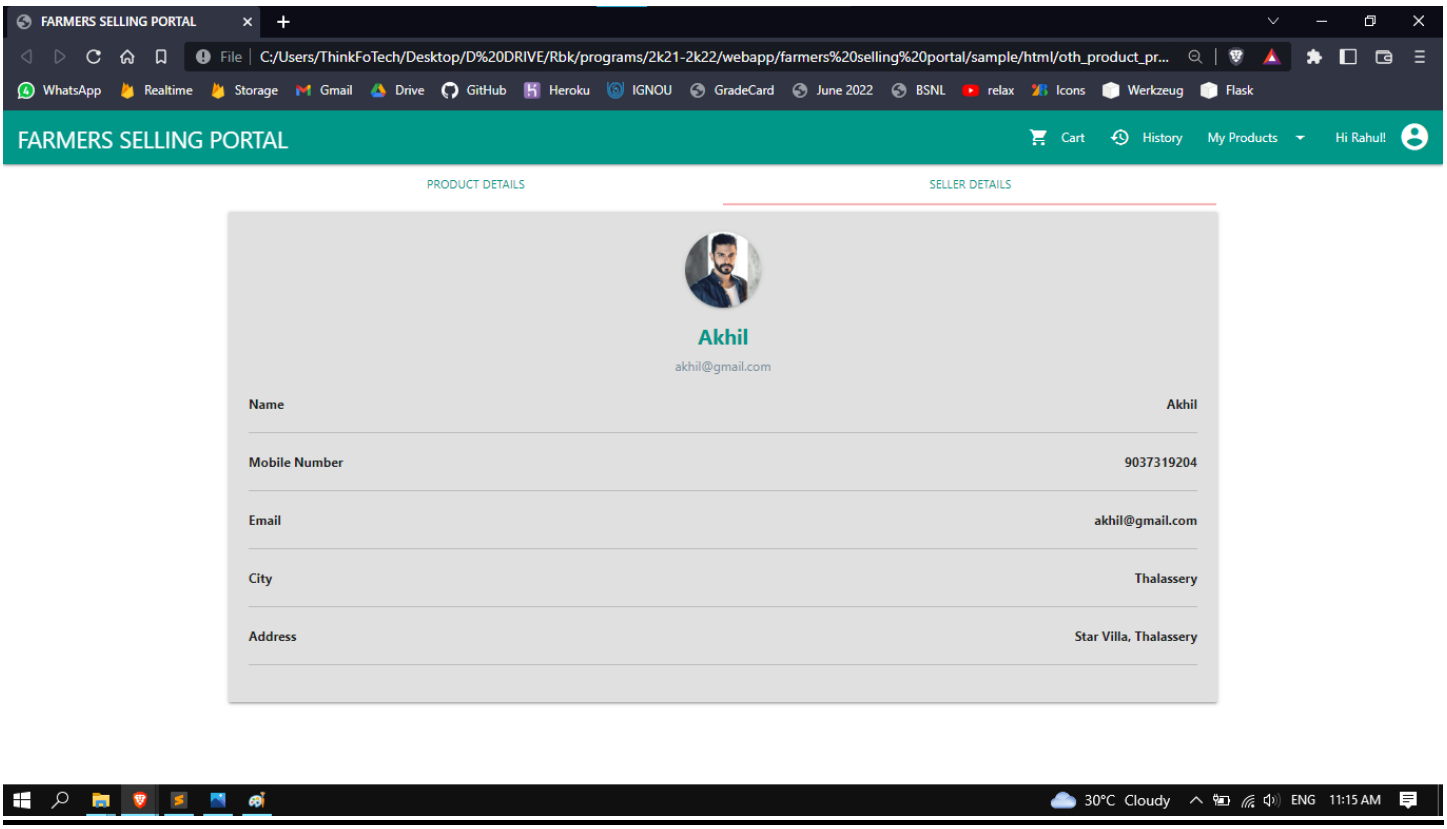
24.FARMER BUY HISTORY



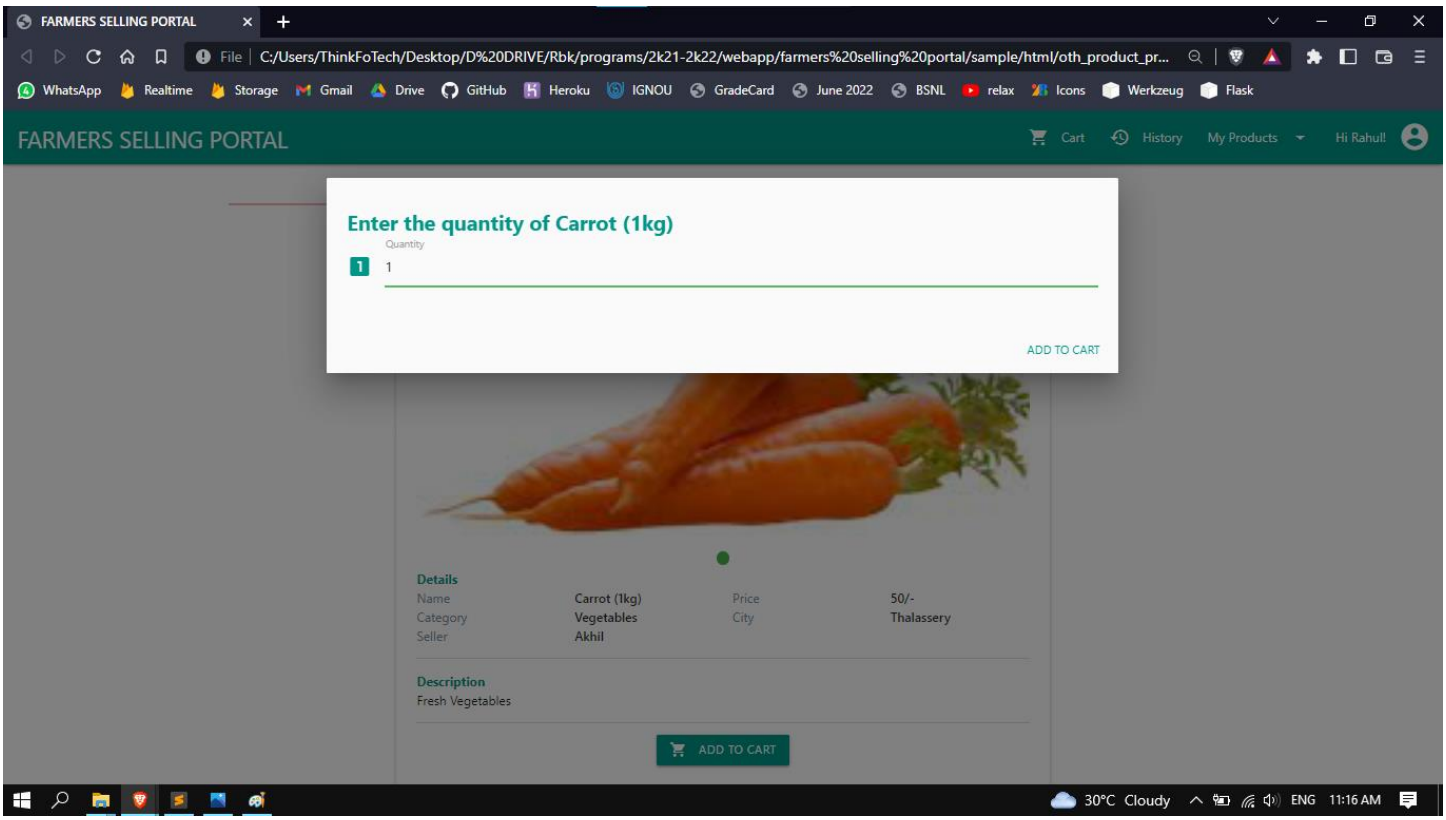
25.OTHER PRODUCT PROFILE(PRODUCT DETAILS)



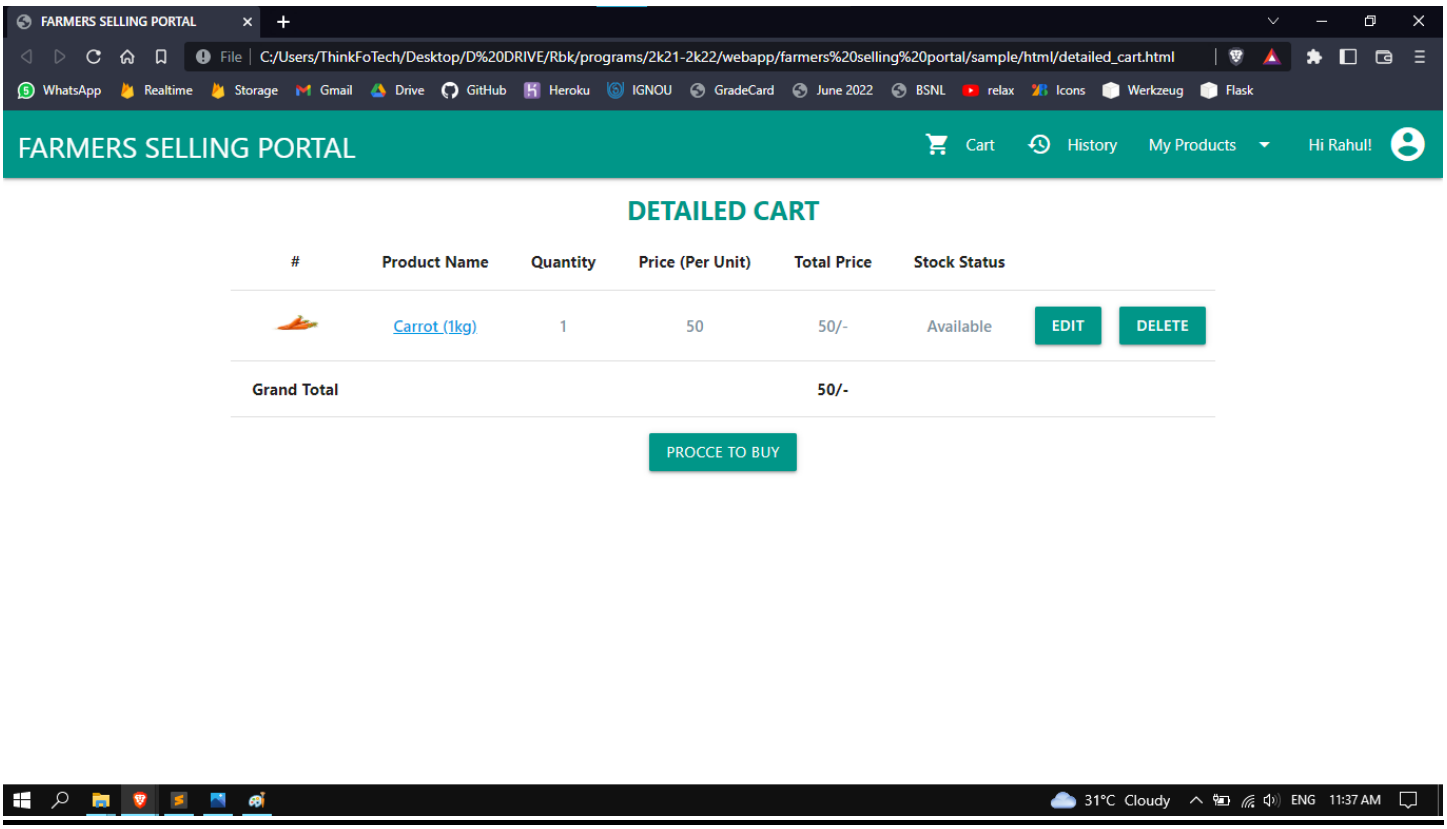
26.OTHER PRODUCT PROFILE(SELLER DETAILS)



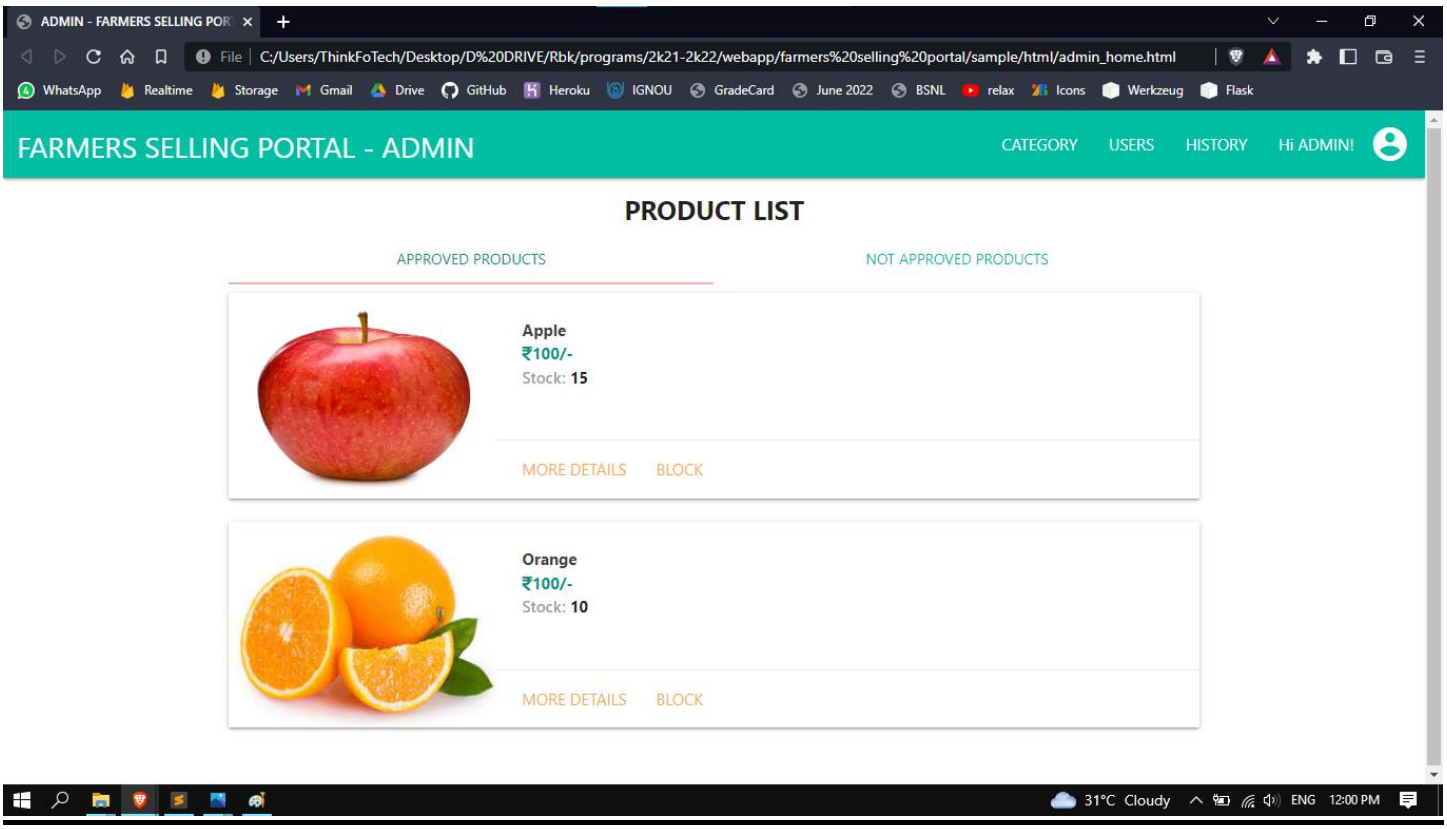
27.OTHER PRODUCT ADDING TO CART



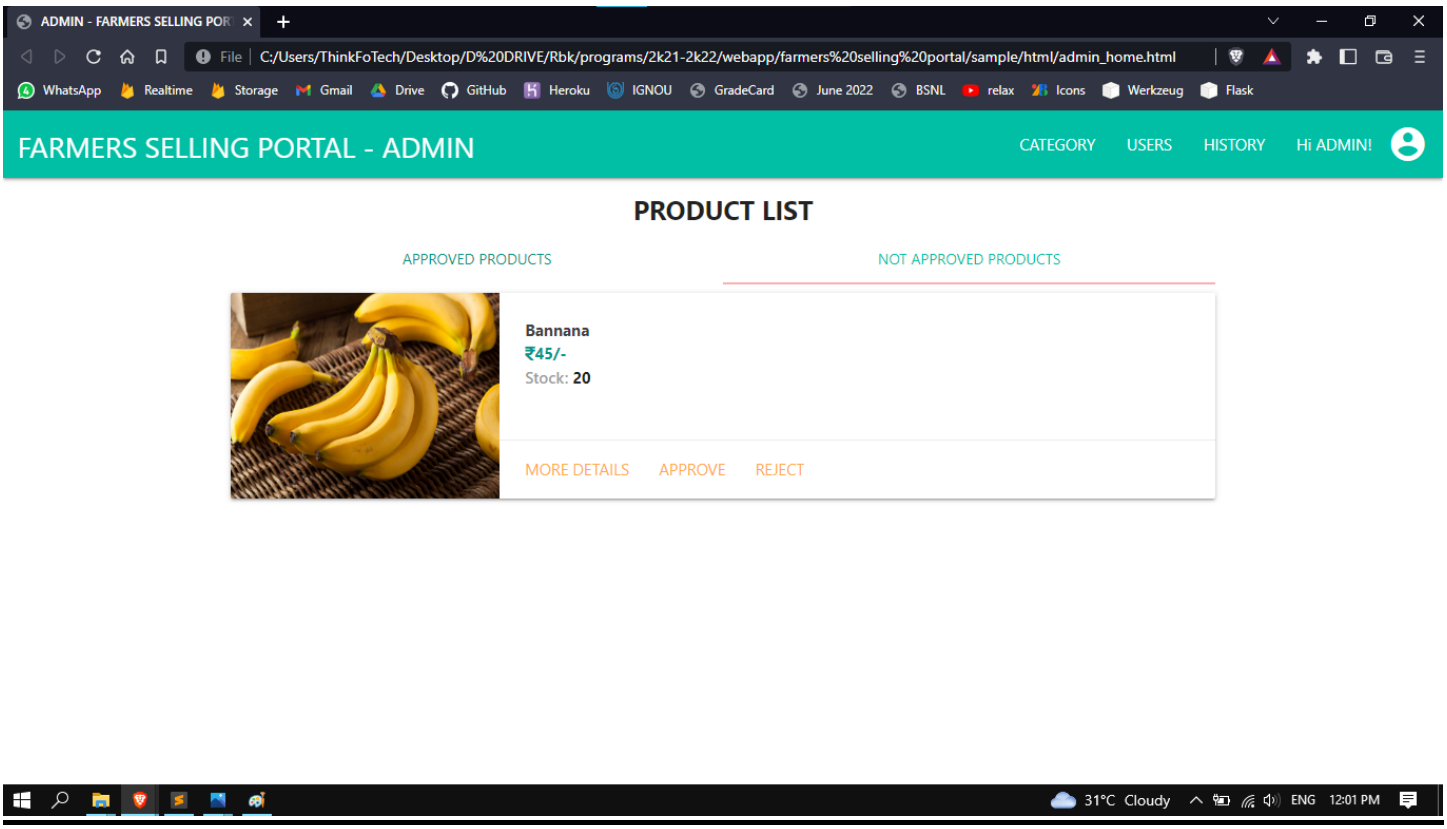
18.DETAILED CART



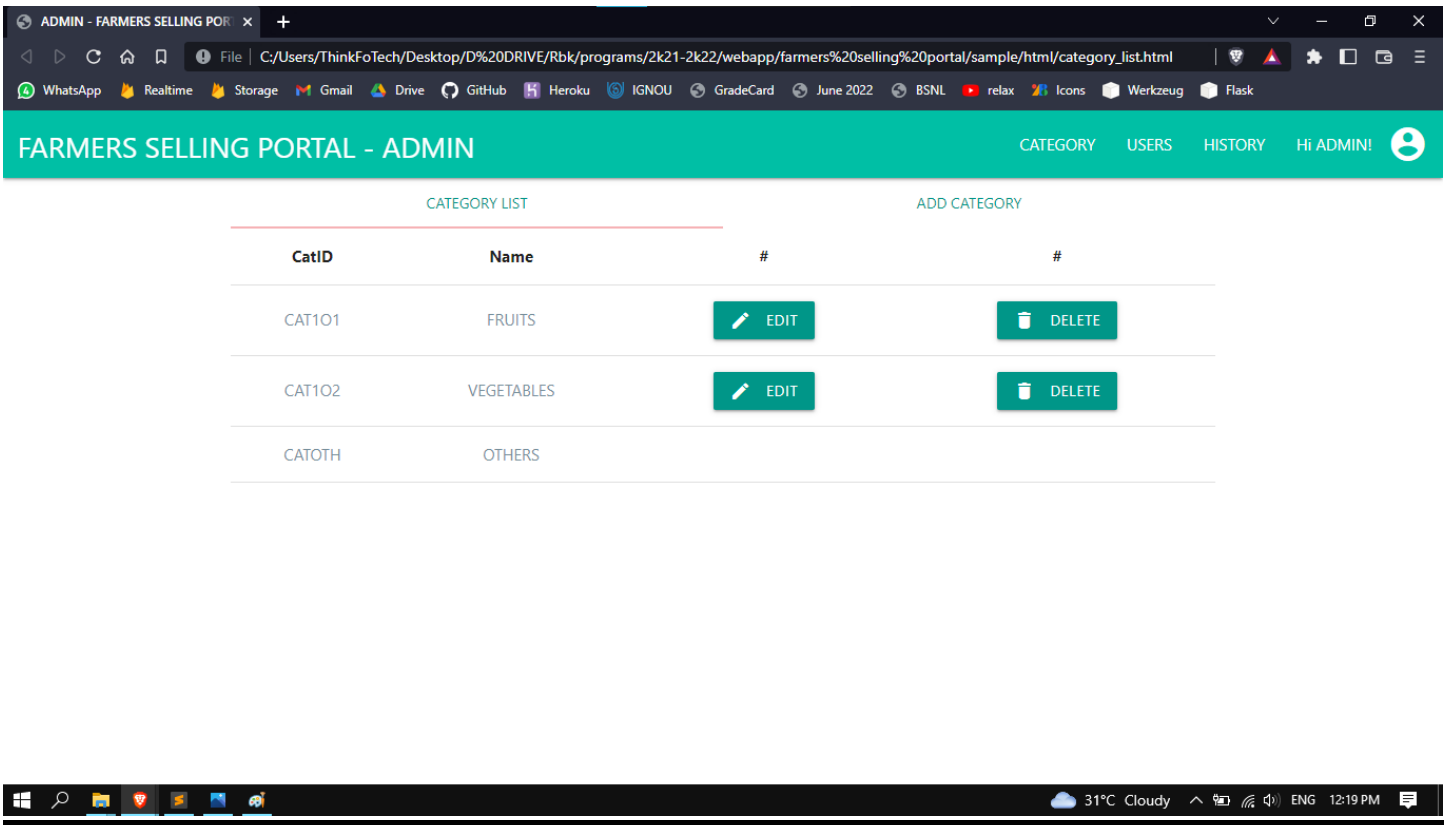
29.ADMIN HOME PAGE(APPROVED PRODUCTS)



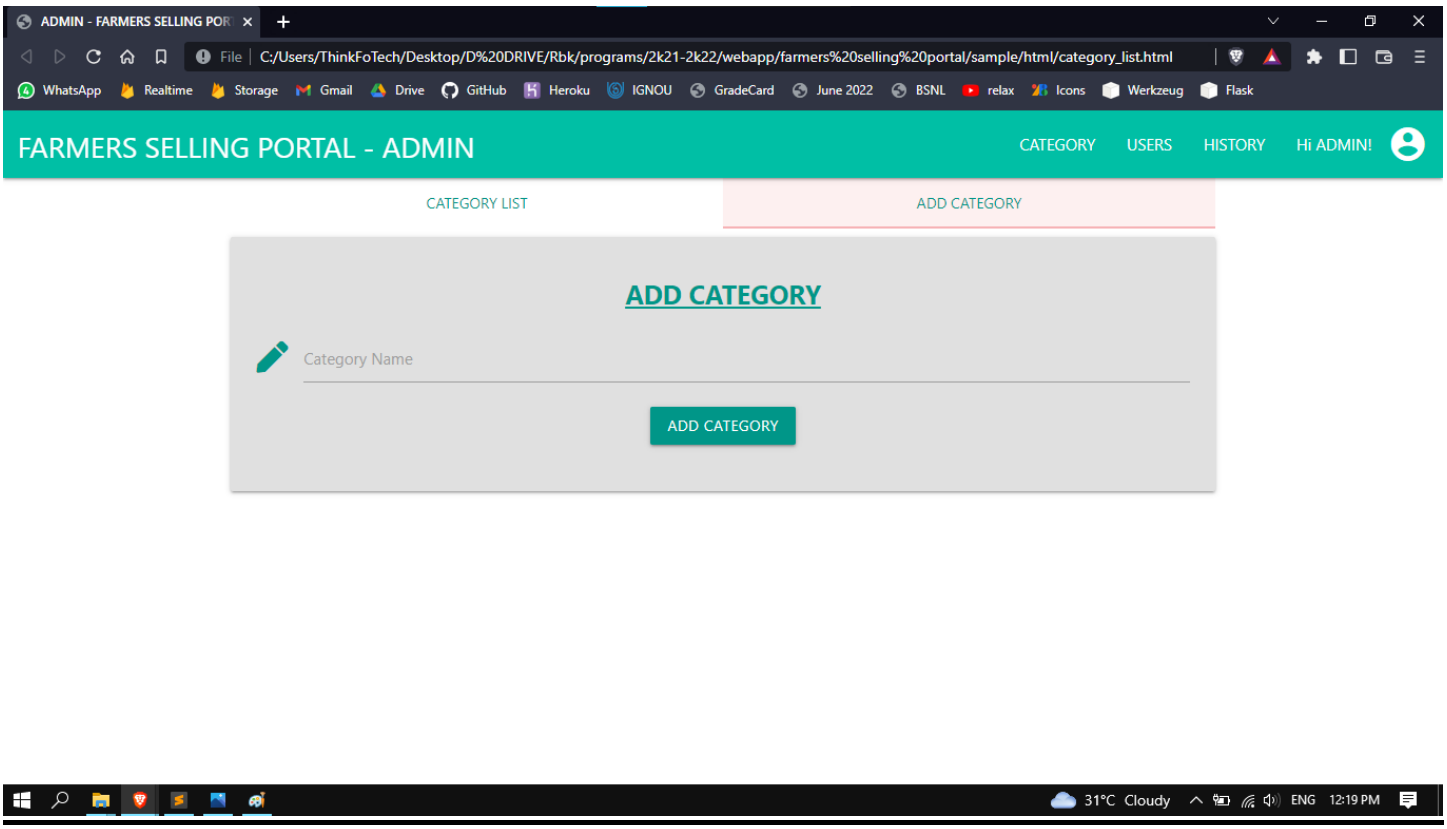
30.ADMINHOME PAGE(NOT APPROVED PRODUCTS)



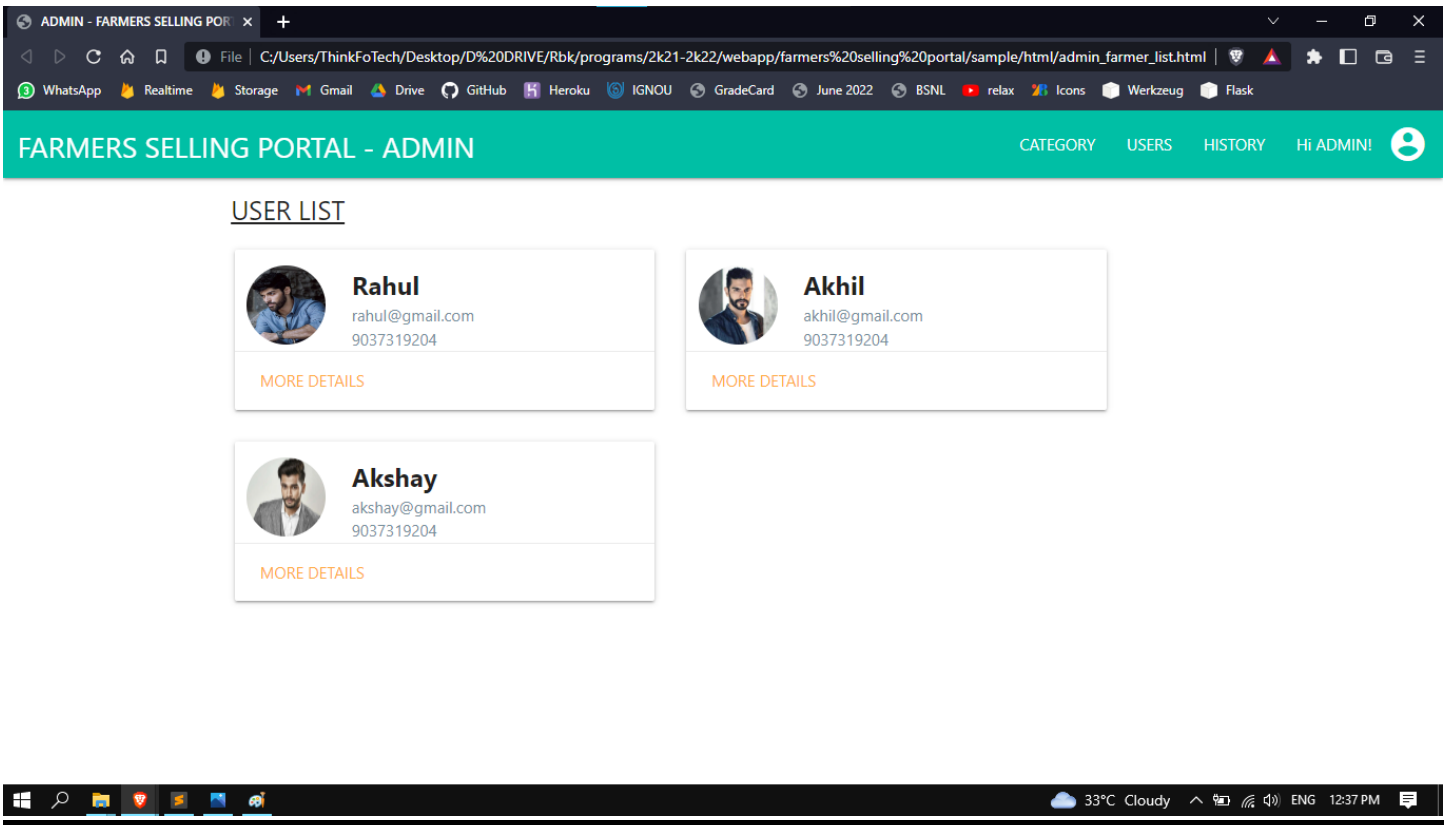
31.ADMIN CATEGORY LIST



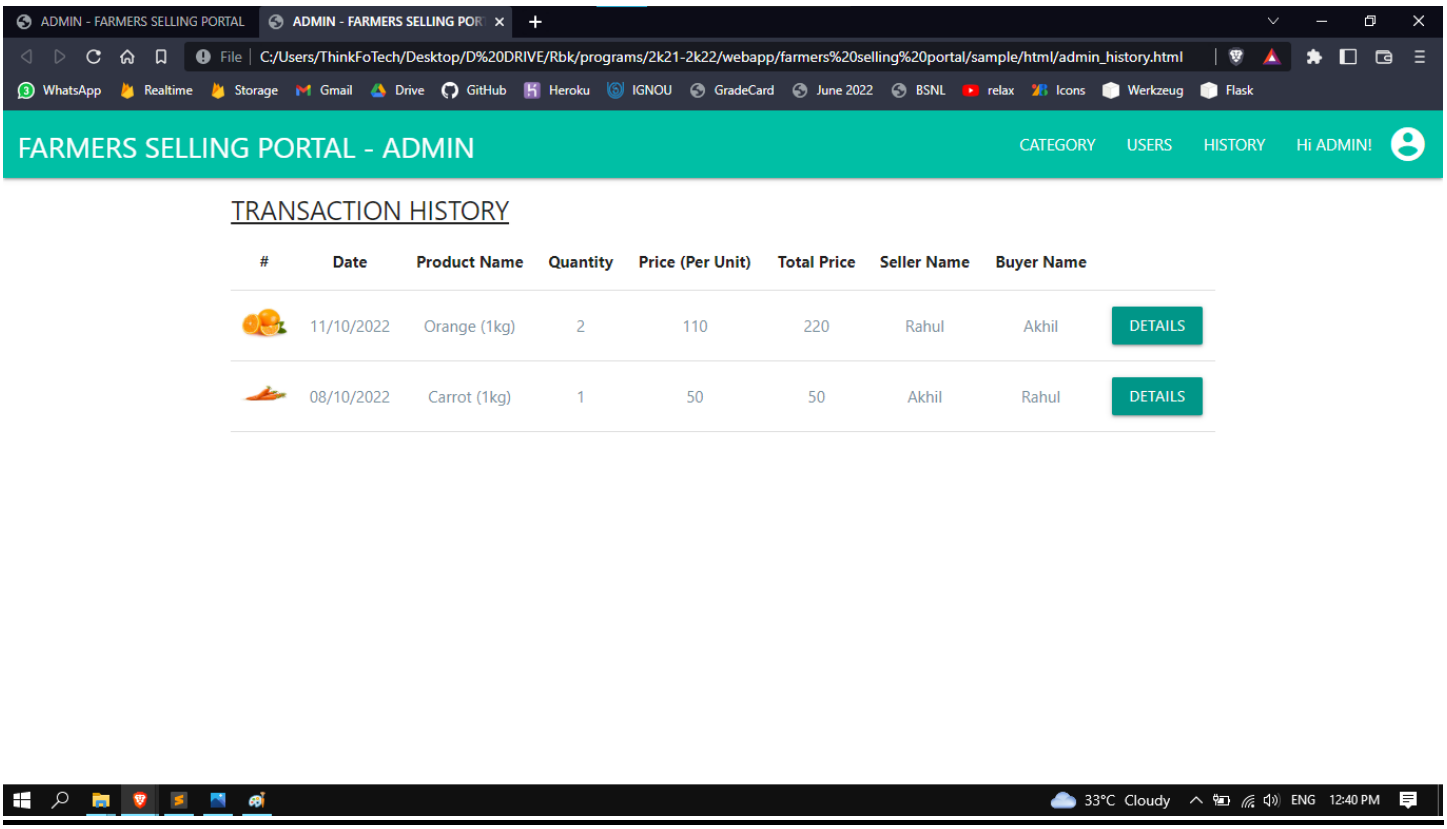
32.ADMIN ADD CATEGORY PAGE



33.ADMIN FARMER LIST



34.ADMIN TRASACTION HISTORY



FUTURE APPLICATION

This Portal can be further upgraded adding more features, where different sellers can sell their agriculture related products such as fertilizers and agriculture tools, etc. New feature like multi language alos will be there. A more advanced way of this portal can be made by providing farmers to sell their products to other countries. Also another feature may contain that the buyer can see the location of farmer product.

LIMITATIONS OF THE PROJECT

- Farmers who don’t understand English will have problem
- Once the order is placed there will not be a cancellation
- Farmers need to get the admin permission
- There will be competition between farmers

Reference

Websites

http://www.google.com/

http://www.wikipedia.com/

Books

1 Python Cookbook: Recipes for Mastering Python 3